RAYTRACERBENCH

A Literate Program

The complete source, explained in reading order

LiterateP · assembled from README.md and twelve chapters

RayTracerBench, a Literate Program

This directory is a *literate programming* account of the RayTracerBench source tree, in the sense Knuth meant the term: the program is presented as an essay written for a human reader, in an order chosen for understanding rather than the order a compiler needs, with the real source code woven directly into the prose rather than summarized from a distance.

It is not a tangle/weave system — there is no tool here that regenerates `RayTracerBench/` from these files, and these files are not compiled. What carries over from the original idea is the *discipline*: every chapter below picks a real, runnable slice of the program, quotes the actual code (with `file:line` citations back to the source of truth), and explains not just what each piece does but why it is written the way it is — the constraint, the alternative that was rejected, or the bug that shaped the final form. Where the "why" is already on record (in `CLAUDE.md`'s project-status notes, in commit history, in the existing `Docs/RayTracerBench-Theory-and-Code` narrative), these chapters cite it rather than re-deriving it from guesswork.

This is a *different* document from `Docs/RayTracerBench-Theory-and-Code`, which is written for a reader who wants the ray-tracing theory first and the engineering second. This one is organized module-by-module, follows the code's own structure, and is meant to be read alongside the source files open in an editor.

Conventions used in every chapter

- Sections are numbered `§1`, `§2`, … **local to each chapter** — think of each chapter as its own fascicle, not one continuously-numbered book.
- A section is prose first, code second: a paragraph motivating a design decision, followed by the fenced code excerpt that embodies it, cited as `path/to/file.ext:startLine–endLine`.
- Code is quoted verbatim from the real files. Nothing here is reconstructed or paraphrased into pseudocode; if a snippet is elided for length, that is marked explicitly (`/ ... /` with a note).
- Each chapter opens with a one-paragraph abstract and a short list of the files it covers, and closes with a "Where this connects" pointer to the neighboring chapters — the same way one section of a real program refers forward and backward to the sections that use or are used by it.
- These files are documentation only. They never modify anything under `RayTracerBench/` or `RayTracerBenchTests/`.

Reading order

The chapters are ordered the way a reader building a mental model from scratch would want them — data model first, then the algorithm shared by both renderers, then each renderer, then the surrounding application, then verification — rather than alphabetically or by directory.

#	Chapter	Modules covered
1	The Core Data Model	`Core/ShaderTypes.h`, `Core/Scene.hpp/.cpp`
2	The Shared Ray-Tracing Core	`Core/RayTraceCore.h`
3	The Metal Shaders	`Shaders/Raytracer.metal`, `Shaders/Blit.metal`
4	The CPU Renderer	`CPU/CPURenderer.hpp/.cpp`
5	The GPU Renderer	`GPU/GPURenderer.hpp/.cpp`
6	Mesh Generation for Export	`Export/EntityMesh.hpp/.cpp`

#	Chapter	Modules covered
7	Scene Export: glTF, OBJ, and Preview Images	`Export/SceneExporter.hpp/.cpp`, `Export/ImageWriter.hpp/.cpp`
8	The App Shell and Lifecycle	`main.cpp`, `App/AppDelegate.hpp/.cpp`
9	UI Controls and Results	`App/ControlsPanel.hpp/.cpp`, `App/ResultsPanel.hpp/.cpp`, `App/AboutAlert.hpp/.cpp`
10	The Metal-Backed Image View, and Filling AppKit's Gaps	`App/ImageDisplayView.hpp/.cpp`, plus the project-local `ThirdParty/metal-cpp-extensions` additions it depends on
11	Tests: Parity, Geometry, and the Core Itself	`RayTracerBenchTests/*`
12	The Build System	`CMakeLists.txt`

What this program is, in one paragraph

RayTracerBench renders the same scene with a CPU ray tracer and a GPU (Metal compute) ray tracer and compares them. The one hard constraint that shapes nearly every file in this list is that Metal Shading Language forbids virtual functions, RTTI, and dynamic allocation — so both renderers share one address-space-parameterized, tagged-struct-and-switch core (`Core/RayTraceCore.h`), compiled twice: once as plain C++ for the CPU path, once as MSL for the GPU kernel. Chapter 2 is the heart of the whole program; chapters 1, 3, 4, and 5 all exist to feed it data or execute it. Chapters 6–10 are the surrounding application — exporting the scene to standard 3D formats, and the AppKit UI, built without Objective-C++ by filling gaps in Apple's `metal-cpp-extensions` bindings by hand. Chapter 11 is how all of the above was checked, including a real Monte-Carlo-variance investigation that overturned an initial pixel-tolerance assumption.

Chapter 1: The Core Data Model

Abstract. Before a single ray is traced, RayTracerBench has to answer a quieter question: what *is* a scene, as data? The answer has to satisfy two masters at once — the CPU renderer, which is ordinary C++17, and the GPU renderer, whose kernel is compiled from the exact same struct definitions as Metal Shading Language, a language that forbids virtual functions, RTTI, and dynamic allocation. This chapter covers the plain structs in `Core/ShaderTypes.h` that make that dual life possible, and `Core/Scene.hpp` and `Core/Scene.cpp`, which use those structs to assemble the classic "Ray Tracing in One Weekend" demo scene — deterministically, from a single seed — as flat, parallel component arrays rather than a tree of shape objects.

Files covered: `Core/ShaderTypes.h`, `Core/Scene.hpp`, `Core/Scene.cpp`.

---

§1. Why the data has to be plain

Every struct in `ShaderTypes.h` looks almost aggressively simple: no constructors beyond aggregate initialization, no methods, no inheritance, no `std::` containers, nothing but `int`, `float`, and `simd_float3` fields. That plainness is not a style preference — it is the price of admission for a struct that has to be `#include`d, byte-for-byte identical, into both a plain C++ translation unit and an MSL kernel:

```
// RayTracerBench/Core/ShaderTypes.h:1–9
#pragma once

// Plain, byte-for-byte-shared data passed between the CPU renderer and the
// GPU kernel's argument buffers. Included verbatim from both plain C++
// (CPURenderer, Scene, GPURenderer) and Raytracer.metal — <simd/simd.h>
// types keep layout identical on both sides.

#include <simd/simd.h>
```

The choice of `<simd/simd.h>` rather than a hand-rolled `Vec3` class is the same idea applied to vectors specifically. Apple's `simd_float3` is one of the few vector types whose memory layout, alignment, and (crucially) whose *header* both a C++17 compiler and Apple's Metal compiler agree on natively. A hand-rolled `struct Vec3 { float x, y, z; }` with operator overloads would work fine on the CPU side, but there would be no guarantee its layout matches whatever MSL's `float3` produces, and every operator would have to be reimplemented (or macro-shimmed) for MSL syntax. Using `simd_float3` directly sidesteps

the whole question: the same eight-line file, unmodified, is a valid header on both sides of the CPU/GPU boundary. This is the foundation the rest of the shared-core design in Chapter 2 is built on top of — `Core/RayTraceCore.h`'s dual compilation only works because the structs it operates on already agree byte-for-byte.

§2. Materials: the first tagged struct

`MaterialGPU` is the simplest of the shared structs, and it previews the pattern the rest of the chapter is really about:

```
// RayTracerBench/Core/ShaderTypes.h:10–23
enum MaterialType
{
        MAT_LAMBERTIAN = 0,
        MAT_METAL      = 1,
        MAT_DIELECTRIC = 2,
};

struct MaterialGPU
{
        int         type; // MaterialType
        simd_float3 albedo;
        float       fuzz; // metal roughness in [0,1]; unused for other types
        float       ir;   // dielectric index of refraction; unused for other types
};
```

In an object-oriented raytracer — including the CUDA port this project takes structural precedent from — this would ordinarily be three classes under a polymorphic `Material` base, each overriding a `scatter()` virtual method. Here it's one struct with an integer tag and every field any material kind might need, with a comment noting which fields a given `type` simply ignores. `fuzz` means nothing for a lambertian material; `ir` means nothing for a metal. That waste (a few unused floats per material) is the deliberate trade for the thing MSL actually requires: no virtual dispatch. Chapter 2's `scatter()` function reads this `type` field and switches on it, the same shape of solution `ShapeGPU` below generalizes to geometry.

§3. Geometry as components, not a class hierarchy

The comment directly above `ShapeType` in `ShaderTypes.h` states the project's central architectural decision as plainly as it will be stated anywhere:

```
// RayTracerBench/Core/ShaderTypes.h:25-34
// Geometry is organized ECS-style: an entity is just an array index into
SceneDescription's
// parallel `transforms`/`shapes` component arrays (see Scene.hpp) rather than a
polymorphic
// "Sphere : Hittable" object — the same tagged-struct idea MaterialGPU/MaterialType
already uses
// for materials, generalized to shapes. See CLAUDE.md for the full rationale.

enum ShapeType
{
        SHAPE_SPHERE  = 0,
        SHAPE_PYRAMID = 1,
};
```

"Ray Tracing in One Weekend" and its CUDA port both model a scene as a list of `Hittable` *— spheres (and whatever else) behind a common virtual interface, with* `hit()` *dispatched dynamically per object. That design is a non-starter for a Metal kernel: MSL simply does not have virtual functions, so a* `Hittable` *base class could never compile as a kernel argument type in the first place. Rather than maintaining two divergent geometry representations — a polymorphic one for the CPU and a flat one for the GPU — RayTracerBench uses one representation everywhere, built out of the same tagged-struct-plus-switch idiom that* `MaterialGPU` *already established for materials. An "entity" here has no class of its own; it is nothing but a shared index into a pair of parallel arrays, one holding each entity's* Transform *component and one holding its* Shape *component — an Entity-Component-System (ECS) layout in miniature. Adding a third primitive later would mean adding one more* `SHAPE_` tag and one more switch case in Chapter 2's `hitEntity()`, not a new subclass and not a new vtable.

§4. The Transform component and its orientation basis

Every entity, regardless of what shape it is, carries a `TransformGPU`:

```
// RayTracerBench/Core/ShaderTypes.h:36-47
// Transform component, shared by every entity regardless of shape. `position` is the
entity's
// local origin (a sphere's center; a pyramid's base-square center).
`right`/`up`/`forward` are an
// orthonormal orientation basis — the identity basis for spheres (rotationally symmetric,
so
// orientation is meaningless for them) and an arbitrary rotation for pyramids, whose `up`
axis
// points from base to apex.
struct TransformGPU
{
        simd_float3 position;
        simd_float3 right;
        simd_float3 up;
        simd_float3 forward;
};
```

The orientation is stored as three explicit basis vectors rather than as a 3×3 rotation matrix or a quaternion. A matrix would carry the same information more compactly in one sense, and a quaternion would avoid any risk of the three vectors drifting out of orthonormality — but both would need supporting machinery (matrix inversion, quaternion-to-basis conversion) that this project would then have to implement twice, once for plain C++ and once in a dialect MSL is willing to compile. Storing `right` / `up` / `forward` directly as three already-orthonormal vectors buys something specific for Chapter 2's ray/shape intersection code: because the inverse of an orthogonal matrix is just its transpose, projecting an incoming ray's direction onto each of these three basis vectors *is* already the inverse rotation — transforming a world-space ray into the entity's local space needs nothing more than three dot products, and no matrix inverse routine has to exist anywhere in this codebase. That consuming code lives in `Core/RayTraceCore.h` and is Chapter 2's subject, not this chapter's, but it's worth naming here because it's the reason this particular representation was chosen over the more familiar alternatives.

For a sphere, orientation is physically meaningless — a sphere looks identical under any rotation — so `Scene.cpp` (§8 below) just gives every sphere the plain identity basis and it is never read by sphere intersection. Pyramids are the shape that actually exercises this field.

§5. The Shape component as a tagged union

`ShapeGPU` plays the same role for geometry that `MaterialGPU` plays for shading — a fixed-size struct wide enough to hold whichever primitive's parameters are needed, plus a reference to another component array:

```
// RayTracerBench/Core/ShaderTypes.h:49-60
// Shape component: a tagged union of per-primitive geometry parameters plus a reference
to this
// entity's Material component. Adding a new primitive means adding one more SHAPE_* tag
and one
// more field group here, dispatched by a switch (see RayTraceCore.h's hitEntity()) rather
than a
// new virtual base class — virtual dispatch is forbidden in MSL kernel code.
struct ShapeGPU
{
        int   type;          // ShapeType
        float radius;        // sphere: radius. pyramid: unused.
        float baseHalfWidth; // pyramid: half the base square's side length. sphere:
unused.
        float height;        // pyramid: apex height above the base. sphere: unused.
        int   materialIndex; // index into SceneDescription::materials
};
```

`materialIndex` is the field that makes this a genuine component system rather than just a struct-of-parameters: it is not a copy of a material, it is a reference into a separate `materials` array, so many entities — the whole randomized field of small spheres, for instance — can point at distinct material records while entities that happen to share a look (unlikely here, since materials are randomized per-entity, but architecturally possible) could share one. It is exactly the same "index into another array" idea that `transforms` / `shapes` themselves use to relate to an entity, just one level removed: entity → shape → material.

§6. Camera and per-frame parameters

Two more structs round out `ShaderTypes.h`, and neither needs the tagged-union treatment because there is only ever one of each per render: `CameraGPU` holds a fully precomputed viewport (origin, corners, basis vectors, lens radius) —

```
// RayTracerBench/Core/ShaderTypes.h:62-72
struct CameraGPU
{
        simd_float3 origin;
        simd_float3 lowerLeftCorner;
        simd_float3 horizontal;
        simd_float3 vertical;
        simd_float3 u;
        simd_float3 v;
        simd_float3 w;
        float       lensRadius;
};
```

— and `RenderParams` holds the handful of scalars that describe *how* to render rather than *what* to render (image dimensions, sample count, bounce depth, and the per-frame RNG seed):

```
// RayTracerBench/Core/ShaderTypes.h:74–81
struct RenderParams
{
        uint32_t width;
        uint32_t height;
        uint32_t samplesPerPixel;
        uint32_t maxDepth;
        uint32_t frameSeed;
};
```

Precomputing the camera's viewport geometry into `CameraGPU` here, rather than recomputing it from `vfov` / `aspectRatio` /etc. inside the per-pixel kernel, means Chapter 2's `getRay()` does no trigonometry per ray — it only needs to do vector arithmetic on values `Scene.cpp`'s `makeCamera()` (§7) already worked out once, on the host, when the scene was built.

§7. `SceneDescription`: the component arrays, assembled

`Scene.hpp` ties the individual structs above together into one object that represents an entire scene:

```
// RayTracerBench/Core/Scene.hpp:7–20
// ECS-style layout: an entity is simply an array index — transforms[i]/shapes[i] are that
// entity's Transform and Shape components (see ShaderTypes.h). There is no separate list
per
// shape type; spheres and pyramids are interleaved in the same two parallel arrays and
// distinguished only by ShapeGPU::type, exactly like RayTraceCore.h's hitEntity()
dispatches on it.
// Each entity's material is itself a component reference (ShapeGPU::materialIndex) into
the
// separate `materials` component array, so many entities can share one material record.
struct SceneDescription
{
        std::vector<TransformGPU> transforms;
        std::vector<ShapeGPU>     shapes;
        std::vector<MaterialGPU>  materials;
        CameraGPU                 camera;
        RenderParams              params;
};
```

Note what is *not* here: there is no `std::vector<Sphere>` alongside a `std::vector<Pyramid>`. Spheres and pyramids are interleaved in the same `transforms` / `shapes` pair, in whatever

order they were added, and the only thing that tells them apart at render time is `shapes[i].type`. This is the single representation both renderers share — `SceneDescription` is not subclassed or specialized for the CPU or the GPU; the GPU renderer simply uploads these same three vectors into Metal buffers, and the CPU renderer walks them with raw pointers. There is no separate CPU/GPU scene representation to keep in sync.

`Scene.hpp` also declares the one function that builds a `SceneDescription`, `buildDefaultScene()`, with a doc comment that lays out its full contract — determinism from a seed, the shape of the classic demo scene, and the `floating` parameter this chapter returns to in §11:

```
// RayTracerBench/Core/Scene.hpp:22-34
// Builds the classic "Ray Tracing in One Weekend" demo scene — a ground sphere, a
randomized
// ~22x22 grid of small spheres, three large feature spheres (glass / lambertian / metal),
and a
// handful of square pyramids at varied 3D orientations — deterministically for a given
seed via a
// seeded std::mt19937. Both the CPU and GPU renderers consume this exact same
SceneDescription, so
// identical seeds produce identical entity layouts/materials on both (per-pixel noise
still
// differs — see CLAUDE.md's verification notes).
//
// `floating`, when true, places the small randomized-field spheres at a random height in
// [radius, kMaxFloatHeight] (see Scene.cpp) instead of resting on the ground —
kMaxFloatHeight was
// chosen and verified by rendering the fixed camera setup below and confirming nothing
clips out
// of frame, not derived from an unverified formula. The three large feature spheres
(glass /
// lambertian / metal) and the pyramids always rest on the ground regardless of
`floating`.
SceneDescription buildDefaultScene( unsigned seed, uint32_t width, float aspectRatio,
uint32_t samplesPerPixel, uint32_t maxDepth, bool floating = false );
```

§8. `Scene.cpp` is host-only, and gets to use the full `simd` API

Everything from here on is `Scene.cpp`, and its opening comment is worth reading before any of its code, because it explains a small but easily mixed-up asymmetry with Chapter 2's `RayTraceCore.h`:

```
// RayTracerBench/Core/Scene.cpp:7-9
// Scene.cpp is pure host-side C++ (never compiled as MSL), so — unlike
Core/RayTraceCore.h —
// it can freely use the full simd_* C API (simd_make_float3, simd_normalize, simd_cross,
// simd_length) instead of hand-rolled vector helpers.
```

`Core/RayTraceCore.h` (Chapter 2) is `#include`d verbatim into an MSL translation unit, and Apple's Metal compiler does not accept function-call-style construction like `simd_float3(x, y, z)` — only brace-initialization or `simd_make_float3` compile under MSL, even though a plain C++ compiler is happy to accept the constructor-style call too. Rather than rely on that inconsistency case by case, `RayTraceCore.h` defines its own small `makeFloat3()` helper and uses it everywhere a 3-vector literal is needed, so the exact same line of source is guaranteed to compile identically under both toolchains. `Scene.cpp`, by contrast, is compiled exactly once, only ever by a normal C++17 compiler — it is part of the framework-free `RayTracerCore` library and never touched by `xcrun -sdk macosx metal` — so it has no reason to route around anything. It calls `simd_make_float3`, `simd_normalize`, `simd_cross`, and `simd_length` straight from `<simd/simd.h>`'s own C API throughout, which is exactly what the code excerpts below show it doing.

§9. `addEntity()`: spawning an entity is pushing two components

The whole ECS pattern collapses to one small function once you have the parallel arrays already declared:

```
// RayTracerBench/Core/Scene.cpp:55-62
// Spawns one entity: pushes its Transform and Shape components onto SceneDescription's
// parallel component arrays at the same index — the ECS analogue of "instantiate an
entity
// with these components" (see ShaderTypes.h/Scene.hpp for the component layout).
void addEntity( SceneDescription& scene, TransformGPU transform, ShapeGPU shape )
{
        scene.transforms.push_back( transform );
        scene.shapes.push_back( shape );
}
```

There is no entity object returned, no handle, nothing to hold onto — the entity's identity *is* its position in these two vectors, which is exactly why `transforms` and `shapes` must always be pushed to together, in lock step, and never independently. `addMaterial()` just above it in the file follows the identical shape for the third component array:

```
// RayTracerBench/Core/Scene.cpp:48-53
// Appends a material to the materials component array and returns its index.
int addMaterial( std::vector<MaterialGPU>& materials, MaterialGPU mat )
{
        materials.push_back( mat );
        return static_cast<int>( materials.size() - 1 );
}
```

— except that this one *does* return something, the freshly-assigned index, because unlike a Transform/Shape pair a material is meant to be referenced by index from elsewhere (`ShapeGPU::materialIndex`), not just spawned.

§10. Building the scene: ground, field, and the three feature spheres

`buildDefaultScene()` follows the book's own scene almost line for line. It seeds one `std::mt19937` from the caller's `seed` and draws everything — material choices, colors, jitter, and (per §11) placement heights — from that single generator, which is what makes the whole scene reproducible from a seed alone. The ground is a giant sphere pushed far below the visible area:

```
// RayTracerBench/Core/Scene.cpp:163-169
        // Ground.
        {
                MaterialGPU ground{ MAT_LAMBERTIAN, simd_make_float3( 0.5f, 0.5f, 0.5f ),
0.0f, 0.0f };
                int         groundMat = addMaterial( scene.materials, ground );
                addEntity( scene, identityTransformAt( simd_make_float3( 0.0f, -1000.0f,
0.0f ) ),
                        ShapeGPU{ SHAPE_SPHERE, 1000.0f, 0.0f, 0.0f, groundMat } );
        }
```

Then the randomized ~22×22 field of small spheres, each independently rolling a material (80% lambertian, 15% metal, 5% dielectric, matching the book's own split) and skipping the cell nearest the large feature spheres so nothing overlaps them:

```
// RayTracerBench/Core/Scene.cpp:173–205
        for ( int a = -11; a < 11; ++a )
        {
                for ( int b = -11; b < 11; ++b )
                {
                        float       chooseMat = unit( rng );
                        simd_float3 center = simd_make_float3( a + 0.9f * unit( rng ),
0.2f, b + 0.9f * unit( rng ) );

                        if ( simd_length( center - featureSphereCenter ) <= 0.9f )
                                continue;

                        MaterialGPU mat{};
                        if ( chooseMat < 0.8f )
                        {
                                mat.type = MAT_LAMBERTIAN;
                                mat.albedo = randomAlbedo( rng, unit );
                        }
                        else if ( chooseMat < 0.95f )
                        {
                                mat.type = MAT_METAL;
                                mat.albedo = simd_make_float3( unit( rng ) * 0.5f + 0.5f,
unit( rng ) * 0.5f + 0.5f, unit( rng ) * 0.5f + 0.5f );
                                mat.fuzz = fuzzDist( rng );
                        }
                        else
                        {
                                mat.type = MAT_DIELECTRIC;
                                mat.ir = 1.5f;
                        }

                        int matIndex = addMaterial( scene.materials, mat );
                        center.y = placementHeight( floating, 0.2f, center.y, rng, unit );
                        addEntity( scene, identityTransformAt( center ), ShapeGPU{
SHAPE_SPHERE, 0.2f, 0.0f, 0.0f, matIndex } );
                }
        }
```

and, always resting on the ground regardless of `floating`, the three large feature spheres — one of each material, the classic RTIOW hero shot:

```
// RayTracerBench/Core/Scene.cpp:207–222
        // Three large feature spheres: glass, lambertian, metal. Always resting on the
ground,
        // regardless of `floating` — only the small randomized field floats.
        {
                MaterialGPU glass{ MAT_DIELECTRIC, simd_make_float3( 0.0f, 0.0f, 0.0f ),
0.0f, 1.5f };
                int         glassMat = addMaterial( scene.materials, glass );
                addEntity( scene, identityTransformAt( simd_make_float3( 0.0f, 1.0f, 0.0f
) ), ShapeGPU{ SHAPE_SPHERE, 1.0f, 0.0f, 0.0f, glassMat } );

                MaterialGPU diffuse{ MAT_LAMBERTIAN, simd_make_float3( 0.4f, 0.2f, 0.1f ),
0.0f, 0.0f };
                int         diffuseMat = addMaterial( scene.materials, diffuse );
                addEntity( scene, identityTransformAt( simd_make_float3( -4.0f, 1.0f, 0.0f
) ),
                        ShapeGPU{ SHAPE_SPHERE, 1.0f, 0.0f, 0.0f, diffuseMat } );

                MaterialGPU metal{ MAT_METAL, simd_make_float3( 0.7f, 0.6f, 0.5f ), 0.0f,
0.0f };
                int         metalMat = addMaterial( scene.materials, metal );
                addEntity( scene, identityTransformAt( simd_make_float3( 4.0f, 1.0f, 0.0f
) ), ShapeGPU{ SHAPE_SPHERE, 1.0f, 0.0f, 0.0f, metalMat } );
        }
```

Every sphere in both the ground/field/feature groups is placed with `identityTransformAt()`, the small helper mentioned in §4 that fills `right` / `up` / `forward` with the identity basis because a sphere has no orientation to speak of:

```
// RayTracerBench/Core/Scene.cpp:64–74
        // Spheres are rotationally symmetric, so their Transform component only ever uses
`position`;
        // `right`/`up`/`forward` are set to the identity basis and simply ignored by
hitSphere().
        TransformGPU identityTransformAt( simd_float3 position )
        {
                TransformGPU t;
                t.position = position;
                t.right = simd_make_float3( 1.0f, 0.0f, 0.0f );
                t.up = simd_make_float3( 0.0f, 1.0f, 0.0f );
                t.forward = simd_make_float3( 0.0f, 0.0f, 1.0f );
                return t;
        }
```

§11. `floating`, and why its RNG draws are ordered so carefully

The `floating` parameter is the one piece of `buildDefaultScene()`'s behavior that postdates the base scene, and its implementation is written specifically to avoid perturbing every scene

that was already relying on the old, ground-only layout for a given seed. `placementHeight()` is the small function that decides, per small-field sphere, where its `y` actually ends up:

```
// RayTracerBench/Core/Scene.cpp:76-89
        // Chosen and verified by rendering the fixed camera setup below (lookfrom
(13,2,3), lookat
        // origin, vfov 20°) at several candidate heights and confirming by eye that
nothing floats out
        // of frame — not derived from an unverified formula. Applies only to the small
randomized-field
        // spheres; the three large feature spheres always rest on the ground regardless
of `floating`.
        constexpr float kMaxFloatHeight = 3.0f;

        // Picks a random height in [radius, kMaxFloatHeight] when floating, or the given
resting
        // height (touching the ground) otherwise.
        float placementHeight( bool floating, float radius, float restingHeight,
std::mt19937& rng, std::uniform_real_distribution<float>& unit )
        {
                if ( !floating )
                        return restingHeight;
                return radius + unit( rng ) * ( kMaxFloatHeight - radius );
        }
```

The critical detail is *where* this function is called relative to the RNG draws around it, back at the call site in the field loop: `center.y = placementHeight( floating, 0.2f, center.y, rng, unit );` runs *after* `chooseMat`, the albedo/fuzz draws, and the `x`/`z` jitter have already all been consumed for that cell — and, in the `floating == false` branch, `placementHeight()` makes no `unit(rng)` call at all. That ordering is what lets the doc comment in `Scene.hpp` promise "identical seeds produce identical entity layouts" as a *strict* guarantee for existing callers: with `floating` defaulting to `false`, the sequence of RNG draws `buildDefaultScene()` makes for a given seed is byte-for-byte the same sequence it made before `floating` existed, so nothing already depending on a seed's exact layout (rendered images, saved exports, tests) shifts underneath it. Only when a caller opts into `floating = true` does the extra `unit(rng)` draw for height get inserted — and only ever for the small field spheres, never for the ground, the three feature spheres, or the pyramids, which are always built by the unconditional resting-position paths in §12.

`kMaxFloatHeight`'s value of `3.0f` is worth pausing on for what it is *not*: it was not derived by working out the camera frustum's bounds analytically and solving for the height at which an object would just start clipping out of frame. It was chosen empirically — render the fixed `lookfrom (13,2,3)` / `lookat origin` / `vfov 20°` camera at a few candidate heights and

look at the result. Section §12's pyramid placement takes the opposite approach for a structurally similar problem, and the contrast is the more instructive story.

§12. Pyramids: an orientation that means something, and a resting height with an exact answer

Spheres cannot demonstrate `TransformGPU`'s `right` / `up` / `forward` basis doing anything, since a sphere is invariant under rotation. Pyramids are added specifically because they can:

```
// RayTracerBench/Core/Scene.cpp:224–230
        // Square pyramids, at a spread of 3D orientations — the shape group that actually
exercises
        // TransformGPU's orientation basis, since a sphere looks identical no matter how
it's rotated.
        // Always resting on the ground (via makeRestingPyramidTransform's exact per-
orientation
        // computation above), never affected by `floating`, and never dielectric:
hitPyramidLocal only
        // resolves the ray's entry face, so a ray whose origin starts inside the solid —
which glass
        // refraction requires once a ray exits the far side — isn't handled. That's an
explicit,
        // documented scope limit for this primitive, not an oversight.
```

`makeOrientedBasis()` builds each pyramid's basis from two independent rotations — a tilt around the local Z axis away from vertical, then a yaw around world Y — deliberately so the five pyramids in the demo scene can show genuinely different 3D poses (some merely spun in place, some visibly tipped over) rather than variations on a single spin axis:

```
// RayTracerBench/Core/Scene.cpp:91-114
        // Builds an orthonormal orientation basis for a pyramid: tilt the identity "up"
axis away from
        // vertical by `tiltDegrees` around the local Z axis, then yaw the whole basis
around world Y by
        // `yawDegrees` — two independent rotations so pyramids can show genuinely
different 3D
        // orientations (some merely spun in place, some visibly tipped over), not just a
spin around
        // one axis.
        void makeOrientedBasis( float yawDegrees, float tiltDegrees, simd_float3&
outRight, simd_float3& outUp, simd_float3& outForward )
        {
                float tilt = tiltDegrees * kPi / 180.0f;
                simd_float3 up = simd_make_float3( std::sin( tilt ), std::cos( tilt ),
0.0f );
                simd_float3 right = simd_make_float3( std::cos( tilt ), -std::sin( tilt ),
0.0f );
                simd_float3 forward = simd_make_float3( 0.0f, 0.0f, 1.0f );

                float yaw = yawDegrees * kPi / 180.0f;
                float cosY = std::cos( yaw );
                float sinY = std::sin( yaw );

                auto rotateY = [ cosY, sinY ]( simd_float3 v ) {
                        return simd_make_float3( v.x * cosY + v.z * sinY, v.y, -v.x * sinY
+ v.z * cosY );
                };

                outRight = rotateY( right );
                outUp = rotateY( up );
                outForward = rotateY( forward );
        }
```

Once a pyramid can be tilted at an arbitrary angle, "resting on the ground" stops being as trivial as setting `position.y` to a constant — a tilted pyramid's lowest point might be the apex, or it might be one of the four base corners, depending on the tilt. Unlike `kMaxFloatHeight`'s frustum question in §11, *this* geometric question does have a closed form: a pyramid has exactly five vertices in local space (the apex plus four base corners), and once you know the orientation basis, the world-space height of each is just a dot product against `up`. The lowest of those five numbers is the exact amount the pyramid needs to be lifted (or dropped) so it touches the ground precisely, at any orientation, with no iteration and no eyeballing:

```
// RayTracerBench/Core/Scene.cpp:116-149
        // Places a pyramid's Transform so its lowest vertex (the apex or one of the 4
base corners,
        // whichever ends up lowest once rotated) touches `groundY` exactly, regardless of
orientation —
        // an exact closed-form computation from the 5 local vertices, unlike
kMaxFloatHeight above
        // (which needed a rendered check against the camera frustum and couldn't be
solved in closed
        // form). This is what lets every pyramid below rest flush on the ground at any
tilt without a
        // per-orientation eyeball check.
        TransformGPU makeRestingPyramidTransform( simd_float3 groundXZ, float groundY,
float yawDegrees, float tiltDegrees,
                float baseHalfWidth, float height )
        {
                simd_float3 right, up, forward;
                makeOrientedBasis( yawDegrees, tiltDegrees, right, up, forward );

                const simd_float3 localVerts[ 5 ] = {
                        simd_make_float3( 0.0f, height, 0.0f ),
                        simd_make_float3( baseHalfWidth, 0.0f, baseHalfWidth ),
                        simd_make_float3( baseHalfWidth, 0.0f, -baseHalfWidth ),
                        simd_make_float3( -baseHalfWidth, 0.0f, -baseHalfWidth ),
                        simd_make_float3( -baseHalfWidth, 0.0f, baseHalfWidth ),
                };

                float minWorldY = 1.0e30f;
                for ( const simd_float3& v : localVerts )
                {
                        float worldY = v.x * right.y + v.y * up.y + v.z * forward.y;
                        minWorldY = std::min( minWorldY, worldY );
                }

                TransformGPU transform;
                transform.position = simd_make_float3( groundXZ.x, groundY - minWorldY,
groundXZ.z );
                transform.right = right;
                transform.up = up;
                transform.forward = forward;
                return transform;
        }
```

`kMaxFloatHeight` and this function are the same chapter's two answers to structurally similar-sounding questions — "how high can something go before it looks wrong" versus "how low does something sit given its orientation" — and the fact that one was solved by rendering-and-looking while the other was solved algebraically is not an inconsistency; it reflects which question actually admits a closed-form answer. The five local vertices these world-space heights are computed from — one apex, four base corners — are exactly the same five points `Core/RayTraceCore.h`'s `hitPyramidLocal()` (Chapter 2) derives its five

half-space planes from, and the same five points Chapter 6's `EntityMesh.hpp` triangulates into a faceted mesh: this one geometric definition of "what a pyramid is" is reused for placement, intersection, and export alike.

With the basis and resting transform in hand, the five demo pyramids are declared as a small table of specs — ground position, yaw, tilt, color, and whether it's metal — and built in a loop, each getting its own material and its `ShapeGPU` filled with `baseHalfWidth` / `height` (its `radius` field left at `0.0f`, unused, exactly as the comment in §5 documents):

```
// RayTracerBench/Core/Scene.cpp:232-259
                struct PyramidSpec
                {
                        simd_float3 groundXZ;
                        float       yawDegrees;
                        float       tiltDegrees;
                        simd_float3 albedo;
                        bool        metal;
                };

                const PyramidSpec pyramids[] = {
                        { simd_make_float3( -2.2f, 0.0f, 1.8f ), 0.0f, 0.0f,
simd_make_float3( 0.85f, 0.2f, 0.2f ), false },
                        { simd_make_float3( 2.2f, 0.0f, 1.8f ), 40.0f, 0.0f,
simd_make_float3( 0.2f, 0.8f, 0.3f ), false },
                        { simd_make_float3( 0.0f, 0.0f, -2.0f ), 20.0f, 25.0f,
simd_make_float3( 0.3f, 0.4f, 0.9f ), false },
                        { simd_make_float3( -4.5f, 0.0f, 2.2f ), 70.0f, -20.0f,
simd_make_float3( 0.75f, 0.75f, 0.2f ), false },
                        { simd_make_float3( 4.5f, 0.0f, 2.2f ), 55.0f, 35.0f,
simd_make_float3( 0.7f, 0.7f, 0.7f ), true },
                };

                const float baseHalfWidth = 0.6f;
                const float pyramidHeight = 1.2f;

                for ( const PyramidSpec& p : pyramids )
                {
                        MaterialGPU mat = p.metal ? MaterialGPU{ MAT_METAL, p.albedo,
0.15f, 0.0f } : MaterialGPU{ MAT_LAMBERTIAN, p.albedo, 0.0f, 0.0f };
                        int         matIndex = addMaterial( scene.materials, mat );

                        TransformGPU transform = makeRestingPyramidTransform( p.groundXZ,
0.0f, p.yawDegrees, p.tiltDegrees, baseHalfWidth, pyramidHeight );
                        addEntity( scene, transform, ShapeGPU{ SHAPE_PYRAMID, 0.0f,
baseHalfWidth, pyramidHeight, matIndex } );
                }
```

Note also that these five specs are fixed, hand-picked constants, not drawn from `rng` — so adding pyramids to the scene draws no extra random numbers at all, which is one more

reason `floating`'s backward-compatibility argument in §11 only had to reason about the field-sphere loop.

§13. Wiring up the camera and finishing the scene

The function's last steps build the one `CameraGPU` the whole scene shares, using the `makeCamera()` helper mentioned in §6, and fill in `RenderParams` from the caller's arguments:

```
// RayTracerBench/Core/Scene.cpp:262–275
        scene.camera = makeCamera(
                simd_make_float3( 13.0f, 2.0f, 3.0f ),
                simd_make_float3( 0.0f, 0.0f, 0.0f ),
                simd_make_float3( 0.0f, 1.0f, 0.0f ),
                20.0f, aspectRatio, 0.1f, 10.0f );

        scene.params.width = width;
        scene.params.height = static_cast<uint32_t>( static_cast<float>( width ) /
aspectRatio );
        scene.params.samplesPerPixel = samplesPerPixel;
        scene.params.maxDepth = maxDepth;
        scene.params.frameSeed = seed;

        return scene;
}
```

`(13, 2, 3)` looking at the origin with a 20° vertical field of view is the exact camera the book itself uses for this scene, and it's also the fixed setup `kMaxFloatHeight` in §11 was verified against by rendering — so that verification and this camera declaration describe the same view, just from two different places in the file. The returned `SceneDescription` at this point is complete and self-contained: every entity's Transform and Shape sit at matching indices in `transforms` / `shapes`, every `materialIndex` points somewhere valid in `materials`, and `camera` / `params` are filled in — ready to be hand to either renderer with nothing further to translate.

Where this connects

Chapter 2, The Shared Ray-Tracing Core, is where these structs stop being passive data. `Core/RayTraceCore.h`'s `hitEntity()` is the direct consumer of `transforms[]` / `shapes[]` walked in lockstep — the tagged-switch dispatch this chapter set up the vocabulary for — and its `hitPyramid()` is where projecting onto `TransformGPU`'s orthonormal basis (§4)

actually pays off as "no matrix inverse needed." That same file's `scatter()` is the tagged dispatch over `MaterialGPU::type` this chapter's §2 previewed.

Chapter 6, Mesh Generation for Export, reuses this exact same Transform + Shape data for a completely different purpose: `Export/EntityMesh.hpp`'s `buildEntityMesh()` is a third system (alongside `hitEntity()`'s ray intersection and `scatter()`'s shading) that dispatches on `ShapeGPU::type` the same way, but produces a triangle mesh instead of a hit test — including building pyramid meshes from the identical five local vertices `makeRestingPyramidTransform()` (§12) already used to figure out where to place them.

Chapter 2: The Shared Ray-Tracing Core

Abstract. This chapter covers `RayTracerBench/Core/RayTraceCore.h`, the single 537-line file that *is* the ray tracer — hit-testing, material scattering, camera-ray generation, and the bounded path-tracing loop that ties them together. Its defining property is not any one algorithm but the fact that it is compiled twice from one source: once as Metal Shading Language, inside `Shaders/Raytracer.metal`'s GPU kernel, and once as plain C++17, inside `CPU/CPURenderer.cpp`. Every design choice in this file — value-returning functions instead of out-parameters, a single `#ifdef __METAL_VERSION__`, hand-rolled vector math instead of library calls — exists to make that dual compilation possible without maintaining two hand-synced copies. As the top-level README puts it, this is "the heart of the whole program"; Chapters 1, 3, 4, and 5 all exist either to feed it data or to execute it.

Files covered: `RayTracerBench/Core/RayTraceCore.h`.

§1. The constraint that shapes every line in this file

Metal Shading Language forbids virtual functions, RTTI, and dynamic allocation inside kernel code. The reference implementations this project is structurally modeled on — "Ray Tracing in One Weekend" and its CUDA port — both lean on a polymorphic `Hittable`/`Material` class hierarchy with virtual `hit()`/`scatter()` methods; CUDA allows that (its device compiler supports virtual dispatch), but MSL does not. Rather than write a CPU renderer with virtual dispatch and a separately-written GPU renderer with tagged-switch dispatch — two implementations that merely *look* similar, and would silently drift the moment someone fixed a bug in only one of them — this project uses tagged structs and switch-based dispatch on **both** sides, and, more importantly, makes both sides execute the *same source file*. The file's own header comment states this plainly:

```
// Dual-compiled: #include'd verbatim by both Shaders/Raytracer.metal and
// CPU/CPURenderer.cpp, so the CPU and GPU renderers execute the exact same
// algorithm source rather than two implementations that merely look similar.
```

`RayTracerBench/Core/RayTraceCore.h:3–5`

"Dual-compiled" is doing real work in that sentence, and it is worth being precise about what it does *not* mean. It does not mean this project keeps two copies of the ray-tracing algorithm — one in a `.metal` file, one in a `.cpp` file — annotated with `// KEEP IN SYNC` comments at the top and bottom, trusting future edits to touch both. That approach was considered and

explicitly rejected as a fallback of last resort: CLAUDE.md's Core section notes it only as "the fallback... if shared compilation proves too fussy," a fallback this project never needed. Instead, `Raytracer.metal` literally `#include`s this header as MSL source, and `CPURenderer.cpp` literally `#include`s the identical header as C++ source (Chapters 3 and 4 show both include sites). The compiler that turns this text into a GPU kernel and the compiler that turns it into CPU machine code are reading character-for-character the same bytes. If `hitSphere()` has a bug, it has that bug on both renderers, simultaneously, by construction — there is no "forgot to update the other copy" failure mode to guard against.

Getting one file to compile as two different, incompatible languages is not free, though — MSL and C++17 disagree about address spaces, about vector-construction syntax, and about which standard-library functions exist. The rest of this chapter is largely the story of how this file papers over those disagreements with the smallest possible number of `#ifdef`s.

§2. One address-space keyword, and the design discipline it forces

MSL requires every pointer that crosses a kernel function's argument boundary to be annotated with an address space — `device`, `constant`, `thread`, or `threadgroup`. Plain C++ has no such concept; a pointer is just a pointer. If this file freely passed pointers or references around — the natural C++ style for an out-parameter like "fill in this `HitRecord` for me" — every one of those parameters would need an address-space annotation on the MSL side, and C++ has no matching keyword to spell for `thread`-space locals. That would force either two divergent function signatures (defeating the entire point of dual-compiling one file) or an even uglier tangle of macros per parameter.

This file avoids that by threading everything through **return values** instead. `RandomFloatSample`, `RandomVec3Sample`, `HitResult`, `ScatterResult`, `RayColorResult`, and `CameraRaySample` are all small structs that exist for exactly this reason: to carry a function's "outputs" — including updated RNG state — back to the caller as a single returned value, rather than through a `HitRecord&` or `uint32_t&` parameter. The header comment names this explicitly:

```
// RNG state and hit/scatter outcomes are threaded through as return values
// (RandomFloatSample, HitResult, ScatterResult, ...) rather than
// out-parameters, precisely so no second address-space keyword is needed.
```

`RayTracerBench/Core/RayTraceCore.h:20–22`

With that discipline in place, exactly *one* address-space keyword ends up needed anywhere in the file, and it shows up in exactly one place: `rayColor()`'s three scene-array pointers, which

must point at the GPU kernel's actual buffer arguments (or, on the CPU, at ordinary heap arrays owned by `SceneDescription` — see Chapter 1). That single keyword is spelled through a macro:

```
#if defined( __METAL_VERSION__ )
        #define RT_DEVICE device
#else
        #include <cmath>
        using std::fabs;
        using std::fmin;
        using std::pow;
        using std::sqrt;
        #define RT_DEVICE
#endif
```

`RayTracerBench/Core/RayTraceCore.h:26–35`

`__METAL_VERSION__` is defined automatically by Apple's Metal compiler and never by a plain C++ compiler, so this `#ifdef` is the one and only branch point between "being compiled as a shader" and "being compiled as a CPU translation unit." On the MSL side, `RT_DEVICE` expands to the `device` keyword; on the C++ side it expands to nothing at all, which is exactly right, because a bare pointer *is* how C++ already spells "pointer to something the caller owns." `rayColor()`'s signature shows the macro in its one use site:

```
inline RayColorResult rayColor( Ray r,
        RT_DEVICE const TransformGPU* transforms, RT_DEVICE const ShapeGPU* shapes,
uint32_t entityCount,
        RT_DEVICE const MaterialGPU* materials, uint32_t maxDepth, uint32_t rngSeed )
```

`RayTracerBench/Core/RayTraceCore.h:467–469`

Every other function in the file — `hitSphere()`, `hitPyramid()`, `hitEntity()`, `scatter()`, `getRay()` — takes its `TransformGPU` / `ShapeGPU` / `MaterialGPU` arguments *by value*. That is not an incidental style choice; it is what makes the rest of the file exempt from needing any address-space annotation at all. `rayColor()`'s inner loop reads `transforms[i]` and `shapes[i]` out of the `device`-space buffers exactly once per entity, into by-value thread-local copies, and only *those* copies get passed onward into `hitEntity()`:

```
HitResult hr = hitEntity( transforms[ i ], shapes[ i ], r, 0.001f, closestSoFar );
```

`RayTracerBench/Core/RayTraceCore.h:484`

MSL treats a `TransformGPU` or `ShapeGPU` value copied out of a `device` buffer as an ordinary `thread`-space value from that point on, needing no further annotation — which is precisely why a second, MSL-only address-space keyword (`thread`) is never needed anywhere in this file, even though a more naively-ported C++ codebase (one that passed hit records by reference) would have needed it constantly.

§3. Vector arithmetic, built from scratch

Before any ray tracing happens, the file has to solve a smaller but equally real portability problem: `simd_float3` (the C++/`<simd/simd.h>` 3-vector type) and MSL's native `float3` are laid out identically but do not share an API. Plain C++'s `simd` library spells dot product `simd_dot`; MSL spells it `dot`. Worse, MSL allows `float3(x, y, z)` as ordinary function-style construction, while C++ rejects that same syntax as an illegal 3-argument functional cast — it insists on `simd_make_float3(x, y, z)` or brace-init instead. Rather than macro away every one of these naming mismatches individually, this file sidesteps essentially all of them by implementing its own tiny vector-math layer using only field access (`.x` / `.y` / `.z`) and arithmetic operators, which *are* spelled identically on both sides:

```
inline simd_float3 makeFloat3( float x, float y, float z )
{
#if defined( __METAL_VERSION__ )
        return float3( x, y, z );
#else
        return simd_make_float3( x, y, z );
#endif
}

inline float dot3( simd_float3 a, simd_float3 b )
{
        return a.x * b.x + a.y * b.y + a.z * b.z;
}
```

`RayTracerBench/Core/RayTraceCore.h:49–62`

`makeFloat3()` is the file's second and last `#ifdef __METAL_VERSION__` — everything downstream of it (`length3`, `normalize3`, `nearZero3`, `reflect3`, `refract3`, `reflectance`) is ordinary, portable arithmetic over `.x` / `.y` / `.z` and needs no further conditional compilation at all. Any new 3-argument vector literal added later to this file is expected to go through `makeFloat3()` rather than reaching for `simd_make_float3` or `float3(...)` directly — `Scene.cpp`, by contrast, is pure host C++ that is *never* Metal-compiled, and uses the native `simd_make_float3` / `simd_dot` C API directly, because it has no dual-compilation constraint to honor.

§4. `pcgHash` / `randomFloat`: one RNG, run identically on both processors

The CUDA port this project takes structural cues from uses NVIDIA's `curand` library for its per-thread random numbers — not an option here, since it is neither portable to MSL nor to a plain CPU. This file replaces it with a pcg-hash-based RNG that is small enough to be genuinely identical, statement for statement, on both processors:

```
inline uint32_t pcgHash( uint32_t input )
{
        uint32_t state = input * 747796405u + 2891336453u;
        uint32_t word = ( ( state >> ( ( state >> 28u ) + 4u ) ) ^ state ) * 277803737u;
        return ( word >> 22u ) ^ word;
}
```

`RayTracerBench/Core/RayTraceCore.h:116–121`

The state itself is nothing more exotic than a `uint32_t`, and per §2's discipline it is threaded through by value rather than by mutable reference: every draw function takes a `seed` and returns both the drawn value *and* the advanced seed in one struct, so a caller chains state explicitly (`seed = sample.seed`) instead of the RNG needing an out-parameter:

```
inline RandomFloatSample randomFloat( uint32_t seed )
{
        uint32_t nextSeed = pcgHash( seed );
        float    value = (float)nextSeed / 4294967296.0f; // 2^32 -> [0,1)
        return RandomFloatSample{ value, nextSeed };
}
```

`RayTracerBench/Core/RayTraceCore.h:136–141`

`randomFloatRange()` and `randomVec3Range()` build on `randomFloat()` the same way, each threading `seed` forward through the next draw (`RayTracerBench/Core/RayTraceCore.h:144–158`). Two derived samplers are worth calling out because of a small but deliberate departure from the book's own code: `randomInUnitSphere()` and `randomInUnitDisk()` are both rejection samplers (draw a point in a cube/square, keep it only if it lands inside the inscribed sphere/disk), and the book's reference implementation expresses that as a `while (true)` loop. An unbounded loop is unacceptable in a kernel meant to run identically as a GPU compute shader, so both are capped at 32 attempts instead, with a zero-vector fallback if the cap is ever hit:

```
inline RandomVec3Sample randomInUnitSphere( uint32_t seed )
{
        for ( int i = 0; i < 32; ++i )
        {
                RandomVec3Sample s = randomVec3Range( seed, -1.0f, 1.0f );
                seed = s.seed;
                if ( dot3( s.value, s.value ) < 1.0f )
                        return s;
        }
        return RandomVec3Sample{ makeFloat3( 0.0f, 0.0f, 0.0f ), seed };
}
```

`RayTracerBench/Core/RayTraceCore.h:162–172`

This is the same "every loop in this shared source must be statically bounded" principle that governs `rayColor()`'s bounce loop in §9 — rejection probability for a random point landing inside an inscribed sphere/disk is high enough (roughly 52% for a sphere in a cube, ~79% for a disk in a square) that 32 tries failing is astronomically unlikely, but the *loop shape itself*, not just the expected iteration count, has to be boundable for MSL kernel code to be well-formed.

§5. `hitSphere()`: the simplest entity, and the shape of a `HitResult`

`hitSphere()` is the first place the file's `HitResult`/`HitRecord` return convention appears in full, and it is a good reference point before §6's more involved pyramid case:

```
inline HitResult hitSphere( TransformGPU transform, ShapeGPU shape, Ray r, float tMin,
float tMax )
{
        HitResult result;
        result.hit = false;

        simd_float3 center = transform.position;
        float       radius = shape.radius;

        simd_float3 oc = r.origin - center;
        float       a = dot3( r.direction, r.direction );
        float       halfB = dot3( oc, r.direction );
        float       c = dot3( oc, oc ) - radius * radius;
        float       discriminant = halfB * halfB - a * c;
        if ( discriminant < 0.0f )
                return result;
        float sqrtd = sqrt( discriminant );

        float root = ( -halfB - sqrtd ) / a;
        if ( root < tMin || root > tMax )
        {
                root = ( -halfB + sqrtd ) / a;
                if ( root < tMin || root > tMax )
                        return result;
        }
        /* ... fills in HitRecord and returns result.hit = true ... */
}
```

`RayTracerBench/Core/RayTraceCore.h:234–270` (elided: `HitRecord` construction, lines 259–269)

This is the book's usual quadratic-formula sphere test, algebraically unchanged — the interesting part, given this chapter's theme, is what it *doesn't* do: `transform` and `shape` arrive by value (per §2), `center` comes from `transform.position` alone, and the comment above the function notes that a sphere's `right` / `up` / `forward` basis fields are simply ignored, "meaningless for a rotationally-symmetric shape" (`RayTracerBench/Core/RayTraceCore.h:230–233`). That basis becomes essential the moment the second primitive — a pyramid, which very much has an orientation — enters the picture.

§6. `hitPyramidLocal()` / `hitPyramid()`: five half-spaces, one slab test

A square pyramid is not a primitive either format or the ray-tracing book gives you for free, and this file's solution generalizes a trick usually reserved for axis-aligned boxes. An AABB is the intersection of 6 half-spaces (one pair per axis), and the standard fast way to ray-test one is the Kay–Kajiya slab method: shrink a valid-`t` interval `[tNear, tFar]` by intersecting it against each pair of parallel planes in turn, and reject the ray the moment the interval

becomes empty. A pyramid has 5 faces instead of 6 and none of them come in parallel pairs, but the *same* narrowing-interval algorithm still applies to any convex solid expressed as an intersection of half-spaces — box or pyramid alike. The header comment above `hitPyramidLocal()` states the generalization directly and also pins down the canonical local frame the closed-form planes below are derived in:

```
// Square pyramid, represented as the intersection of 5 half-spaces (1 base + 4 triangular
sides)
// and solved with the Kay-Kajiya slab method — the same technique an axis-aligned box
uses with
// its 6 planes, generalized to these 5. Canonical local space: base square in the local
XZ plane
// at y=0 (corners at (+-halfWidth, 0, +-halfWidth)), apex at local (0, height, 0).
```

`RayTracerBench/Core/RayTraceCore.h:273–277`

Each face's plane equation is not fit numerically from the corner points — it falls straight out of the pyramid's symmetry. Take the +X side face: it passes through the two base corners `(halfWidth, 0, ±halfWidth)` and the apex `(0, height, 0)`. Solving for that plane's equation by hand gives `height·x + halfWidth·y = halfWidth·height`, i.e. an (unnormalized) outward normal of `(height, halfWidth, 0)` at signed distance `halfWidth·height` from the local origin. The other three side faces are the same equation under the pyramid's obvious symmetries — negate `x` for the −X face, swap `x` / `z` for the two Z-facing faces — so all four can be written down without solving anything a second time:

```
const simd_float3 normals[ 5 ] = {
        makeFloat3( 0.0f, -1.0f, 0.0f ),        // base
        makeFloat3( height, halfWidth, 0.0f ),  // +X side
        makeFloat3( -height, halfWidth, 0.0f ), // -X side
        makeFloat3( 0.0f, halfWidth, height ),  // +Z side
        makeFloat3( 0.0f, halfWidth, -height ), // -Z side
};
const float planeDist = halfWidth * height;
const float dists[ 5 ] = { 0.0f, planeDist, planeDist, planeDist, planeDist };
```

`RayTracerBench/Core/RayTraceCore.h:292–300`

(The base plane is simpler still — its outward normal is just world-down, `(0, -1, 0)`, at distance 0 from the local origin, since the base sits exactly at local `y = 0`.)

With the five planes fixed, the slab loop narrows `[tNear, tFar]` one face at a time. For each face, `numer` / `denom` gives the ray parameter `t` where the ray crosses that face's plane;

whether the face is being *entered* or *exited* is decided by the sign of `denom` (the ray direction's component along the outward normal):

```
for ( int i = 0; i < 5; ++i )
{
        float denom = dot3( normals[ i ], localRay.direction );
        float numer = dists[ i ] - dot3( normals[ i ], localRay.origin );

        if ( fabs( denom ) < 1e-8f )
        {
                // Ray parallel to this face: if the origin is already outside its half-
space, the ray
                // can never enter the pyramid at all, regardless of the other four faces.
                if ( numer < 0.0f )
                        return LocalHitResult{ false, 0.0f, makeFloat3( 0.0f, 0.0f, 0.0f )
};
                continue;
        }

        float t = numer / denom;
        if ( denom < 0.0f )
        {
                if ( t > tNear )
                {
                        tNear = t;
                        enterFace = i;
                }
        }
        else
        {
                if ( t < tFar )
                        tFar = t;
        }
}
```

`RayTracerBench/Core/RayTraceCore.h:306–334`

`denom < 0` means the ray is heading *into* that face's half-space (entering), so it can only ever raise `tNear`, and whichever face last did so is remembered in `enterFace` — that becomes the hit normal if the ray ultimately hits at all. `denom > 0` means the ray is heading *out* of that half-space (exiting), so it can only lower `tFar`. A ray parallel to a face (`denom ≈ 0`) never changes the interval either way, except in the degenerate case where the ray's origin is already outside that face's plane, in which case no value of `t` can ever put it back inside and the whole test is immediately rejected (`RayTracerBench/Core/RayTraceCore.h:311–318`). After all five faces are folded into the interval, the pyramid was hit only if some face actually raised `tNear` above `tMin` (`enterFace >= 0`) *and* that interval never went empty (`tNear <= tFar`):

```
if ( enterFace < 0 || tNear > tFar )
        return LocalHitResult{ false, 0.0f, makeFloat3( 0.0f, 0.0f, 0.0f ) };

return LocalHitResult{ true, tNear, normalize3( normals[ enterFace ] ) };
```

`RayTracerBench/Core/RayTraceCore.h:340–343`

That comment above the rejection check is where this file documents an explicit, deliberate scope limit rather than a bug: `enterFace < 0` can also mean the ray's origin already starts *inside* the pyramid (every plane already satisfied at `tMin`), a case this test does not resolve correctly. That case matters specifically for glass: refraction sends a ray *through* a dielectric surface, so a correct glass pyramid would need to detect the ray exiting from inside the solid on its far side — exactly the situation this slab test's entry-face-only logic can't handle. Rather than special-case that, `Scene.cpp` (Chapter 1) simply never assigns a `MAT_DIELECTRIC` material to a pyramid entity; pyramids in this scene are always lambertian or metal, a documented restriction rather than an oversight.

`hitPyramidLocal()` only ever sees the ray in the pyramid's own canonical local frame, though — the ray this file actually receives is in world space, where the pyramid can sit at any position and any orientation. `hitPyramid()` is the wrapper that converts between the two, and it is where `TransformGPU`'s orthonormal `right` / `up` / `forward` basis (Chapter 1) earns its keep. The trick it relies on is that the inverse of an orthogonal matrix is just its transpose — so instead of building and inverting a 3×3 rotation matrix, transforming a world-space vector into the pyramid's local frame is just three dot products, one against each basis vector:

```
simd_float3 oc = r.origin - transform.position;
Ray         localRay;
localRay.origin = makeFloat3( dot3( oc, transform.right ), dot3( oc, transform.up ), dot3(
oc, transform.forward ) );
localRay.direction = makeFloat3(
        dot3( r.direction, transform.right ), dot3( r.direction, transform.up ), dot3(
r.direction, transform.forward ) );
```

`RayTracerBench/Core/RayTraceCore.h:356–360`

and mapping the resulting local-space hit normal back out to world space runs the same basis the other direction — not a transpose this time, but literally interpreting the local normal's components as coefficients on the basis vectors themselves:

```
simd_float3 outwardNormal =
        lh.localNormal.x * transform.right + lh.localNormal.y * transform.up +
lh.localNormal.z * transform.forward;
```

`RayTracerBench/Core/RayTraceCore.h:369-370`

The accompanying comment notes one more simplification this buys: because translating and rotating a ray is an isometry (it doesn't stretch distances), the intersection parameter `t` computed in local space is identical to `t` in world space, so the world-space hit point itself can be computed directly via `rayAt(r, lh.t)` on the *original* world-space ray — no inverse transform of the hit point is ever needed (`RayTracerBench/Core/RayTraceCore.h:346-350`).

§7. `hitEntity()`: the collision system

With both primitives' intersection tests in hand, `hitEntity()` is the function that ties `ShapeGPU`'s tag to the right test — and, per the project's ECS framing (Chapter 1), this is properly thought of as a "collision system" operating over the Transform+Shape component arrays, not a shape method:

```
inline HitResult hitEntity( TransformGPU transform, ShapeGPU shape, Ray r, float tMin,
float tMax )
{
        switch ( shape.type )
        {
                case SHAPE_SPHERE:
                        return hitSphere( transform, shape, r, tMin, tMax );
                case SHAPE_PYRAMID:
                        return hitPyramid( transform, shape, r, tMin, tMax );
        }
        HitResult miss;
        miss.hit = false;
        return miss;
}
```

`RayTracerBench/Core/RayTraceCore.h:385-397`

The comment directly above it is explicit about what this replaces:

```
// The "collision system": iterates one entity's Transform + Shape components and
dispatches the
// intersection test by the Shape component's tag — the ECS analogue of scatter()'s tagged
switch
// below, and the direct replacement for a virtual Hittable::hit() call (forbidden in MSL
kernels;
// see CLAUDE.md's core correctness constraint). Adding a third primitive means one more
case here,
// not a new subclass.
```

`RayTracerBench/Core/RayTraceCore.h:380-384`

This is the entire cost of adding a new primitive shape to the renderer: one more `SHAPE_*` enumerator in `ShaderTypes.h`, one more intersection function, and one more `case` here — never a new class, a new vtable slot, or a change to any caller. `rayColor()` (§9) never needs to know how many shape kinds exist; it only ever calls `hitEntity()` once per entity, and `hitEntity()` is the only place that switches on `shape.type` at all.

§8. `scatter()`: the same tagged-switch idea, applied to materials

`scatter()` is `hitEntity()`'s counterpart on the shading side — a tagged switch replacing the book's virtual `Material::scatter()`, over the three `MaterialType` tags from `ShaderTypes.h` (Chapter 1):

```
inline ScatterResult scatter( Ray rayIn, HitRecord rec, MaterialGPU mat, uint32_t rngSeed
)
{
        switch ( mat.type )
        {
                case MAT_LAMBERTIAN:
                {
                        RandomVec3Sample rv = randomUnitVector( rngSeed );
                        simd_float3      scatterDirection = rec.normal + rv.value;
                        if ( nearZero3( scatterDirection ) )
                                scatterDirection = rec.normal;
                        return ScatterResult{ true, mat.albedo, Ray{ rec.p,
scatterDirection }, rv.seed };
                }
                case MAT_METAL:
                {
                        RandomVec3Sample rv = randomInUnitSphere( rngSeed );
                        simd_float3      reflected = reflect3( normalize3( rayIn.direction
), rec.normal );
                        Ray              scattered = Ray{ rec.p, reflected + mat.fuzz *
rv.value };
                        bool             ok = dot3( scattered.direction, rec.normal ) >
0.0f;
                        return ScatterResult{ ok, mat.albedo, scattered, rv.seed };
                }
                case MAT_DIELECTRIC:
                {
                        float             refractionRatio = rec.frontFace ? ( 1.0f /
mat.ir ) : mat.ir;
                        simd_float3       unitDirection = normalize3( rayIn.direction );
                        float             cosTheta = fmin( dot3( -unitDirection,
rec.normal ), 1.0f );
                        float             sinTheta = sqrt( 1.0f - cosTheta * cosTheta );
                        bool              cannotRefract = refractionRatio * sinTheta >
1.0f;
                        RandomFloatSample rf = randomFloat( rngSeed );
                        simd_float3       direction;
                        if ( cannotRefract || reflectance( cosTheta, refractionRatio ) >
rf.value )
                                direction = reflect3( unitDirection, rec.normal );
                        else
                                direction = refract3( unitDirection, rec.normal,
refractionRatio );
                        return ScatterResult{ true, makeFloat3( 1.0f, 1.0f, 1.0f ), Ray{
rec.p, direction }, rf.seed };
                }
        }
        return ScatterResult{ false, makeFloat3( 0.0f, 0.0f, 0.0f ), rayIn, rngSeed };
}
```

`RayTracerBench/Core/RayTraceCore.h:415–452`

All three branches are the book's standard physical models — cosine-weighted diffuse scatter for Lambertian (with a degenerate-direction guard via `nearZero3()`, §3), fuzzy mirror reflection for metal, and Schlick-approximated reflect/refract branching (via `reflectance()`, §3) for dielectric — but every one of them ends the same way this file always does: reading `rngSeed` in, drawing whatever randomness it needs, and packaging a new `rngSeed` back out inside `ScatterResult` rather than mutating anything by reference. `mat` itself is a plain by-value copy, per the same discipline as `hitSphere()` / `hitPyramid()`'s `shape` parameter — the comment directly above the struct spells this out:

```
// Tagged-switch scatter, replacing the book's virtual material dispatch (forbidden in MSL
kernel
// code). `mat` is likewise a by-value thread-local copy, read out of the materials buffer
by the
// caller.
```

`RayTracerBench/Core/RayTraceCore.h:400-402`

§9. `rayColor()`: the bounded loop that both renderers actually call

Every function discussed so far is a building block; `rayColor()` is the function that assembles them into the path tracer that a caller — the CPU's per-pixel loop in Chapter 4, or the GPU kernel's per-thread body in Chapter 3 — invokes once per sample. The book's reference implementation expresses bounce depth as *recursion*: `ray_color()` calls itself with `depth - 1` on every scatter. Recursion of unbounded (or even just data-dependent) depth is exactly the kind of thing that risks a call-stack blowup on a GPU thread, where stack space per thread is far smaller and less forgiving than on a CPU — and, independent of that risk, keeping the two renderers' control flow *structurally* identical (not just algorithmically equivalent) is the whole premise of dual-compiling one source file at all. So `rayColor()` is iterative: a single bounded `for` loop up to `maxDepth`, which the app's default settings put around 50:

```
inline RayColorResult rayColor( Ray r,
        RT_DEVICE const TransformGPU* transforms, RT_DEVICE const ShapeGPU* shapes,
uint32_t entityCount,
        RT_DEVICE const MaterialGPU* materials, uint32_t maxDepth, uint32_t rngSeed )
{
        simd_float3 accumulated = makeFloat3( 1.0f, 1.0f, 1.0f );

        for ( uint32_t depth = 0; depth < maxDepth; ++depth )
        {
                HitRecord closestRecord;
                bool      hitAnything = false;
                float     closestSoFar = 1.0e30f;

                for ( uint32_t i = 0; i < entityCount; ++i )
                {
                        HitResult hr = hitEntity( transforms[ i ], shapes[ i ], r, 0.001f,
closestSoFar );
                        if ( hr.hit )
                        {
                                hitAnything = true;
                                closestSoFar = hr.record.t;
                                closestRecord = hr.record;
                        }
                }

                if ( !hitAnything )
                {
                        simd_float3 unitDirection = normalize3( r.direction );
                        float       t = 0.5f * ( unitDirection.y + 1.0f );
                        simd_float3 skyColor = ( 1.0f - t ) * makeFloat3( 1.0f, 1.0f, 1.0f
) + t * makeFloat3( 0.5f, 0.7f, 1.0f );
                        return RayColorResult{ accumulated * skyColor, rngSeed };
                }

                MaterialGPU    mat = materials[ closestRecord.materialIndex ];
                ScatterResult  sr = scatter( r, closestRecord, mat, rngSeed );
                rngSeed = sr.rngSeed;

                if ( !sr.scattered )
                        return RayColorResult{ makeFloat3( 0.0f, 0.0f, 0.0f ), rngSeed };

                accumulated = accumulated * sr.attenuation;
                r = sr.scatteredRay;
        }

        return RayColorResult{ makeFloat3( 0.0f, 0.0f, 0.0f ), rngSeed }; // exceeded
bounce budget
}
```

`RayTracerBench/Core/RayTraceCore.h:467–513`

Each iteration is the book's `world.hit()` loop, but ECS-flavored: instead of asking a polymorphic scene object "did anything get hit," it walks the `transforms[]` / `shapes[]` component arrays in lockstep and calls `hitEntity()` (§7) per entity, keeping whichever hit has the smallest `t` so far. That inner scan is annotated in the source as "the render system," drawing the same ECS parallel `hitEntity()` itself draws to "the collision system" (`RayTracerBench/Core/RayTraceCore.h:479–481`). If nothing was hit, the loop terminates immediately with the sky gradient (a simple vertical lerp between white and light blue, unchanged from the book). If something *was* hit, `scatter()` (§8) is called once, `accumulated` is attenuated by whatever color that material contributed, `r` becomes the new scattered ray, and the loop continues — recursion turned into plain iteration by carrying the running product forward as a local variable instead of a return-value multiplication unwound on the call stack. A ray absorbed by a material (`sr.scattered == false`, which only `MAT_METAL` can produce, when the fuzzed reflection points back into the surface) or one that survives all `maxDepth` bounces without escaping to sky both return black — the former physically (no light reached the camera along that path), the latter as the same bounded-loop safety net every other loop in this file relies on (§4).

§10. `getRay()`: camera-ray sampling with defocus blur

The last piece is where a ray originates in the first place. `getRay()` builds one camera ray for a normalized image coordinate `(s, t)`, including thin-lens defocus-blur jitter — sampling a random point on the lens disk (via `randomInUnitDisk()`, §4) and offsetting the ray's origin by it, which is what produces a depth-of-field effect rather than a pinhole camera's perfectly sharp focus at every distance:

```
inline CameraRaySample getRay( CameraGPU cam, float s, float t, uint32_t rngSeed )
{
        RandomVec3Sample diskSample = randomInUnitDisk( rngSeed );
        simd_float3      rd = cam.lensRadius * diskSample.value;
        simd_float3      offset = cam.u * rd.x + cam.v * rd.y;

        simd_float3 origin = cam.origin + offset;
        simd_float3 direction = cam.lowerLeftCorner + s * cam.horizontal + t *
cam.vertical - cam.origin - offset;

        return CameraRaySample{ Ray{ origin, direction }, diskSample.seed };
}
```

`RayTracerBench/Core/RayTraceCore.h:527–537`

`cam.u` / `cam.v` here are the camera's own right/up basis vectors (part of `CameraGPU`, Chapter 1) — the same "project onto an orthonormal basis" idea §6 used to move a ray into a pyramid's local space, applied instead to spread the lens-disk sample across the camera's image plane. As with every other sampling function in this file, the caller — one loop iteration per sample per pixel, in both `CPURenderer.cpp` and `Raytracer.metal` — gets back both the ray and the advanced RNG seed in one struct, ready to feed straight into `rayColor()`.

§11. Trusted before it was ever wired up

This file's dual-compilation claim was not left to be discovered the first time the GPU renderer was assembled and something failed to link. Before `GPURenderer` existed at all, `RayTraceCore.h` was validated standalone against the real Metal compiler — `xcrun -sdk macosx metal -c ... -Wall -Wextra` — and it compiled clean, zero warnings, on the first attempt. That is the kind of claim this project treats as needing independent evidence rather than confidence: the value-returning, single-address-space-keyword design described in §2 is only actually validated by a real compiler accepting it as legal MSL, not by inspection. Chapter 3 picks up from exactly this point — how `Raytracer.metal` wraps this file's `rayColor()` in an actual `kernel void renderKernel(...)` entry point and wires `RT_DEVICE`'s one real buffer argument to Metal's `dispatchThreads:` machinery.

Where this connects

- **Chapter 1 (The Core Data Model)** defines every struct this file consumes without owning: `TransformGPU`, `ShapeGPU` / `ShapeType`, `MaterialGPU` / `MaterialType`, `CameraGPU`. This chapter's `hitPyramid()` (§6) and `getRay()` (§10) are the two places that basis vectors (`right` / `up` / `forward`, or `u` / `v`) defined in Chapter 1 actually get used as an orthonormal frame rather than just carried around as data.
- **Chapter 3 (The Metal Shaders)** is one of this file's two compile targets: `Shaders/Raytracer.metal` `#include`s `RayTraceCore.h` verbatim and wraps `rayColor()` in a `kernel void renderKernel(...)` entry point, supplying the real `device`-space buffer arguments that make `RT_DEVICE` (§2) resolve to something concrete.
- **Chapter 4 (The CPU Renderer)** is this file's other compile target: `CPU/CPURenderer.cpp` `#include`s the same header as plain C++, where `RT_DEVICE` disappears entirely, and drives `rayColor()` / `getRay()` from an `std::thread`-partitioned per-row loop instead of a GPU dispatch grid.

- **Chapter 11 (Tests: Parity, Geometry, and the Core Itself)** is how the claim underlying this entire chapter — that the CPU and GPU renderers really are executing the same algorithm, not just similar-looking ones — gets checked mechanically: a fixed-seed scene run through both compiled outputs and diffed, plus the standalone Metal-compiler validation mentioned in §11.

Chapter 3: The Metal Shaders

Abstract. This chapter covers the two `.metal` files in the program: `Shaders/Raytracer.metal`, the GPU compute kernel that actually renders the scene, and `Shaders/Blit.metal`, the small textured-quad shader that gets a rendered image (from either renderer) onto screen. The two files could not be more different in how they get compiled, and that difference is itself the most instructive thing about them: one is compiled ahead of time from a real file on disk because it `#include`s the shared core headers verbatim; the other is compiled from an in-memory string at runtime because it has no `#include`s to resolve. Blit.metal also carries a second, unrelated job — the magnifying-glass loupe used to compare CPU and GPU output pixel-for- pixel — which this chapter walks through in full since the shader math is where that feature actually lives.

Files covered: `RayTracerBench/Shaders/Raytracer.metal`, `RayTracerBench/Shaders/Blit.metal`.

§1. Raytracer.metal is not a shader with logic in it — it's a thin wrapper around Chapter 2

The first thing to notice about `Raytracer.metal` is how little of it is actually raytracing logic:

```
// RayTracerBench/Shaders/Raytracer.metal:1–11
#include <metal_stdlib>
using namespace metal;

#include "../Core/ShaderTypes.h"
#include "../Core/RayTraceCore.h"

// One thread per pixel. entityCount/camera/params arrive via setBytes() (small, read-
only,
// uniform across all threads) rather than being folded into RenderParams —
transforms/shapes/
// materials stay separate `device` buffers (the ECS component arrays; see
ShaderTypes.h/Scene.hpp)
// exactly like the CPU renderer's raw pointers, per RayTraceCore.h's RT_DEVICE design
(see
// CLAUDE.md).
```

Everything that actually decides what color a pixel is — sphere and pyramid intersection, material scattering, the bounded bounce loop — lives in `Core/RayTraceCore.h` (Chapter 2), and everything that describes *what* is being rendered — the transform/shape/material component arrays, the camera, the render parameters — lives in `Core/ShaderTypes.h`

(Chapter 1). This file does not redeclare or reimplement any of it. It `#include`s both headers verbatim, at lines 4–5, exactly the way `CPU/CPURenderer.cpp` does on the C++ side (Chapter 4). That's the whole point of the dual-compile design described in Chapter 2: the same source text becomes a Metal kernel here and a plain C++ function there, so there is no second implementation to drift out of sync with the first.

§2. Why this file can't be compiled from a source string, and what failed when it was tried

`Blit.metal`, as we'll see in §4, is compiled from an in-memory string via `newLibrary(sourceString, ...)`. `Raytracer.metal` cannot be, and the reason is sitting right there in lines 4–5: `#include "../Core/ShaderTypes.h"` and `#include "../Core/RayTraceCore.h"` are *relative* includes. A relative `#include` is resolved against the location of the file containing it — but when MSL source is handed to `newLibrary()` as a bare in-memory string, there is no file location for the compiler to resolve anything against. The string has no path; the `#include` has nothing to be relative *to*.

Per `CLAUDE.md`'s project-status notes, this is not a design that was reasoned out in advance and avoided — it "failed loudly and immediately at runtime the first time it was tried." Compiling `Raytracer.metal` in-memory the way `Blit.metal` is compiled produces exactly the include-resolution failure the paragraph above predicts, and it does so as soon as `newLibrary()` is called, not as some later, subtler bug.

The fix is to give the file a real location to be `#include`d from, which means giving up on runtime string compilation for this one shader and compiling it ahead of time as an actual file on disk. `CMakeLists.txt` (Chapter 12) does this with a build-time `add_custom_command` that shells out to Apple's command-line Metal compiler, not Xcode's own "Metal Compiler" build phase — kept deliberately as plain `xcrun` invocations, consistent with this project's headless build style described elsewhere in `CLAUDE.md`:

```
# CMakeLists.txt:61-87
# Raytracer.metal #includes Core/ShaderTypes.h and Core/RayTraceCore.h verbatim (the dual-
compile
# design), so it CANNOT be compiled from a raw in-memory source string the way Blit.metal
is —
# there's no file location for a relative #include to resolve against. Compile it to a
real
# .metallib at build time instead (still command-line xcrun, not an Xcode "Metal Compiler"
build
# phase, to stay consistent with this project being built headlessly), and load that
compiled
# library at runtime via newLibrary(filepath, &error) rather than newLibrary(sourceString,
...).
set(RAYTRACER_METAL_SOURCE ${CMAKE_SOURCE_DIR}/RayTracerBench/Shaders/Raytracer.metal)
set(RAYTRACER_METAL_AIR ${CMAKE_BINARY_DIR}/Raytracer.air)
set(RAYTRACER_METALLIB ${CMAKE_BINARY_DIR}/Raytracer.metallib)

add_custom_command(
    OUTPUT ${RAYTRACER_METALLIB}
    COMMAND xcrun -sdk macosx metal -c ${RAYTRACER_METAL_SOURCE} -o ${RAYTRACER_METAL_AIR}
    COMMAND xcrun -sdk macosx metallib ${RAYTRACER_METAL_AIR} -o ${RAYTRACER_METALLIB}
    DEPENDS
        ${RAYTRACER_METAL_SOURCE}
        ${CMAKE_SOURCE_DIR}/RayTracerBench/Core/ShaderTypes.h
        ${CMAKE_SOURCE_DIR}/RayTracerBench/Core/RayTraceCore.h
    COMMENT "Compiling Raytracer.metal -> Raytracer.metallib"
    VERBATIM
)
add_custom_target(RaytracerMetalLib DEPENDS ${RAYTRACER_METALLIB})
add_dependencies(RayTracerBench RaytracerMetalLib)

target_compile_definitions(RayTracerBench PRIVATE
    RT_RAYTRACER_METALLIB="${RAYTRACER_METALLIB}"
)
```

The `DEPENDS` list is worth pausing on: it names not just `Raytracer.metal` itself but `ShaderTypes.h` and `RayTraceCore.h` too, so that editing either shared header (Chapters 1 and 2) correctly invalidates and recompiles the `.metallib`, even though those headers are never listed as sources of any CMake target in their own right.

The resulting `.metallib` path is injected into the build as the `RT_RAYTRACER_METALLIB` compile definition, and `GPU/GPURenderer.cpp` (Chapter 5) loads it as a real file path rather than a source string:

```
// RayTracerBench/GPU/GPURenderer.cpp:20-25
// RT_RAYTRACER_METALLIB (injected by CMakeLists.txt) is a real compiled .metallib, not a
// source string like ImageDisplayView's Blit.metal — Raytracer.metal #includes
Core/ShaderTypes.h
// and Core/RayTraceCore.h verbatim, and a raw source string has no file location for
those
// relative #includes to resolve against. See CMakeLists.txt's comment for the build-time
step.
NS::Error*    pError = nullptr;
MTL::Library* pLibrary = _pDevice->newLibrary( NS::String::string( RT_RAYTRACER_METALLIB,
UTF8StringEncoding ), &pError );
```

The overload being called here — `newLibrary(NS::String filepath, NS::Error)` — *is a different metal-cpp entry point than* `Blit.metal`*'s* `newLibrary(NS::String source, MTL::CompileOptions, NS::Error*)`; the argument happens to be an `NS::String` in both cases, but in this call it's a path Metal opens and parses as a precompiled library, not MSL text Metal has to lex and compile. Chapter 5 covers the rest of `GPURenderer`'s construction — pipeline state creation, command queue setup — that follows this load.

§3. The kernel signature: six buffers mirroring the CPU renderer's raw pointers

`renderKernel` is a `kernel void` function — a Metal compute kernel, one invocation per grid position, with no return value; results are written directly into a texture. Its argument list is a direct, buffer-by-buffer translation of the same data `CPURenderer.cpp` walks with raw C++ pointers (Chapter 4):

```
// RayTracerBench/Shaders/Raytracer.metal:12-20
kernel void renderKernel(
        device const TransformGPU* transforms [[buffer( 0 )]],
        device const ShapeGPU*     shapes [[buffer( 1 )]],
        constant uint32_t&         entityCount [[buffer( 2 )]],
        device const MaterialGPU*  materials [[buffer( 3 )]],
        constant CameraGPU&        camera [[buffer( 4 )]],
        constant RenderParams&     params [[buffer( 5 )]],
        texture2d<float, access::write> outTexture [[texture( 0 )]],
        uint2                      gid [[thread_position_in_grid]] )
```

Buffers 0, 1, and 3 — `transforms`, `shapes`, `materials` — are the ECS component arrays from `Scene.hpp` (Chapter 1), each qualified `device const`: potentially large, read-only, and indexed per-entity rather than uniform across threads. This is precisely the one spot in the shared core that needs the `RT_DEVICE` address-space macro described in Chapter 2 — `hitEntity()` and `rayColor()` take these three as `device`-qualified pointers, the only

address-space keyword the dual-compiled core ever needs, because CPU code has no equivalent concept and the macro compiles away to nothing there.

Buffers 2, 4, and 5 — `entityCount`, `camera`, `params` — are `constant` references instead of `device` pointers. The comment at lines 7–11 explains the distinction directly: these three are small and identical for every thread in the dispatch (there is one camera, one entity count, one set of render parameters for the whole frame), so they're passed via `setBytes()` on the host side and read as `constant` — Metal's address space for uniform, read-only, broadcast data — rather than being folded into `RenderParams` itself or treated as another per-entity `device` array.

`outTexture` is the kernel's actual output — a `texture2d<float, access::write>`, written to exactly once per thread at the end of the function — and `gid`, `[[thread_position_in_grid]]`, is Metal's built-in per-thread grid coordinate, used both as the pixel being rendered and as part of the per-pixel RNG seed.

§4. What the kernel body does, briefly — and where the dispatch itself lives

The body is short precisely because it delegates almost everything to Chapter 2's functions. After an early-out for threads past the image bounds (line 22–23, needed because the dispatch grid is rounded up to a multiple of the threadgroup size — see below), it hashes a per-pixel seed, then loops `samplesPerPixel` times:

```
// RayTracerBench/Shaders/Raytracer.metal:29-46
for ( uint32_t s = 0; s < params.samplesPerPixel; ++s )
{
        RandomFloatSample ru = randomFloat( seed );
        seed = ru.seed;
        RandomFloatSample rv = randomFloat( seed );
        seed = rv.seed;

        float u = ( float( gid.x ) + ru.value ) / float( params.width - 1 );
        float v = ( float( params.height - 1 - gid.y ) + rv.value ) / float( params.height
- 1 );

        CameraRaySample cameraRay = getRay( camera, u, v, seed );
        seed = cameraRay.rngSeed;

        RayColorResult sample = rayColor( cameraRay.ray, transforms, shapes, entityCount,
materials, params.maxDepth, seed );
        seed = sample.rngSeed;

        colorSum = colorSum + sample.color;
}
```

Every function called here — `randomFloat`, `getRay`, `rayColor` — is declared in `RayTraceCore.h` and explained in Chapter 2; nothing about their internals is specific to the GPU. The `seed = ru.seed` / `seed = sample.rngSeed` threading visible on nearly every line is Chapter 2's value-in/value-out RNG convention showing up directly at the call site: no function here mutates shared state, so the same loop, unchanged, is also what `CPURenderer.cpp` runs per pixel.

After averaging the accumulated samples, the kernel applies gamma correction and a clamp before writing the pixel:

```
// RayTracerBench/Shaders/Raytracer.metal:48-52
simd_float3 averageColor = colorSum / float( params.samplesPerPixel );
simd_float3 gammaCorrected = sqrt( max( averageColor, 0.0f ) );
simd_float3 clamped = min( gammaCorrected, makeFloat3( 0.999f, 0.999f, 0.999f ) );

outTexture.write( float4( clamped, 1.0 ), gid );
```

`sqrt` here is gamma-2 correction (the standard RTIOW convention), and the `0.999f` clamp keeps the written color strictly below 1.0. `makeFloat3()` is the same helper from `RayTraceCore.h` mentioned in Chapter 2's note that `simd_float3(x, y, z)` function-call construction doesn't compile as plain C++, even though it's fine in MSL — used here for consistency with the CPU-side call sites, even though this file, being pure MSL, could have used `float3(x, y, z)` directly.

The comment at line 7 that opens the file also notes this kernel is "one thread per pixel," dispatched 8×8 per threadgroup:

```
// RayTracerBench/GPU/GPURenderer.cpp:130–132
const MTL::Size threadsPerThreadgroup( 8, 8, 1 );
...
pEncoder->dispatchThreads( threadsPerGrid, threadsPerThreadgroup );
```

That call — computing `threadsPerGrid`, encoding the compute command, binding the six buffers to buffer indices 0–5 in the exact order this kernel signature expects, and reading back the timed result — is `GPURenderer`'s job and belongs to Chapter 5. What matters here is just that the kernel's bounds check at lines 22–23 (`if ( gid.x >= params.width || gid.y >= params.height ) return;`) exists *because* an 8×8 threadgroup grid, dispatched over an arbitrary image width and height, is rounded up to the next multiple of 8 in each dimension — some threads in the last row or column of threadgroups fall outside the actual image and must do nothing.

§5. Blit.metal: simple enough to need no file at all

`Blit.metal` has a completely different job: it doesn't compute any pixel colors from scratch, it draws an already-rendered texture onto the screen. Its header comment states its two jobs plainly:

```
// RayTracerBench/Shaders/Blit.metal:1–11
// Fullscreen textured-quad blit: draws ImageDisplayView's source texture (the CPU
renderer's
// RGBA8 output today; a GPU-rendered texture directly, once that exists) onto the view's
// CAMetalLayer drawable. Two triangles, no vertex buffer — positions/texCoords are baked
into
// constant arrays indexed by vertex_id.
//
// Also implements the magnifying-glass loupe: when active, fragments inside a circular
region
// around a lens center sample the source texture from a smaller, de-magnified region
instead of
// 1:1, producing a zoomed inset. MagnifierUniforms is compiled from a raw in-memory
source string
// (see ImageDisplayView.cpp's readShaderSource() comment — unlike Raytracer.metal, this
file has
// no local #includes, so that's fine), so its layout is hand-duplicated in
ImageDisplayView.cpp
// rather than shared via a header — KEEP IN SYNC if either side changes.
```

That last clause of the comment is the direct counterpart of §2's discussion: `Blit.metal` has no `#include "../Core/..."` anywhere in it — its only include is the standard `<metal_stdlib>` — so there is nothing a missing file location could break. That's what makes it safe to compile from an in-memory string, and `ImageDisplayView.cpp` (Chapter 10) does exactly that:

```
// RayTracerBench/App/ImageDisplayView.cpp:16-29
// RT_SHADERS_DIR is injected by CMakeLists.txt as the absolute path to
RayTracerBench/Shaders.
// Loading .metal source from disk at runtime (rather than a compiled-in default.metallib)
is a
// deliberate consequence of this project deliberately not being an app bundle (see
CLAUDE.md's
// "Command Line Tool template, not App template" decision) — Apple's own LearnMetalCPP
samples
// use the equivalent technique of compiling MSL from a source string via newLibrary()
rather
// than newDefaultLibrary(), just with the string embedded instead of read from a sibling
file.
std::string readShaderSource( const char* fileName )
{
        std::string   path = std::string( RT_SHADERS_DIR ) + "/" + fileName;
        std::ifstream file( path );
        std::stringstream buffer;
        buffer << file.rdbuf();
        return buffer.str();
}
```

```
// RayTracerBench/App/ImageDisplayView.cpp:57-60
std::string source = readShaderSource( "Blit.metal" );

NS::Error* pError = nullptr;
MTL::Library* pLibrary = _pDevice->newLibrary( NS::String::string( source.c_str(),
UTF8StringEncoding ), nullptr, &pError );
```

Note the two-step split: the file is still read from disk at the path `RT_SHADERS_DIR` (a plain `CMakeLists.txt`-injected compile definition pointing at `RayTracerBench/Shaders`, per `CMakeLists.txt:57-59`) so that editing `Blit.metal` doesn't require a rebuild of the host executable — but once the bytes are in hand, they're handed to Metal as a source *string*, not a source *path*. `newLibrary()` compiles that string at that moment, during `ImageDisplayView`'s constructor, rather than reading a precompiled `.metallib`. This is the `newLibrary(source, options, &error)` overload, distinct from the `newLibrary(filepath, &error)` overload `GPURenderer` uses for `Raytracer.metal` in §2. `Blit.metal`'s own vertex data needs no buffer either — the two triangles making up the fullscreen quad are baked directly into `constant` arrays and indexed by `vertex_id`:

```
// RayTracerBench/Shaders/Blit.metal:22-32
constant float2 kQuadPositions[ 6 ] = {
        float2( -1.0, -1.0 ), float2( 1.0, -1.0 ), float2( -1.0, 1.0 ),
        float2( -1.0, 1.0 ), float2( 1.0, -1.0 ), float2( 1.0, 1.0 )
};

// texCoord.y=0 at the top row: matches ImageDisplayView's source texture, which is
uploaded via
// replaceRegion() directly from a row-major, row-0-is-top RGBA8 buffer (see
CPURenderer.hpp).
constant float2 kQuadTexCoords[ 6 ] = {
        float2( 0.0, 1.0 ), float2( 1.0, 1.0 ), float2( 0.0, 0.0 ),
        float2( 0.0, 0.0 ), float2( 1.0, 1.0 ), float2( 1.0, 0.0 )
};
```

`blitVertex` (lines 46–52) does nothing but pick out one of these six position/texCoord pairs by `vertex_id` and pass it through to the rasterizer as `RasterizerData`; there is no transform, no camera, no projection — the quad already spans clip space `[-1, 1]` on both axes.

§6. The magnifying-glass loupe: circular, aspect-corrected, de-magnifying sample

`blitFragment` is where the loupe effect actually lives. Its uniform struct, `MagnifierUniforms`, is deliberately not shared via a header with `Core/ShaderTypes.h` — since `Blit.metal` is compiled from a source string at runtime rather than alongside a build-time include path, there's no convenient shared compile step for a small file like this one, so its layout is hand-duplicated on the host side in `ImageDisplayView.cpp` instead, with an explicit `KEEP IN SYNC` comment marking both copies:

```
// RayTracerBench/Shaders/Blit.metal:34-43
// KEEP IN SYNC with the identical struct in ImageDisplayView.cpp.
struct MagnifierUniforms
{
        float centerU;
        float centerV;
        float viewAspect; // width / height, so the lens reads as a circle rather than an
ellipse
        float radius;     // normalized, as a fraction of view height
        float zoom;
        int   active;     // 0 = disabled; draw the source texture unmodified
};
```

The fragment shader itself branches on `magnifier.active`, falling through to a plain 1:1 texture sample when the loupe isn't in use:

```
// RayTracerBench/Shaders/Blit.metal:56-86
fragment float4 blitFragment( RasterizerData in [[stage_in]],
        texture2d<float, access::sample> sourceTexture [[texture( 0 )]],
        constant MagnifierUniforms& magnifier [[buffer( 0 )]] )
{
        constexpr sampler linearSampler( coord::normalized, address::clamp_to_edge,
filter::linear );

        if ( magnifier.active != 0 )
        {
                float2 center = float2( magnifier.centerU, magnifier.centerV );
                float2 delta = in.texCoord - center;
                // Correct for aspect ratio: 1 unit of U spans `width` pixels but 1 unit
of V spans
                // `height` pixels, so scale U by viewAspect to compare both in "V-
equivalent" units —
                // otherwise the lens would read as an ellipse on a non-square view.
                float2 deltaScreen = float2( delta.x * magnifier.viewAspect, delta.y );
                float  dist = length( deltaScreen );

                if ( dist < magnifier.radius )
                {
                        float2 zoomedUV = center + delta / magnifier.zoom;
                        float4 color = sourceTexture.sample( linearSampler, zoomedUV );

                        // Thin white border ring so the lens's edge reads clearly against
any image content.
                        float ringWidth = magnifier.radius * 0.035;
                        float ring = smoothstep( magnifier.radius - ringWidth,
magnifier.radius, dist );
                        color.rgb = mix( color.rgb, float3( 1.0, 1.0, 1.0 ), ring * 0.85
);
                        return color;
                }
        }

        return sourceTexture.sample( linearSampler, in.texCoord );
}
```

Walking through the math: `delta` is the current fragment's texture coordinate offset from the lens center, in normalized `[0,1]` UV units. Because UV space treats U and V as running over the same `[0,1]` range regardless of the view's actual pixel width and height, a fixed-radius circle in raw UV distance would look like an ellipse on any non-square view — one unit of U movement covers `width` pixels, one unit of V movement covers `height` pixels. `deltaScreen` fixes this by scaling `delta.x` by `viewAspect` (`width / height`, computed once in `ImageDisplayView`'s constructor and stored in the uniforms), converting both axes into a common "V-equivalent" unit before measuring `dist = length(deltaScreen)`. Only then does comparing `dist` against `magnifier.radius` describe an actual circle on screen.

Inside that circle, `zoomedUV = center + delta / magnifier.zoom` is the de-magnification step: rather than sampling `in.texCoord` directly, the shader samples a point pulled *toward* the lens center by a factor of `zoom` — with `zoom = 4.0` (the default, per `ImageDisplayView.cpp:41`'s `_magnifier{ 0.5f, 0.5f, 1.0f, 0.18f, 4.0f, 0 }` initializer), a fragment one full lens-radius away from center samples a source texel only a quarter of a lens-radius away from center. That shrinkage is what makes the region inside the lens look like a 4x-zoomed inset of the texture around `center`, rather than the same image repeated at a different position.

The `ring` calculation is a cosmetic finishing touch, not part of the zoom math: `smoothstep(radius - ringWidth, radius, dist)` produces a value that ramps from 0 to 1 only in a thin band just inside the lens's edge (`ringWidth` is 3.5% of the radius), and `mix(color.rgb, white, ring * 0.85)` blends that band toward white — a border thin and subtle enough not to obscure the zoomed content, but visible enough that the lens's boundary reads clearly against any image behind it.

None of this — `centerU`/`centerV`, `active` — is computed by the shader itself; those come from `MagnifierUniforms` values the host side sets each frame from live mouse position, which is `ImageDisplayView`'s job (Chapter 10) via its `NS::EventMaskMouseMoved` local event monitor. What's covered here is only the shader-side geometry: given a center and a state, how the fragment shader decides which texel to actually draw.

§7. How the lens math was checked, not just written

`CLAUDE.md`'s project-status notes describe this feature as "verified three ways rather than assumed," which is worth restating here since it applies directly to the shader code just walked through:

1. **An offscreen shader test with a known 2px marker**, confirming exact 4x magnification with no positional drift — i.e., feeding `blitFragment` a source texture with a marker at a known location and confirming the zoomed output places that marker at exactly the position the `zoomedUV` formula above predicts, at exactly 4x scale.
2. **A coordinate-math test using production-matching window/view frames**, confirming that `NS::View::convertPoint()` (the window-to-view-local coordinate conversion done on the host side before the lens center ever reaches this shader) plus the AppKit-Y-up-to-texture-V-down flip compose correctly — this is a host-side concern, covered fully in Chapter 10, but it's the reason `centerU`/`centerV` can be trusted to mean what `blitFragment` assumes they mean.

3. **An end-to-end test through the real scene/CPURenderer/Blit.metal pipeline**, confirming the lens zooms into actual, recognizable, correctly-positioned scene detail — not just a synthetic test pattern.

The point of listing all three is that the shader math in §6 — the aspect-corrected circular distance test, the de-magnified sample offset — was checked against a known ground truth at each of three different levels (shader-only, coordinate-math-only, full pipeline) rather than written once and trusted by inspection.

Where this connects

- **Chapter 2** (`Core/RayTraceCore.h`) is what `Raytracer.metal` `#include`s verbatim at line 5, and is the reason this file exists in its current thin-wrapper form: every ray/hit/scatter/RNG function called from `renderKernel`'s sample loop is defined and explained there, not here.
- **Chapter 5** (`GPU/GPURenderer.hpp/.cpp`) is the host-side counterpart to both shaders in this chapter: it's what loads `Raytracer.metal`'s compiled `.metallib` via `newLibrary(filepath, ...)`, builds the compute pipeline state, binds the six buffers this chapter's §3 describes to buffer indices 0–5, and issues the 8×8 `dispatchThreads` call this chapter only gestures at.
- **Chapter 10** (`App/ImageDisplayView.hpp/.cpp`) is the host-side counterpart to `Blit.metal`: it compiles the shader from an in-memory string read from `RT_SHADERS_DIR`, builds the render pipeline state, defines the host-side twin of `MagnifierUniforms`, and drives the lens center via mouse tracking every frame — the piece of the loupe feature this chapter deliberately left to that chapter to cover in full.

Chapter 4: The CPU Renderer

Abstract. This chapter covers `CPU/CPURenderer.hpp` and `CPU/CPURenderer.cpp` — the plain-C++ half of RayTracerBench's dual-compile design. Chapter 2 showed `Core/RayTraceCore.h` written so it could be compiled twice; Chapter 3 showed one of those compiles, `Shaders/Raytracer.metal`, feeding it to the Metal compiler as a GPU kernel. This chapter is the other compile: the same header, `#include`d verbatim into an ordinary `.cpp` file, driven by a hand-rolled `std::thread` row-partitioning scheme instead of a GPU dispatch grid. Nothing in this file re-implements ray/sphere/pyramid intersection, material scattering, or path-trace bouncing — all of that already exists in Chapter 2's header. What's here is: turning a `SceneDescription` into a byte buffer, deciding how to split the work across CPU cores, and timing the result.

Files covered: `RayTracerBench/CPU/CPURenderer.hpp` (28 lines), `RayTracerBench/CPU/CPURenderer.cpp` (120 lines).

§1. The other half of the dual compile

`Shaders/Raytracer.metal` opens with two `#include`s: `ShaderTypes.h` and `RayTraceCore.h`, compiled as Metal Shading Language by `xcrun -sdk macosx metal` (Chapter 3, Chapter 12). `CPURenderer.cpp` opens the same way, compiled instead by whatever C++17 compiler Xcode invokes for the rest of the app:

```
// RayTracerBench/CPU/CPURenderer.cpp:1–6
#include "CPURenderer.hpp"

#include "../Core/RayTraceCore.h"

#include <algorithm>
#include <thread>
```

This single `#include "../Core/RayTraceCore.h"` line is the whole point of Chapter 2's design. `RayTraceCore.h` doesn't have a CPU version and a GPU version — it has one version, guarded by exactly one `#ifdef __METAL_VERSION__` (the `RT_DEVICE` address-space shim), and each of the two translation units that pull it in supplies the other half of the environment: `Raytracer.metal` supplies `#include <metal_stdlib>` and MSL's `float3` / `sqrt` built-ins; `CPURenderer.cpp` supplies `<simd/simd.h>` (via `ShaderTypes.h`) and `<cmath>`'s `std::sqrt` / `std::pow` / `std::fabs` / `std::fmin`, pulled into scope by

`RayTraceCore.h`'s own `using std::fabs; using std::fmin; using std::pow; using std::sqrt;` in its non-Metal branch. Neither file duplicates `hitSphere()`, `hitPyramid()`, `hitEntity()`, `scatter()`, or `rayColor()` — they call the identical inline functions, compiled twice by two different compilers. `CPURenderer.cpp` is framework-free pure C++17 apart from that: no AppKit, no CoreGraphics, no Metal headers — it doesn't even know a GPU exists in the process. That's deliberate; per `CLAUDE.md`'s architecture notes, the CPU renderer is kept framework-free so it stays a fair, unadorned CPU baseline rather than something the GPU-side plumbing could bias.

§2. The public interface: what `renderCPU()` takes and returns

Everything a caller needs is declared in the 28-line header. First, the threading mode is a two-value enum, not a boolean or a thread-count parameter:

```
// RayTracerBench/CPU/CPURenderer.hpp:9–16
namespace CPUThreading
{
        enum Mode
        {
                SingleThreaded,
                MultiThreaded,
        };
}
```

Then the result type — a pixel buffer plus a duration, nothing else:

```
// RayTracerBench/CPU/CPURenderer.hpp:18–22
struct CPURenderResult
{
        std::vector<uint8_t>                      pixels; // RGBA8, row-major, row 0 =
top
        std::chrono::duration<double, std::milli>  renderTime;
};
```

And the function itself:

```
// RayTracerBench/CPU/CPURenderer.hpp:24–28
// Renders `scene` on the CPU by calling the exact same Core/RayTraceCore.h functions the
GPU
// renderer will later call from its kernel. Default is MultiThreaded (realistic CPU
perf);
// SingleThreaded is exposed explicitly so a UI-facing speedup number is never ambiguous
about
// which CPU baseline it was measured against.
CPURenderResult renderCPU( const SceneDescription& scene, CPUThreading::Mode mode =
CPUThreading::MultiThreaded );
```

The signature is narrow on purpose: one `const SceneDescription&` in (the same struct Chapter 1 showed `buildDefaultScene()` producing, with no CPU-specific scene representation), one `CPURenderResult` out. There's no callback, no output parameter, no reference to an `ImageDisplayView` or an `NS::Alert` anywhere in this file — `renderCPU()` doesn't know or care who's calling it. That's what lets three different call sites in later chapters reuse it unchanged: `ControlsPanel` / `AppDelegate`'s `Render CPU` and `Compare` buttons (Chapter 8/9) call it on a background `std::thread` and marshal `pixels` / `renderTime` back to the UI via `dispatch_async`; and `Export/SceneExporter.hpp`'s preview-PNG path (Chapter 7) calls the exact same function on the exact same `SceneDescription` it just exported to glTF/OBJ, feeding `result.pixels` straight into `ImageWriter`'s `writePNG()` with no adapter code in between. The `pixels` / `renderTime` split also mirrors what the GPU renderer's public interface returns (Chapter 5), which is what lets `ResultsPanel` compute a speedup ratio without special-casing either side.

§3. Why the threading toggle is explicit, not automatic

It would be simpler for `renderCPU()` to always use `std::thread::hardware_concurrency()` threads and drop the `mode` parameter entirely. The header comment states directly why it doesn't: "**Default is MultiThreaded (realistic CPU perf); SingleThreaded is exposed explicitly so a UI-facing speedup number is never ambiguous about which CPU baseline it was measured against**" (`CPURenderer.hpp:26–27`). A benchmark's headline number is a ratio — GPU time divided by CPU time — and that ratio means something different depending on whether the CPU side used one core or all of them. A tool that silently always parallelizes the CPU path can still report an honest number, but it can never report the *other* number (single-core vs. GPU) without a code change, and a reader of the UI has no way to tell which one they're looking at. `CLAUDE.md`'s CPU-renderer notes make the same point: this toggle is surfaced in `ControlsPanel` precisely so the comparison is never ambiguous about its own baseline. `SingleThreaded` isn't a debug leftover or a fallback for

some platform limitation — it's a first-class, user-selectable mode whose entire reason to exist is to make the speedup number legible.

§4. Row-partitioning across `std::thread`s

The project note in `CLAUDE.md`'s CPU-renderer section is explicit that this threads via **plain `std::thread` row-partitioning — not GCD** — the same distinction drawn for why UI-completion marshaling uses `dispatch_async` while the actual render work doesn't. `renderCPU()` is the function that makes that choice concrete:

```
// RayTracerBench/CPU/CPURenderer.cpp:77-120
CPURenderResult renderCPU( const SceneDescription& scene, CPUThreading::Mode mode )
{
        const uint32_t width = scene.params.width;
        const uint32_t height = scene.params.height;

        CPURenderResult result;
        result.pixels.resize( static_cast<size_t>( width ) * height * 4 );

        const auto startTime = std::chrono::high_resolution_clock::now();

        if ( mode == CPUThreading::SingleThreaded || height == 0 )
        {
                renderRows( scene, result.pixels, 0, height );
        }
        else
        {
                unsigned threadCount = std::thread::hardware_concurrency();
                if ( threadCount == 0 )
                        threadCount = 4;
                threadCount = std::min<unsigned>( threadCount, height );

                std::vector<std::thread> threads;
                threads.reserve( threadCount );

                const uint32_t rowsPerThread = ( height + threadCount - 1 ) / threadCount;

                for ( unsigned t = 0; t < threadCount; ++t )
                {
                        const uint32_t rowStart = t * rowsPerThread;
                        const uint32_t rowEnd = std::min( rowStart + rowsPerThread, height
);
                        if ( rowStart >= rowEnd )
                                continue;
                        threads.emplace_back( renderRows, std::cref( scene ), std::ref(
result.pixels ), rowStart, rowEnd );
                }

                for ( std::thread& t : threads )
                        t.join();
        }

        const auto endTime = std::chrono::high_resolution_clock::now();
        result.renderTime = endTime - startTime;

        return result;
}
```

The shape of this is ordinary embarrassingly-parallel image work, but a few details are worth calling out because they're the kind of edge case that only shows up once: `height == 0` is folded into the `SingleThreaded` branch (`renderRows(scene, result.pixels, 0, 0)` is a no-

op loop, cheaper than standing up a thread pool for zero rows) rather than being a separate guard. `hardware_concurrency()` can return `0` on a platform that can't determine the figure, so a `4`-thread fallback keeps the multithreaded path from degenerating into zero threads and zero work. `threadCount` is then clamped to `height` (`std::min<unsigned>(threadCount, height)`) so a tiny image — say, a 3-row preview — never spins up more threads than there are rows to give them; every thread is guaranteed at least one row. `rowsPerThread` uses the `(height + threadCount - 1) / threadCount` ceiling-division idiom, so the last thread's `rowEnd` is clamped with `std::min(rowStart + rowsPerThread, height)` and can legitimately end up with a shorter band than the others (or, via the `if (rowStart >= rowEnd) continue;` guard, no rows at all, though the prior clamp against `height` makes that path unreachable in practice — it's defensive against future changes to the partitioning arithmetic). Timing wraps the whole dispatch-and-join block with `std::chrono::high_resolution_clock`, so `renderTime` measures wall-clock time for the full render including thread startup/teardown, not just the inner ray-tracing loop — the fair, literal answer to "how long did this take."

Each worker thread is handed the scene by `std::cref` and the shared pixel buffer by `std::ref`, both passed into `std::thread`'s constructor alongside the free function `renderRows` and that thread's own `[rowStart, rowEnd)` band. Because every thread writes only its own disjoint row range of `result.pixels`, there's no shared mutable state to synchronize and no lock anywhere in this file — the partitioning itself is the synchronization strategy.

§5. `renderRows()`: walking `transforms[]` / `shapes[]` in lockstep

The actual per-pixel work — the part that both the `SingleThreaded` and `MultiThreaded` paths funnel into — lives in an anonymous-namespace helper, `renderRows()`, which renders one contiguous band of rows into the shared `pixels` buffer:

```
// RayTracerBench/CPU/CPURenderer.cpp:19-32
void renderRows( const SceneDescription& scene, std::vector<uint8_t>& pixels, uint32_t
rowStart, uint32_t rowEnd )
{
        const uint32_t width = scene.params.width;
        const uint32_t height = scene.params.height;
        const uint32_t samplesPerPixel = scene.params.samplesPerPixel;
        const uint32_t maxDepth = scene.params.maxDepth;

        const TransformGPU* transforms = scene.transforms.data();
        const ShapeGPU*     shapes = scene.shapes.data();
        const uint32_t      entityCount = static_cast<uint32_t>( scene.transforms.size()
);
        const MaterialGPU*  materials = scene.materials.data();
```

This is exactly Chapter 1's `SceneDescription`: `transforms`, `shapes`, and `materials` are the three parallel ECS component arrays, and `.data()` here is the plain-pointer side of the same `RT_DEVICE` design Chapter 2 described — on the CPU these are ordinary `const TransformGPU*` / *`const ShapeGPU*`* / `const MaterialGPU*`, with no address-space annotation needed at all, whereas `Raytracer.metal`'s kernel arguments carry `device` because they arrive through a Metal buffer binding (`Shaders/Raytracer.metal:13-16`). Nothing else about the arrays differs. `entityCount` is just `scene.transforms.size()` cast down to `uint32_t` — there is no separate sphere-count/pyramid-count bookkeeping to keep straight, because spheres and pyramids already live interleaved in the same two arrays, distinguished only by `ShapeGPU::type`.

The body of the row/column loop is where those arrays actually get used, and it reads almost line-for-line like `renderKernel`'s per-thread body in `Raytracer.metal:25-52` — same seed derivation, same jittered-sample loop, same call into `getRay()` and `rayColor()`, just iterated explicitly over `i` and `j` instead of arriving as a `[[thread_position_in_grid]]`:

```
// RayTracerBench/CPU/CPURenderer.cpp:33-71
for ( uint32_t j = rowStart; j < rowEnd; ++j )
{
        for ( uint32_t i = 0; i < width; ++i )
        {
                // Distinct per-pixel seed derivation; only randomFloat()/pcgHash()
themselves need
                // to match the GPU renderer bit-for-bit, not this seeding formula (see
CLAUDE.md's
                // "eyeball correctness" note — per-pixel noise legitimately differs CPU
vs GPU).
                uint32_t seed = pcgHash( scene.params.frameSeed ^ ( j * 9781u + i * 6271u
+ 1u ) );

                simd_float3 colorSum = makeFloat3( 0.0f, 0.0f, 0.0f );

                for ( uint32_t s = 0; s < samplesPerPixel; ++s )
                {
                        RandomFloatSample ru = randomFloat( seed );
                        seed = ru.seed;
                        RandomFloatSample rv = randomFloat( seed );
                        seed = rv.seed;

                        float u = ( static_cast<float>( i ) + ru.value ) /
static_cast<float>( width - 1 );
                        float v = ( static_cast<float>( height - 1 - j ) + rv.value ) /
static_cast<float>( height - 1 );

                        CameraRaySample cameraRay = getRay( scene.camera, u, v, seed );
                        seed = cameraRay.rngSeed;

                        RayColorResult sample = rayColor( cameraRay.ray, transforms,
shapes, entityCount, materials, maxDepth, seed );
                        seed = sample.rngSeed;

                        colorSum = colorSum + sample.color;
                }

                simd_float3 averageColor = colorSum / static_cast<float>( samplesPerPixel
);

                const size_t pixelIndex = ( static_cast<size_t>( j ) * width + i ) * 4;
                pixels[ pixelIndex + 0 ] = toByte( averageColor.x );
                pixels[ pixelIndex + 1 ] = toByte( averageColor.y );
                pixels[ pixelIndex + 2 ] = toByte( averageColor.z );
                pixels[ pixelIndex + 3 ] = 255;
        }
}
```

`rayColor(cameraRay.ray, transforms, shapes, entityCount, materials, maxDepth, seed)` on line 57 is the entire "collision system plus shading" pipeline Chapter 2 documented in one call: internally, `rayColor()` (`Core/RayTraceCore.h:467-513`) loops `i` from `0` to

`entityCount`, calling `hitEntity(transforms[i], shapes[i], r, 0.001f, closestSoFar)` per entity to find the closest hit — the tagged dispatch to `hitSphere()` or `hitPyramid()` that stands in for a forbidden virtual `Hittable::hit()` call — and then hands the winning hit record to `scatter()` for tagged-switch material shading, bouncing up to `maxDepth` times. `renderRows()` itself never calls `hitEntity()` or `scatter()` directly, and doesn't need to know how many shape types exist or how materials are resolved; it only walks pixels and samples, and defers the entire "what does this ray hit and what color does it come back as" question to Chapter 2's shared function. That's the payoff of Chapter 2's design appearing concretely here: adding a third primitive type someday would mean touching `RayTraceCore.h`'s `hitEntity()` switch once, not this file.

The per-pixel RNG seed on line 40 is derived from `scene.params.frameSeed` mixed with the pixel coordinates via `pcgHash()` — but the exact mixing formula (`j *9781u + i* 6271u + 1u`) is local to this loop, not part of the shared core, and the comment says so directly: only `randomFloat()` / `pcgHash()` themselves need to match the GPU renderer bit-for-bit; this seeding arithmetic doesn't, which is exactly why CPU and GPU renders of the same scene produce visibly different per-pixel noise while still agreeing on geometry, materials, and overall lighting — `CLAUDE.md`'s "eyeball correctness" verification note this comment points at.

§6. Byte conversion: `toByte()`

The last piece is the free function that turns a linear, possibly-out-of-range float color channel into a displayable `uint8_t`:

```
// RayTracerBench/CPU/CPURenderer.cpp:10–17
// Gamma-2 correction (sqrt) plus the book's [0,0.999] clamp before quantizing to a byte,
so a
// component of exactly 1.0 rounds down to 255 rather than overflowing to 0.
uint8_t toByte( float c )
{
        float gammaCorrected = std::sqrt( std::max( 0.0f, c ) );
        float clamped = std::min( gammaCorrected, 0.999f );
        return static_cast<uint8_t>( 256.0f * clamped );
}
```

This is the CPU-side twin of the last three lines of `renderKernel()` in `Raytracer.metal:49–52` (`sqrt` / `max` / `min` / `clamped` there operate on a whole `simd_float3` at once via MSL's vector overloads and write straight into a `texture2d`, whereas `toByte()` here operates one channel at a time and writes into a plain `RGBA8` byte array) — same gamma-2 approximation (`sqrt`, standing in for the more expensive `pow(c, 1/2.2)`), same `[0, 0.999]` clamp before

quantizing so an exact `1.0` input rounds down to `255` instead of wrapping to `0` at `256.0f * 1.0f`. `toByte()` is the one place in this file where the CPU path's `RGBA8` output format and the GPU path's `MTL::Texture` output format visibly diverge — not because the tone-mapping math differs, but because `ImageDisplayView` (Chapter 10) consumes the two renderers' results through different upload paths (`replaceRegion` for the CPU buffer, direct texture write for the GPU kernel), and each renderer produces its native format for its own path rather than either one converting to match the other.

§7. What isn't here

It's worth noting what `CPURenderer.cpp` deliberately does *not* do, since the omissions are as much a design statement as the code that's present. There's no threading library beyond `<thread>` itself — no thread pool, no work queue, no GCD dispatch — because the workload here is a single, one-shot, statically-known partition (N rows into T threads, joined once), which doesn't need a general-purpose pool. There's no scene construction — `buildDefaultScene()` (Chapter 1) already ran by the time a `SceneDescription` reaches `renderCPU()`. And there's no UI awareness at all: no knowledge of `ControlsPanel`'s settings fields, no `NS::Alert`, no file I/O. That narrowness is what makes the same function usable, unmodified, from three different call sites across the rest of the app.

Where this connects

- **Chapter 2 (The Shared Ray-Tracing Core)** is the header this file compiles as its second target — every intersection, scatter, and RNG function `renderRows()` calls (`hitEntity()`, `scatter()`, `rayColor()`, `getRay()`, `pcgHash()`/`randomFloat()`) is defined there once and shared verbatim with the GPU kernel, not reimplemented here.
- **Chapter 1 (The Core Data Model)** defines the `SceneDescription` this file only ever reads from (`scene.transforms`, `scene.shapes`, `scene.materials`, `scene.camera`, `scene.params`) — `CPURenderer.cpp` never constructs or mutates a scene, only consumes one.
- **Chapter 3 (The Metal Shaders)** is the GPU side of the same dual-compile design: `Raytracer.metal`'s `renderKernel()` and this file's `renderRows()` run the same per-pixel algorithm (seed derivation aside) over the same `RayTraceCore.h`, just dispatched across a Metal thread grid instead of `std::thread` row bands.
- **Chapter 8 (The App Shell and Lifecycle)** shows `AppDelegate` calling `renderCPU()` on a background `std::thread` for the `Render CPU` and `Compare` actions, then marshaling

`CPURenderResult` back to the main thread via `dispatch_async` to update `ImageDisplayView`/`ResultsPanel`.

- **Chapter 7 (Scene Export)** shows `Export/SceneExporter.hpp`'s preview-PNG path calling this exact same `renderCPU()` against the scene it just exported to glTF/OBJ, then handing `result.pixels` to `Export/ImageWriter.hpp`'s `writePNG()` — the second, independent call site that this file's narrow, UI-agnostic interface makes possible without any export-specific code inside `CPURenderer.cpp` itself.

Chapter 5: The GPU Renderer

Abstract. `GPURenderer` is the GPU half of the CPU/GPU comparison the whole app exists to make. It is written directly against `metal-cpp` — raw `MTL::Device`, `MTL::Buffer`, `MTL::ComputePipelineState`, `MTL::Texture` — with no Objective-C++ and no MetalKit convenience layer. Its shape is set by two goals that pull in slightly different directions: the expensive, one-time costs (device setup, shader compilation, pipeline-state creation) must happen exactly once, at construction, so they never leak into a timed render; and every timed render must still be preceded by its own untimed warm-up dispatch, because even a *cached* pipeline pays some first-dispatch cost. This chapter walks the constructor, the buffer/texture setup, the dispatch itself, and the two timing numbers (wall-clock and GPU-only) that `GPURenderer::render()` hands back to the UI.

Files covered: `RayTracerBench/GPU/GPURenderer.hpp`, `RayTracerBench/GPU/GPURenderer.cpp`.

§1. Written directly against metal-cpp, no Objective-C++

`GPURenderer` includes exactly one Apple-side header, `<Metal/Metal.hpp>` — `metal-cpp`'s C++ header-only binding to the Metal framework — and nothing from AppKit or Foundation beyond what that header pulls in. Every type it touches (`MTL::Device`, `MTL::Buffer`, `MTL::CommandQueue`, `MTL::ComputePipelineState`, `MTL::ComputeCommandEncoder`, `MTL::Texture`, `MTL::TextureDescriptor`) is a `metal-cpp` wrapper around the Objective-C Metal API, called with ordinary C++ method syntax:

```
// RayTracerBench/GPU/GPURenderer.hpp:42-51
MTL::Device*               _pDevice;
MTL::CommandQueue*         _pCommandQueue;
MTL::ComputePipelineState* _pPipelineState;

MTL::Buffer*  _pTransformBuffer;
MTL::Buffer*  _pShapeBuffer;
MTL::Buffer*  _pMaterialBuffer;
MTL::Texture* _pOutputTexture;
uint32_t      _textureWidth;
uint32_t      _textureHeight;
```

This is consistent with the project's stated architecture goal (see `CLAUDE.md`, "What this project is"): zero hand-written `.mm` files, with an isolated Objective-C++ shim as the documented fallback only if a specific widget proves unworkable through `metal-cpp` / `metal-`

`cpp-extensions`. The GPU renderer never needed that fallback — everything it does (loading a library, building a pipeline, allocating buffers, encoding a compute pass) has a direct `metal-cpp` equivalent, unlike some of the AppKit widgets covered in later chapters.

§2. Device, queue, and pipeline: created once, cached for the object's lifetime

The constructor is the only place `GPURenderer` ever creates its `MTL::Device`-derived, expensive-to-build state. The header spells out why in its own class comment:

```
// RayTracerBench/GPU/GPURenderer.hpp:18-21
// Written directly against metal-cpp (no MetalKit). Device/queue/pipeline are created
once in the
// constructor and cached so shader-compile latency never pollutes a render's timing.
Every
// render() call does its own untimed warm-up dispatch before the timed one, so a single
call's
// gpuTimeMs is never inflated by cold-start effects.
```

The device itself is handed in from outside (retained, not owned from scratch) and stored for the object's whole lifetime, alongside the command queue and pipeline state built from it:

```
// RayTracerBench/GPU/GPURenderer.cpp:9-17
GPURenderer::GPURenderer( MTL::Device* pDevice )
        : _pDevice( pDevice->retain() )
        , _pTransformBuffer( nullptr )
        , _pShapeBuffer( nullptr )
        , _pMaterialBuffer( nullptr )
        , _pOutputTexture( nullptr )
        , _textureWidth( 0 )
        , _textureHeight( 0 )
{
```

The buffers and texture start out null and are not built here at all — they are rebuilt per-scene inside `render()` (§4), because their contents (and even their sizes) depend on the `SceneDescription` passed to each render call, whereas the device/queue/pipeline triad depends on nothing but the GPU hardware and the compiled shader, both fixed for the process's whole run. This is the same "cache what's expensive and scene-independent, rebuild what's cheap and scene-dependent" split that shows up again in `rebuildBuffers()`'s own comment (§4).

§3. Loading Raytracer.metallib as a real file, not a source string

Immediately after storing the device, the constructor loads the compiled shader library:

```
// RayTracerBench/GPU/GPURenderer.cpp:18-47
using NS::StringEncoding::UTF8StringEncoding;

// RT_RAYTRACER_METALLIB (injected by CMakeLists.txt) is a real compiled .metallib, not a
// source string like ImageDisplayView's Blit.metal — Raytracer.metal #includes
Core/ShaderTypes.h
// and Core/RayTraceCore.h verbatim, and a raw source string has no file location for
those
// relative #includes to resolve against. See CMakeLists.txt's comment for the build-time
step.
NS::Error*    pError = nullptr;
MTL::Library* pLibrary = _pDevice->newLibrary( NS::String::string( RT_RAYTRACER_METALLIB,
UTF8StringEncoding ), &pError );
if ( !pLibrary )
{
        std::fprintf( stderr, "GPURenderer: failed to load Raytracer.metallib: %s\n",
                pError ? pError->localizedDescription()->utf8String() : "unknown error" );
        std::abort();
}

MTL::Function* pKernelFn = pLibrary->newFunction( NS::String::string( "renderKernel",
UTF8StringEncoding ) );

_pPipelineState = _pDevice->newComputePipelineState( pKernelFn, &pError );
if ( !_pPipelineState )
{
        std::fprintf( stderr, "GPURenderer: failed to create compute pipeline state:
%s\n",
                pError ? pError->localizedDescription()->utf8String() : "unknown error" );
        std::abort();
}

pKernelFn->release();
pLibrary->release();

_pCommandQueue = _pDevice->newCommandQueue();
```

`newLibrary( filepath, &pError )` here is doing something categorically different from the sibling call in `ImageDisplayView`'s blit setup, which loads `Blit.metal` as an in-memory source string via `newLibrary( sourceString, ... )`. The comment names the reason directly: `Raytracer.metal` `#include`s `Core/ShaderTypes.h` and `Core/RayTraceCore.h` verbatim (the dual-compiled algorithm core that Chapter 2 covers), and those are relative `#include`s that only resolve against a real file location on disk — a source string handed to `newLibrary` has no such location. Chapter 3 covers `Raytracer.metal` itself and why it takes this path instead of Blit.metal's; Chapter 12 covers the `CMakeLists.txt`

`add_custom_command` that actually shells out to `xcrun -sdk macosx metal` / `metallib` at build time and injects the resulting path as the `RT_RAYTRACER_METALLIB` compile definition consumed above. Both failure paths here (`newLibrary` and `newComputePipelineState`) `abort()` rather than leaving a half-usable renderer around — a deliberate fail-fast choice matching the header comment's "Aborts on failure … rather than leaving a half-usable renderer."

Both the destructor and this constructor make retain/release symmetry explicit rather than relying on any RAII wrapper: `pKernelFn` and `pLibrary` are transient (only needed to build the pipeline state) and are released immediately after use, while `_pDevice`, `_pCommandQueue`, and `_pPipelineState` are held for the object's lifetime and released in the destructor:

```
// RayTracerBench/GPU/GPURenderer.cpp:49–63
GPURenderer::~GPURenderer()
{
        if ( _pOutputTexture )
                _pOutputTexture->release();
        if ( _pMaterialBuffer )
                _pMaterialBuffer->release();
        if ( _pShapeBuffer )
                _pShapeBuffer->release();
        if ( _pTransformBuffer )
                _pTransformBuffer->release();
        _pPipelineState->release();
        _pCommandQueue->release();
        _pDevice->release();
}
```

§4. Per-scene buffers: transforms, shapes, materials, mirroring the CPU renderer's raw pointers

`rebuildBuffers()` re-creates and re-uploads the three ECS component arrays every time `render()` is called, on the theory that a handful of `Shared` buffer allocations is cheap next to the one-time cost cached in §2:

```
// RayTracerBench/GPU/GPURenderer.cpp:65-89
void GPURenderer::rebuildBuffers( const SceneDescription& scene )
{
        // Recreated on every render() call rather than capacity-cached:
device/queue/pipeline are the
        // expensive, cache-worthy resources (shader compile latency); a few Shared-mode
buffer
        // allocations per on-demand render is negligible in comparison.
        if ( _pTransformBuffer )
                _pTransformBuffer->release();
        if ( _pShapeBuffer )
                _pShapeBuffer->release();
        if ( _pMaterialBuffer )
                _pMaterialBuffer->release();

        const size_t transformBytes = scene.transforms.size() * sizeof( TransformGPU );
        const size_t shapeBytes = scene.shapes.size() * sizeof( ShapeGPU );
        const size_t materialBytes = scene.materials.size() * sizeof( MaterialGPU );

        _pTransformBuffer = _pDevice->newBuffer( transformBytes,
MTL::ResourceStorageModeShared );
        std::memcpy( _pTransformBuffer->contents(), scene.transforms.data(),
transformBytes );

        _pShapeBuffer = _pDevice->newBuffer( shapeBytes, MTL::ResourceStorageModeShared );
        std::memcpy( _pShapeBuffer->contents(), scene.shapes.data(), shapeBytes );

        _pMaterialBuffer = _pDevice->newBuffer( materialBytes,
MTL::ResourceStorageModeShared );
        std::memcpy( _pMaterialBuffer->contents(), scene.materials.data(), materialBytes
);
}
```

These three buffers are the GPU-side counterpart of exactly the raw pointers `CPURenderer.cpp` pulls out of the same `SceneDescription` before its own per-row loop (Chapter 4):

```
// RayTracerBench/CPU/CPURenderer.cpp:27-30
const TransformGPU* transforms = scene.transforms.data();
const ShapeGPU*     shapes = scene.shapes.data();
const uint32_t      entityCount = static_cast<uint32_t>( scene.transforms.size() );
const MaterialGPU*  materials = scene.materials.data();
```

On the CPU side those are plain pointers into `std::vector` storage passed straight into `rayColor()`. On the GPU side the same three arrays have to cross into `device`-address-space buffers a compute kernel can read, which is exactly what `rebuildBuffers()`'s `newBuffer` + `memcpy` pairs do — one `MTL::Buffer` per component array, byte-for-byte the same `TransformGPU` / `ShapeGPU` / `MaterialGPU` layouts `ShaderTypes.h` defines (Chapter 1),

so no translation step sits between the CPU's `SceneDescription` and the buffers the kernel actually reads.

`rebuildTextureIfNeeded()` handles the fourth resource, the output texture, with a different caching policy than the buffers above — it *is* cached across calls, and only rebuilt when the requested width/height actually change (the same "reuse resource, replace ImageDisplayView's contents" economy `ImageDisplayView::updatePixels()` applies to its own source texture, covered in Chapter 10):

```
// RayTracerBench/GPU/GPURenderer.cpp:91-110
void GPURenderer::rebuildTextureIfNeeded( uint32_t width, uint32_t height )
{
        if ( _pOutputTexture && width == _textureWidth && height == _textureHeight )
                return;

        if ( _pOutputTexture )
                _pOutputTexture->release();

        MTL::TextureDescriptor* pDesc = MTL::TextureDescriptor::texture2DDescriptor(
MTL::PixelFormatRGBA8Unorm, width, height, false );
        // Shared (not Private): ImageDisplayView blits it directly on Apple Silicon's
unified memory,
        // and the deterministic-parity check reads it back on the CPU via getBytes() —
both need it
        // to not be Private.
        pDesc->setStorageMode( MTL::StorageModeShared );
        pDesc->setUsage( MTL::TextureUsageShaderWrite | MTL::TextureUsageShaderRead );

        _pOutputTexture = _pDevice->newTexture( pDesc );
        _textureWidth = width;
        _textureHeight = height;
}
```

§5. `MTL::ResourceStorageModeShared`: unified memory, no explicit copy-back

Every allocation `GPURenderer` makes — the three component buffers in §4 and the output texture just above — is created in `Shared` storage mode, never `Private`. On Apple Silicon's unified memory architecture, `Shared` means the CPU and GPU see the same physical memory, so there is no explicit copy-back step after the kernel writes its results: the app reads `_pOutputTexture`'s contents (via `ImageDisplayView`'s blit, or via `getBytes()` in a parity check) the moment the command buffer signals completion, with no `MTL::BlitCommandEncoder` synchronizing a private GPU buffer back to host-visible memory in between. The texture descriptor's comment makes the two consumers that specifically require `Shared` (rather than `Private`) explicit: `ImageDisplayView` blits the texture directly,

and the deterministic-parity tests (Chapter 11) read it back on the CPU via `getBytes()` — both need direct CPU visibility into what the GPU wrote.

§6. Dispatching the kernel: buffers 0–5 and an 8×8 threadgroup

`dispatchOnce()` is where a single compute pass over the whole image actually gets encoded, bound, and dispatched:

```
// RayTracerBench/GPU/GPURenderer.cpp:112-139
// Encodes and dispatches one compute pass over the whole image, waits for completion, and
returns
// the GPU-only elapsed time in milliseconds (from the command buffer's own timestamps).
double GPURenderer::dispatchOnce( const SceneDescription& scene )
{
        uint32_t entityCount = static_cast<uint32_t>( scene.transforms.size() );

        MTL::CommandBuffer*         pCommandBuffer = _pCommandQueue->commandBuffer();
        MTL::ComputeCommandEncoder* pEncoder = pCommandBuffer->computeCommandEncoder();

        pEncoder->setComputePipelineState( _pPipelineState );
        pEncoder->setBuffer( _pTransformBuffer, 0, 0 );
        pEncoder->setBuffer( _pShapeBuffer, 0, 1 );
        pEncoder->setBytes( &entityCount, sizeof( entityCount ), 2 );
        pEncoder->setBuffer( _pMaterialBuffer, 0, 3 );
        pEncoder->setBytes( &scene.camera, sizeof( CameraGPU ), 4 );
        pEncoder->setBytes( &scene.params, sizeof( RenderParams ), 5 );
        pEncoder->setTexture( _pOutputTexture, 0 );

        const MTL::Size threadsPerThreadgroup( 8, 8, 1 );
        const MTL::Size threadsPerGrid( scene.params.width, scene.params.height, 1 );
        pEncoder->dispatchThreads( threadsPerGrid, threadsPerThreadgroup );
        pEncoder->endEncoding();

        pCommandBuffer->commit();
        pCommandBuffer->waitUntilCompleted();

        return ( pCommandBuffer->GPUEndTime() - pCommandBuffer->GPUStartTime() ) * 1000.0;
}
```

The six `setBuffer` / `setBytes` calls at indices 0–5 line up argument for argument with `renderKernel`'s own parameter list in `Shaders/Raytracer.metal` (Chapter 3):

```
// RayTracerBench/Shaders/Raytracer.metal:12-18
kernel void renderKernel(
        device const TransformGPU* transforms [[buffer( 0 )]],
        device const ShapeGPU*     shapes [[buffer( 1 )]],
        constant uint32_t&         entityCount [[buffer( 2 )]],
        device const MaterialGPU*  materials [[buffer( 3 )]],
        constant CameraGPU&        camera [[buffer( 4 )]],
        constant RenderParams&     params [[buffer( 5 )]],
```

Buffers 0, 1, and 3 are the `MTL::Buffer*` objects `rebuildBuffers()` built in §4, bound with `setBuffer`. `entityCount`, `camera`, and `params` are each small, fixed-size values rather than persistent allocations, so they go in via `setBytes` (index 2, 4, 5) instead — Metal copies the bytes inline for the encoder rather than requiring a backing `MTL::Buffer` the caller has to manage. `entityCount` mirrors the same `scene.transforms.size()` count the CPU renderer computes locally in §4's quoted snippet; `camera` and `params` are `scene.camera` / `scene.params` verbatim, the same `CameraGPU` / `RenderParams` structs `Scene.hpp` builds once per scene (Chapter 1). The output texture is bound separately via `setTexture`, since it is the kernel's write target rather than one of its `device` / `constant` read-only arguments.

The dispatch itself uses `dispatchThreads:threadsPerThreadgroup:` with an 8×8 threadgroup and a grid sized exactly to the image (`width` × `height` × 1) — one GPU thread per output pixel, tiled into 8×8 groups, with no manual padding or bounds math needed in the kernel because `dispatchThreads` (rather than the older `dispatchThreadgroups:threadsPerThreadgroup:`) handles a grid size that isn't an exact multiple of the threadgroup size correctly on its own.

After `commit()` and a blocking `waitUntilCompleted()`, the function returns `GPUEndTime() - GPUStartTime()` in milliseconds — the command buffer's own hardware-reported timestamps for this one dispatch, which is what makes it possible to time *just* the kernel's execution, isolated from CPU-side encoding overhead, queue latency, or the `waitUntilCompleted()` call itself.

§7. Two timing numbers, and the warm-up dispatch that keeps them honest

`render()` is the public entry point, and it calls `dispatchOnce()` twice — once thrown away, once timed:

```
// RayTracerBench/GPU/GPURenderer.cpp:141-159
// Rebuilds this scene's buffers/texture, does one untimed warm-up dispatch, then one
timed
// dispatch; returns the output texture plus wall-clock and GPU-only timings.
GPURenderResult GPURenderer::render( const SceneDescription& scene )
{
        rebuildBuffers( scene );
        rebuildTextureIfNeeded( scene.params.width, scene.params.height );

        dispatchOnce( scene ); // untimed warm-up — see header comment

        const auto startTime = std::chrono::high_resolution_clock::now();
        const double gpuTimeMs = dispatchOnce( scene );
        const auto endTime = std::chrono::high_resolution_clock::now();

        GPURenderResult result;
        result.pTexture = _pOutputTexture;
        result.wallClockTime = endTime - startTime;
        result.gpuTimeMs = gpuTimeMs;
        return result;
}
```

Note that the pipeline state, command queue, and device are already cached from construction (§2) by the time `render()` runs at all — so the warm-up dispatch here is not compensating for shader-compile latency (that cost was already paid once, at app launch, and never repeats). It exists because even a fully-cached pipeline still has some GPU/driver first-dispatch cost on a freshly rebuilt set of buffers and a freshly (re)bound texture, and `CLAUDE.md`'s "Benchmark validity" section is explicit that a warm-up dispatch should always precede a timed run, and that each configuration should really be run multiple times with the minimum reported — this per-`render()`-call warm-up is the mechanism inside `GPURenderer` that keeps a single call's `gpuTimeMs` from being skewed by that first-dispatch effect, independent of whatever repeat/min-of-3 policy the caller layers on top.

`GPURenderResult` carries both numbers out to the caller, with the class comment already flagging which one matters more:

```
// RayTracerBench/GPU/GPURenderer.hpp:10-16
struct GPURenderResult
{
        // Not owned by the caller; valid until the next render() call on this
GPURenderer.
        MTL::Texture* pTexture;
        std::chrono::duration<double, std::milli> wallClockTime;
        double gpuTimeMs; // CommandBuffer::GPUEndTime() - GPUStartTime(); the headline
metric
};
```

`wallClockTime` is measured with `std::chrono::high_resolution_clock` around the single timed `dispatchOnce()` call — it includes command-buffer encoding, `commit()`, and the CPU-side wait for `waitUntilCompleted()` to return, so it is closer to what a user experiences (the interval between "render started" and "results are back") but can be inflated by CPU-side scheduling noise unrelated to the GPU's own work. `gpuTimeMs`, by contrast, comes straight from the command buffer's own `GPUStartTime()` / `GPUEndTime()` hardware timestamps (§6) and reflects only the time the GPU itself spent executing the kernel — which is why it, not `wallClockTime`, is documented as "the headline metric" both here and in `ResultsPanel`'s presentation of it (Chapter 9).

§8. What is *not* here: the private-implementation macros

Unlike `App/ImageDisplayView.cpp`, `GPURenderer.cpp` defines none of Metal/QuartzCore's *`_PRIVATE_IMPLEMENTATION`* *macros — it only* *`#include`*`s` *`GPURenderer.hpp`, which pulls in* *`<Metal/Metal.hpp>`* *without instantiating the private implementation tables those macros switch on. That's deliberate, not an oversight: each such macro may be defined in only the one* *`.cpp`* *that first needs the private, non-header-only bodies for its corresponding framework's classes, and this project puts* *`MTL_PRIVATE_IMPLEMENTATION`* */* *`CA_PRIVATE_IMPLEMENTATION`* *in* *`ImageDisplayView.cpp`* *(with* *`NS_PRIVATE_IMPLEMENTATION`* *separately in* *`main.cpp`, *next to the* *`AppKit/AppKit.hpp`* *include it belongs with). Chapter 10 covers that placement rule and the linker errors (undefined* *`Private::Class`* */* *`Private::Selector`* *symbols) that motivate it in full;* *`GPURenderer.cpp`* *simply relies on whichever translation unit already defined* *`MTL_PRIVATE_IMPLEMENTATION`* *for the whole link, the same way any other* *`.cpp`* *in the app that merely* uses* `MTL::` types does.

Where this connects

- **Chapter 1** (`Core/ShaderTypes.h`, `Core/Scene.hpp/.cpp`) defines the `TransformGPU` / `ShapeGPU` / `MaterialGPU` / `CameraGPU` / `RenderParams` structs that `rebuildBuffers()` copies byte-for-byte into GPU buffers, and builds the `SceneDescription` that `GPURenderer::render()` takes as its only input.
- **Chapter 2** (`Core/RayTraceCore.h`) is the algorithm `renderKernel` actually runs per pixel — `GPURenderer` never touches ray-tracing logic itself, only the plumbing (buffers, dispatch, timing) around a kernel whose body is this shared, dual-compiled core.
- **Chapter 3** (`Shaders/Raytracer.metal`, `Shaders/Blit.metal`) is the `.metal` source `renderKernel` lives in, including why it must be compiled to a real `.metallib` rather than loaded from an in-memory string the way `Blit.metal` is (§3 above).

- **Chapter 4** (`CPU/CPURenderer.hpp/.cpp`) runs the same algorithm over the same `transforms`/`shapes`/`entityCount`/`materials` arrays as raw C++ pointers instead of Metal buffers (§4 above) — the two renderers are the two arms of the comparison this whole app exists to make.
- **Chapter 12** (`CMakeLists.txt`) is where `RT_RAYTRACER_METALLIB` is actually produced, via the `xcrun -sdk macosx metal`/`metallib` `add_custom_command` referenced in §3.
- **Chapters 8–9** (`App/AppDelegate.hpp/.cpp`, `App/ControlsPanel.hpp/.cpp`, `App/ResultsPanel.hpp/.cpp`) are where `Render GPU` and `Compare` actually construct/invoke a `GPURenderer` on a background thread and where `GPURenderResult::gpuTimeMs`/`wallClockTime` end up displayed as the GPU-only and wall-clock numbers in `ResultsPanel`.

Chapter 6: Mesh Generation for Export

Abstract. `Core/RayTraceCore.h` (Chapter 2) already knows how to answer two questions about an entity's Transform and Shape components: "does this ray hit it?" (`hitEntity()`) and "how does light scatter off it?" (`scatter()`). This chapter covers a third question asked of the exact same two components: "what triangle mesh does this entity look like?" `Export/EntityMesh.hpp/.cpp` answers it with `buildEntityMesh()`, a function with no ray in sight — it exists purely to hand `Export/SceneExporter.hpp/.cpp` (Chapter 7) real geometry to write into a glTF or OBJ file. The interesting design decisions here are architectural (this is the same tagged-dispatch ECS pattern doing a third job, not a new idea), geometric (spheres need approximating, pyramids don't — and reuse the same 5 local vertices Chapter 2's `hitPyramidLocal()` already derived), and methodological (triangle winding was hand-derived and then caught wrong by a test before it ever shipped).

Files covered: `RayTracerBench/Export/EntityMesh.hpp`, `RayTracerBench/Export/EntityMesh.cpp`. `RayTracerBenchTests/EntityMeshTests.cpp` is cited as the verification for §5 below and covered fully in Chapter 11.

---

§1. A third ECS system over the same two components

Chapter 2 introduced `hitEntity()` as "the collision system": it takes an entity's `TransformGPU` and `ShapeGPU` components, switches on `ShapeGPU::type`, and dispatches to `hitSphere()` or `hitPyramid()`. The project status notes in `CLAUDE.md` describe `scatter()`'s tagged switch over `MaterialGPU::type` the same way — a second system, dispatching on a different component, doing a different job (shading instead of intersection). `buildEntityMesh()` is explicitly the third member of that family, and the header says so directly:

```
// Builds a world-space mesh for one entity's Transform+Shape components, dispatching on
the Shape
// component's tag — the mesh-export analogue of RayTraceCore.h's hitEntity() intersection
system:
// same ECS dispatch-by-tag idea (see CLAUDE.md), a different "system" operating over the
same
// components. Spheres are tessellated (no native sphere primitive in glTF/OBJ); pyramids
are
// exact, faceted geometry (flat per-triangle normals — see buildPyramidMesh's winding-
order note).
MeshData buildEntityMesh( TransformGPU transform, ShapeGPU shape );
```

`RayTracerBench/Export/EntityMesh.hpp:17–22`

The implementation's own dispatch function reads almost like a restatement of `hitEntity()` with the ray parameters removed:

```
// Dispatches on the Shape component's tag to build that entity's mesh — the export-time
"system"
// alongside RayTraceCore.h's hitEntity() (intersection) and scatter() (shading).
MeshData buildEntityMesh( TransformGPU transform, ShapeGPU shape )
{
        switch ( shape.type )
        {
                case SHAPE_SPHERE:
                        return buildSphereMesh( transform.position, shape.radius );
                case SHAPE_PYRAMID:
                        return buildPyramidMesh( transform, shape.baseHalfWidth,
shape.height );
        }
        return MeshData{};
}
```

`RayTracerBench/Export/EntityMesh.cpp:144–156`

Compare this against `hitEntity()` itself (Chapter 2):

```
inline HitResult hitEntity( TransformGPU transform, ShapeGPU shape, Ray r, float tMin,
float tMax )
{
        switch ( shape.type )
        {
                case SHAPE_SPHERE:
                        return hitSphere( transform, shape, r, tMin, tMax );
                case SHAPE_PYRAMID:
                        return hitPyramid( transform, shape, r, tMin, tMax );
        }
        HitResult miss;
        miss.hit = false;
        return miss;
}
```

`RayTracerBench/Core/RayTraceCore.h:385–397`

The shape of the two functions is identical: switch on `shape.type`, dispatch to one of two per-shape helpers, fall through to a harmless default if the tag is ever something else. This is the payoff of `ShaderTypes.h`'s ECS design (Chapter 1) actually being followed through rather than just used once. The comment on `ShapeGPU` itself frames adding a primitive as "one more `SHAPE_` *tag and one more field group… dispatched by a switch… rather than a new virtual base class*" (*`RayTracerBench/Core/ShaderTypes.h:49–52`*) — *and that promise now has to hold for*

three* switches, not one, every time a new primitive is added: one in `hitEntity()`, one in `scatter()`'s material counterpart, and one here. `buildEntityMesh()` is the proof that the pattern generalizes past the two cases it was designed against.

§2. Framework-free, and why that matters here specifically

`EntityMesh.cpp` opens with a comment placing it alongside `Core/Scene.cpp`:

```
// Pure host-side C++ (never compiled as MSL), like Core/Scene.cpp — free to use the full
simd_*
// C API directly instead of RayTraceCore.h's hand-rolled portable helpers.
```

`RayTracerBench/Export/EntityMesh.cpp:5–6`

Unlike `RayTraceCore.h`, this file is never dual-compiled into MSL — there is no GPU-side use for a triangle mesh, since neither renderer draws triangles; they both ray-march the analytic sphere/pyramid primitives directly. That frees `EntityMesh.cpp` to call `simd_make_float3`, `simd_normalize`, and `simd_cross` straight from Apple's `<simd/simd.h>`, the same C API `Core/Scene.cpp` uses (Chapter 1), rather than routing through `RayTraceCore.h`'s `makeFloat3()` / `normalize3()` / `dot3()` helpers that exist only to paper over MSL's stricter constructor rules.

`CMakeLists.txt` backs up the "framework-free" claim structurally, not just in prose: `EntityMesh.cpp` is compiled into the same static library as `Core/Scene.cpp` and `CPU/CPURenderer.cpp`, not the `RayTracerBench` app target:

```
# RayTraceCore.h and ShaderTypes.h are header-only and dual-compiled into
# Shaders/Raytracer.metal directly (not through this library) once the GPU
# renderer exists. CPURenderer.cpp is framework-free C++17 like Core/, so it
# lives in the same static library rather than a separate CMake target.
add_library(RayTracerCore STATIC
    RayTracerBench/Core/Scene.cpp
    RayTracerBench/CPU/CPURenderer.cpp
    RayTracerBench/Export/EntityMesh.cpp
    RayTracerBench/Export/SceneExporter.cpp
)
```

`CMakeLists.txt:20–29`

`Export/ImageWriter.cpp` — the sibling module that actually needs CoreGraphics/ImageIO to write PNG bytes (Chapter 7) — is conspicuously *not* in that list; it's built only into the `RayTracerBench` executable target further down the same file (`CMakeLists.txt:37–46`), alongside the AppKit and Metal sources. `EntityMesh.cpp` needing neither Metal nor AppKit

nor even CoreGraphics is what earns it a place in `RayTracerCore` instead: it can be linked into `RayTracerBenchTests` (Chapter 11) and exercised as a plain command-line C++ program, with no GPU device, no window server, and no macOS UI session required to run its tests — which is exactly the environment this project's test binary runs in (see `CLAUDE.md`'s note that `RayTracerBenchTests` is "a plain command-line C++ tool"). A module that needed Metal or AppKit to construct a mesh would have dragged that dependency into every test run for no reason the mesh-generation logic itself needs.

§3. Sphere tessellation: a UV grid standing in for a primitive neither format has

Both glTF and OBJ describe geometry as triangle meshes; neither has a native analytic sphere. `buildSphereMesh()` approximates one with a latitude/longitude grid, at a fixed resolution chosen as a deliberate size/quality tradeoff (the comment cites the ~480-sphere randomized field as the dominant cost, per `CLAUDE.md`):

```
// Tessellation density for exported spheres: no native sphere primitive exists in either
// glTF or OBJ, so every sphere (including the giant ground sphere) is approximated as a
UV
// grid. Chosen as a size/smoothness balance after estimating the full scene's export size
at
// a few candidate densities (see CLAUDE.md) — the ~480-sphere randomized field dominates
// total triangle count, so doubling this from 8x12 would roughly double the file size for
a
// visually marginal smoothness gain at typical viewing distance.
constexpr int kSphereLatSegments = 8;
constexpr int kSphereLonSegments = 12;
```

`RayTracerBench/Export/EntityMesh.cpp:12–19`

The grid itself walks latitude from the south pole to the north pole (`theta` from `−π/2` to `+π/2`) and longitude all the way around (`phi` from `0` to `2π`), generating one vertex per `(lat, lon)` pair with an outward radial normal equal to its own position direction — this is what makes the sphere *smooth*-shaded rather than faceted, unlike the pyramid in §4:

```
for ( int lat = 0; lat <= latN; ++lat )
{
        float theta = -kPi / 2.0f + kPi * (float)lat / (float)latN; // -90..+90 degrees
        float sinTheta = std::sin( theta );
        float cosTheta = std::cos( theta );

        for ( int lon = 0; lon <= lonN; ++lon )
        {
                float       phi = 2.0f * kPi * (float)lon / (float)lonN;
                simd_float3 dir = simd_make_float3( cosTheta * std::cos( phi ), sinTheta,
cosTheta * std::sin( phi ) );
                mesh.normals.push_back( dir );
                mesh.positions.push_back( center + radius * dir );
        }
}
```

`RayTracerBench/Export/EntityMesh.cpp:32-45`

§4. The pole-row skip

A naive UV grid turns each `(lat, lon)` quad into two triangles by connecting its four corners. That works everywhere except the two end rows. At `lat == 0`, every vertex at every longitude sits at the same point — the south pole — because `theta = -π/2` collapses `cosTheta` to zero regardless of `phi`; symmetrically, at `lat == latN` every longitude collapses to the north pole. A quad that touches one of those rows therefore has two of its four corners identical in world space, and the triangle built from *both* collapsed corners has zero area — three vertices, two of which are the same point, is a degenerate triangle that contributes nothing to the rendered mesh but does bloat the file and can confuse loaders that assume non-degenerate input.

`buildSphereMesh()` handles this not by special-casing the pole rows separately, but by observing that each grid quad emits *two* triangles sharing a diagonal, and only one of those two ever touches both collapsed corners at once:

```
// Two triangles per grid quad; winding chosen so cross(v10-v00, v11-v00) etc. point
// outward (verified in RayTracerBenchTests/EntityMeshTests.cpp against a known point). At
// lat==0 every longitude collapses to the south pole (v00==v01), and at lat==latN-1 every
// longitude collapses to the north pole (v10==v11) — the triangle that would use both
// collapsed vertices is zero-area, so it's skipped rather than emitted as useless
// degenerate geometry; the other triangle in that quad (touching the pole only once) is
// real and still emitted.
for ( int lat = 0; lat < latN; ++lat )
{
        for ( int lon = 0; lon < lonN; ++lon )
        {
                uint32_t v00 = vertIndex( lat, lon );
                uint32_t v01 = vertIndex( lat, lon + 1 );
                uint32_t v10 = vertIndex( lat + 1, lon );
                uint32_t v11 = vertIndex( lat + 1, lon + 1 );

                if ( lat != latN - 1 ) // degenerate here: v10 == v11 (north pole)
                {
                        mesh.indices.push_back( v00 );
                        mesh.indices.push_back( v10 );
                        mesh.indices.push_back( v11 );
                }
                if ( lat != 0 ) // degenerate here: v00 == v01 (south pole)
                {
                        mesh.indices.push_back( v00 );
                        mesh.indices.push_back( v11 );
                        mesh.indices.push_back( v01 );
                }
        }
}
```

`RayTracerBench/Export/EntityMesh.cpp:49-78`

Walk it through concretely. At the very bottom row, `lat == 0`: `v00` and `v01` are both the south pole (`v00 == v01`, since every `lon` maps to the same collapsed point at that latitude), while `v10`/`v11` are two distinct, real vertices one latitude ring up. The triangle `(v00, v10, v11)` uses only one copy of the pole and two genuinely distinct points — that's a real, non-degenerate triangle, and it's the one the `if ( lat != latN - 1 )` branch emits (that condition is unrelated to this row; it's guarding the *other* pole at the top). The triangle `(v00, v11, v01)`, by contrast, would use `v00` and `v01` — the same point twice — plus `v11`; that's the zero-area triangle, and the `if ( lat != 0 )` guard skips it exactly at `lat == 0`. The symmetric argument holds at the top row, `lat == latN - 1`, where `v10 == v11` is now the collapsed pair and it's the *first* `if` that guards against emitting it. Each pole ring therefore contributes exactly one real triangle per longitude segment instead of two, and no triangle with a repeated vertex ever reaches `mesh.indices`.

§5. Pyramids: one geometric truth feeding two systems

Where a sphere needs approximating, a pyramid does not — `hitPyramidLocal()` in Chapter 2 already has an *exact* representation of the pyramid's solid as the intersection of five half-spaces (one base, four sides), each derived in closed form from `baseHalfWidth` / `height`:

```
const simd_float3 normals[ 5 ] = {
        makeFloat3( 0.0f, -1.0f, 0.0f ),        // base
        makeFloat3( height, halfWidth, 0.0f ),  // +X side
        makeFloat3( -height, halfWidth, 0.0f ), // -X side
        makeFloat3( 0.0f, halfWidth, height ),  // +Z side
        makeFloat3( 0.0f, halfWidth, -height ), // -Z side
};
```

`RayTracerBench/Core/RayTraceCore.h:292–298`

`buildPyramidMesh()` doesn't re-derive that geometry from scratch or tessellate an approximation of it — it builds its mesh from the *same* five local points (apex plus four base corners) that those plane equations implicitly define, restated explicitly as vertex positions this time instead of half-space normals:

```
// An exact (not tessellated) faceted mesh: the same 5 local vertices (apex + 4 base
corners,
// in the same corner order C0..C3) RayTraceCore.h's hitPyramidLocal() derives its 5 half-
space
// planes from, transformed to world space via the Transform component's orthonormal basis
// (world = position + x*right + y*up + z*forward — the same convention hitPyramid()
uses).
//
// Winding order for the 4 side faces, triangle(C[i], C[(i+1)%4], apex), was derived by
hand
// (cross(C[i+1]-C[i], apex-C[i]) checked against each face's known outward normal from
// RayTraceCore.h's plane-equation comment — e.g. the +X face's (height, halfWidth, 0))
and is
// re-verified in RayTracerBenchTests/EntityMeshTests.cpp rather than trusted by
inspection
// alone. The base is split into triangles (C0,C2,C1) and (C0,C3,C2), whose outward normal
is
// (0,-1,0) by the same derivation.
MeshData buildPyramidMesh( TransformGPU transform, float baseHalfWidth, float height )
{
        MeshData mesh;

        simd_float3 localApex = simd_make_float3( 0.0f, height, 0.0f );
        simd_float3 localCorners[ 4 ] = {
                simd_make_float3( baseHalfWidth, 0.0f, baseHalfWidth ),
                simd_make_float3( baseHalfWidth, 0.0f, -baseHalfWidth ),
                simd_make_float3( -baseHalfWidth, 0.0f, -baseHalfWidth ),
                simd_make_float3( -baseHalfWidth, 0.0f, baseHalfWidth ),
        };

        auto toWorld = [ & ]( simd_float3 local ) {
                return transform.position + local.x * transform.right + local.y *
transform.up + local.z * transform.forward;
        };

        simd_float3 apex = toWorld( localApex );
        simd_float3 corners[ 4 ];
        for ( int i = 0; i < 4; ++i )
                corners[ i ] = toWorld( localCorners[ i ] );

        for ( int i = 0; i < 4; ++i )
                addFlatTriangle( mesh, corners[ i ], corners[ ( i + 1 ) % 4 ], apex );

        addFlatTriangle( mesh, corners[ 0 ], corners[ 2 ], corners[ 1 ] );
        addFlatTriangle( mesh, corners[ 0 ], corners[ 3 ], corners[ 2 ] );

        return mesh;
}
```

`RayTracerBench/Export/EntityMesh.cpp:102-141`

This is deliberately "one geometric truth, two systems": `hitPyramidLocal()` answers "does a ray cross this half-space boundary" using the plane's normal and distance, while `buildPyramidMesh()` answers "what does this solid's surface look like as triangles" using the same corners those planes pass through — the pyramid's shape is defined exactly once (as `baseHalfWidth` and `height` in local space), and both the intersection system and the export system read off that single definition rather than each carrying its own copy that could drift out of sync. The world-space transform is likewise the same convention `hitPyramid()` uses: `toWorld()`'s `position + xright + yup + zforward` (*`EntityMesh.cpp:125–127`) is the forward half of exactly the same orthonormal-basis projection `hitPyramid()` runs in the other direction to bring a world-space ray* into* local space (`RayTraceCore.h:356–360`).

The mesh this produces is faceted rather than smooth: `addFlatTriangle()` gives each of the six triangles (four sides plus two base triangles) its own private three vertices and one shared face normal computed directly from that triangle's own edges, rather than sharing vertices — and therefore normals — across faces the way the sphere does:

```
// Appends one flat-shaded triangle (its own 3 vertices, not shared with any other
triangle,
// since the pyramid's edges are meant to look sharp rather than smoothed).
void addFlatTriangle( MeshData& mesh, simd_float3 a, simd_float3 b, simd_float3 c )
{
        simd_float3 normal = simd_normalize( simd_cross( b - a, c - a ) );
        uint32_t    base = (uint32_t)mesh.positions.size();

        mesh.positions.push_back( a );
        mesh.positions.push_back( b );
        mesh.positions.push_back( c );
        mesh.normals.push_back( normal );
        mesh.normals.push_back( normal );
        mesh.normals.push_back( normal );

        mesh.indices.push_back( base );
        mesh.indices.push_back( base + 1 );
        mesh.indices.push_back( base + 2 );
}
```

`RayTracerBench/Export/EntityMesh.cpp:83–100`

That's the correct choice for a pyramid's actual geometry: a real pyramid has hard edges between its faces, and averaging normals across them (as a smooth-shaded sphere legitimately does) would fake a curvature the shape doesn't have.

§6. Winding order: hand-derived, then caught wrong by a test

The comment on `buildPyramidMesh()` above is explicit about method: the side faces' winding, `triangle(C[i], C[(i+1)%4], apex)`, was worked out by hand — computing `cross(C[i+1]−C[i], apex−C[i])` and checking the result against each face's already-known outward normal from `hitPyramidLocal()`'s plane comment (e.g. the +X face's `(height, halfWidth, 0)`) — and the base's two triangles, `(C0,C2,C1)` and `(C0,C3,C2)`, were derived the same way against the known base normal `(0,−1,0)`. The sphere's winding comment makes the identical claim about method: "winding chosen so `cross(v10−v00, v11−v00)` etc. point outward" (`EntityMesh.cpp:49–50`).

But `CLAUDE.md`'s own account of this module is blunt about hand-derivation alone not being trusted: "Both mesh generators' winding orders were derived by hand and then verified programmatically... rather than trusted by inspection — an initial sphere-winding attempt was in fact backwards and caught this way." In other words, the first version of `buildSphereMesh()`'s triangle winding — worked out by exactly the same by-hand cross-product reasoning documented in the comments above — was actually wrong, and it was a test, not a code reviewer or a 3D viewer, that caught it.

That test lives in `RayTracerBenchTests/EntityMeshTests.cpp` (Chapter 11 covers the whole test suite; this is the specific check relevant here). It doesn't re-derive the correct winding by hand and compare — it defines "correct" operationally, as "every triangle's face normal points away from a point known to be inside the shape":

```
// True if every triangle in `mesh` winds so its face normal (via cross product) points
away
// from `interiorPoint` — the same check used to hand-derive EntityMesh.cpp's winding
orders,
// re-run here so a future edit that breaks winding fails a test instead of only looking
wrong
// in a 3D viewer.
bool allTrianglesFaceAwayFrom( const MeshData& mesh, simd_float3 interiorPoint )
{
        for ( size_t i = 0; i + 2 < mesh.indices.size(); i += 3 )
        {
                simd_float3 a = mesh.positions[ mesh.indices[ i ] ];
                simd_float3 b = mesh.positions[ mesh.indices[ i + 1 ] ];
                simd_float3 c = mesh.positions[ mesh.indices[ i + 2 ] ];
                simd_float3 faceNormal = simd_cross( b - a, c - a );
                simd_float3 centroid = ( a + b + c ) / 3.0f;
                if ( dot3( faceNormal, centroid - interiorPoint ) <= 0.0f )
                        return false;
        }
        return true;
}
```

`RayTracerBenchTests/EntityMeshTests.cpp:23–40`

For a sphere the interior point is trivially its own center (`EntityMeshTests.cpp:62`, `allTrianglesFaceAwayFrom( mesh, center )`); for a pyramid it's "a point just above the base center," valid for any height greater than 0.4 (`EntityMeshTests.cpp:91–92`). Both tests run every single triangle in the generated mesh through this check, not a hand-picked sample — which is precisely the exhaustiveness a by-eye inspection or a one-off hand-computed cross product can't offer, and precisely why the backwards sphere winding surfaced here instead of shipping. A third test in the same file, `buildEntityMesh_pyramid_respectsOrientationBasis` (`EntityMeshTests.cpp:95–122`), checks that a pyramid rotated so its local "up" points along world +X actually places its apex at world `(1,0,0)` — confirming `toWorld()`'s basis projection, not just the winding, survives an arbitrary orientation.

Where this connects

- **Chapter 1** (`Core/ShaderTypes.h`, `Core/Scene.hpp/.cpp`) defines the `TransformGPU` / `ShapeGPU` components this chapter's `buildEntityMesh()` consumes — the same two components every system in this project reads, never a mesh-specific representation of its own.

- **Chapter 2** (`Core/RayTraceCore.h`) is where `hitEntity()` and `scatter()` establish the tagged-dispatch pattern `buildEntityMesh()` reuses, and where `hitPyramidLocal()`'s five half-space planes are the same geometric definition `buildPyramidMesh()` reads its five vertices from (§5 above).
- **Chapter 7** (`Export/SceneExporter.hpp/.cpp`) is `buildEntityMesh()`'s only caller: it invokes it once per entity — see `RayTracerBench/Export/SceneExporter.cpp:237` and `:317` — to assemble the full scene's geometry into a glTF or OBJ+MTL file, and is also where the companion preview PNG (`Export/ImageWriter.hpp/.cpp`) gets written into the same output directory.
- **Chapter 11** (`RayTracerBenchTests/*`) covers `EntityMeshTests.cpp` in full, including the degenerate-triangle and vertex-on-surface checks for spheres and the exact-triangle-count check for pyramids that this chapter only used selectively (§6) to narrate the winding-order bug.

Chapter 7: Scene Export — glTF, OBJ, and Preview Images

Abstract. The renderers turn a `SceneDescription` into pixels; this chapter covers the code that instead turns it into files a 3D modeling tool can open. `Export/SceneExporter.hpp/.cpp` walks the same entity arrays Chapter 1 describes, calls Chapter 6's `buildEntityMesh()` per entity, and serializes the result as either a single self-contained `.gltf` file or an `.obj` + `.mtl` pair, with a from-scratch material mapping into each format's native shading model. Alongside the 3D file, `Export/ImageWriter.hpp/.cpp` writes a same-named PNG — a real render at the current settings, not a placeholder — which is why `AppDelegate::saveScene()` (Chapter 8) had to become a background-thread operation instead of the fast, synchronous, main-thread call it started out as. This chapter closes with the verification story CLAUDE.md records for this feature: an external-tool check of a real ~490-entity export, and a temporary env-var-gated hook run through the built app and then reverted.

Files covered: `Export/SceneExporter.hpp`, `Export/SceneExporter.cpp`, `Export/ImageWriter.hpp`, `Export/ImageWriter.cpp`.

§1. Why this module exists, and where it sits

Everything up through Chapter 6 is in service of turning a `SceneDescription` into pixels. The `ControlsPanel` also exposes two buttons — "Save glTF" and "Save OBJ" — that instead turn the *same* scene into a standard 3D-interchange file, so it can be opened in Blender, Preview, or any glTF/OBJ-aware tool outside this app entirely. `SceneExporter.hpp` states the intent plainly:

```
#pragma once

#include "../Core/Scene.hpp"

#include <string>
#include <vector>

// Result of an export attempt: `ok` plus a human-readable summary (a success message
naming the
// written path(s), or an error) and the absolute path(s) actually written.
struct SceneExportResult
{
        bool                     ok;
        std::string              message;
        std::vector<std::string> writtenFilePaths;
};
```

`Export/SceneExporter.hpp:1–15`

`Export/`, like `Core/`, is framework-free C++17 and is compiled into the same `RayTracerCore` static library — it has no dependency on Metal or AppKit and needs neither to run. That placement matters for `ImageWriter`, covered in §7 below, which *does* need two Apple frameworks and is consequently built differently.

The module has four public entry points, each covered in its own section: `executableDirectory()` (§2), `sceneFilenameStem()` (§3), `exportSceneAsGLTF()` (§4), and `exportSceneAsOBJ()` (§5), plus `ensureSavedScenesDirectoryPath()` (§6), which exists purely for the preview-PNG feature to reuse.

§2. `executableDirectory()`: resolving the real binary location

Every export lands in a `SavedScenes/` directory next to the running executable. Finding "next to the running executable" correctly is less trivial than it sounds, and the header comment is explicit about the two wrong answers that were considered and rejected:

```
// Absolute path to the directory containing the currently-running executable, resolved
via
// _NSGetExecutablePath (not argv[0] or getcwd() — this app can be launched from any
working
// directory, e.g. Xcode's "Product > Run" or a Finder double-click).
std::string executableDirectory();
```

`Export/SceneExporter.hpp:17–20`

`argv[0]` is unreliable because it's whatever the parent process chose to pass — it can be a bare program name with no path component at all, depending on how the process was launched. `getcwd()` is unreliable for the opposite reason: the current working directory reflects wherever the *launcher* happened to be (Xcode's own build directory when run via "Product > Run", the Finder's notion of "current directory" — effectively undefined — for a double-click), not where the binary itself lives. Both would make `SavedScenes/` show up in an unpredictable place depending on how the app happens to be started that particular time.

The fix is to ask the OS directly. `_NSGetExecutablePath` is a Darwin-specific `mach-o/dyld.h` API with an unusual two-call contract: call it once with a null buffer to learn the required size, then call it again with a buffer of that size to get the actual path.

```
// Resolves _NSGetExecutablePath's two-call pattern (first call reports the required
buffer size)
// and canonicalizes the result so a relative or symlinked launch path still resolves
correctly.
std::string executableDirectory()
{
        uint32_t size = 0;
        _NSGetExecutablePath( nullptr, &size ); // always "fails" here; size is now the
required length

        std::vector<char> buffer( size );
        _NSGetExecutablePath( buffer.data(), &size );

        std::error_code       ec;
        std::filesystem::path exePath = std::filesystem::canonical( std::filesystem::path(
buffer.data() ), ec );
        if ( ec )
                return std::filesystem::path( buffer.data() ).parent_path().string();
        return exePath.parent_path().string();
}
```

`Export/SceneExporter.cpp:141–156`

The `std::filesystem::canonical` pass is a second layer of correctness on top of the syscall itself: `_NSGetExecutablePath` is documented to potentially return a path containing symlinks or `..` components rather than a fully resolved absolute path, so `canonical()` resolves those before `parent_path()` is taken. If canonicalization fails for some reason (`ec` set), the code falls back to `parent_path()` on the raw, uncanonicalized path rather than propagating an error — a directory that's merely un-canonicalized is still usable, so the fallback degrades gracefully instead of failing the whole export over a cosmetic issue.

§3. `sceneFilenameStem()`: what varies the file, and what doesn't

Every export — `.gltf`, `.obj` / `.mtl`, and the companion `.png` from §8 — shares one filename stem, computed here:

```
// A filename stem encoding the scene-identifying parameters — seed, image width (which
feeds the
// camera's aspect ratio), and the Floating? state — plus a timestamp, so re-exporting
identical
// settings doesn't silently overwrite a prior export. samplesPerPixel/maxDepth are
deliberately
// excluded: they affect only rendering, not the exported geometry itself.
std::string sceneFilenameStem( unsigned seed, uint32_t width, bool floating );
```

`Export/SceneExporter.hpp:22–26`

```
// Encodes the scene-identifying parameters plus a timestamp — see the header comment for
why
// samplesPerPixel/maxDepth are excluded.
std::string sceneFilenameStem( unsigned seed, uint32_t width, bool floating )
{
        std::time_t t = std::chrono::system_clock::to_time_t(
std::chrono::system_clock::now() );
        std::tm     localTime{};
        localtime_r( &t, &localTime );

        char buf[ 160 ];
        std::snprintf( buf, sizeof( buf ), "scene_seed%u_w%u_%s_%04d%02d%02d-
%02d%02d%02d",
                seed, width, floating ? "floating" : "grounded",
                localTime.tm_year + 1900, localTime.tm_mon + 1, localTime.tm_mday,
                localTime.tm_hour, localTime.tm_min, localTime.tm_sec );
        return std::string( buf );
}
```

`Export/SceneExporter.cpp:158–172`

The subtle point worth dwelling on is *which* settings make it into the stem and which don't. `RenderSettings` (surfaced by `ControlsPanel::currentSettings()`, Chapter 9) bundles six values: image width, samples per pixel, max depth, CPU-threading mode, scene seed, and the Floating? checkbox. Only three of those six — seed, width, and floating — actually change what geometry `buildDefaultScene()` produces. The seed drives the `std::mt19937` that places the randomized-field spheres (Chapter 1); width feeds the camera's aspect ratio, which changes the camera's frustum and therefore which of the *fixed* scene entities are laid out identically but which region of them a render would frame (Chapter 1's `buildDefaultScene()` also takes width for this reason); and floating decides whether those

spheres sit on the ground or at a random height per Chapter 1's `kMaxFloatHeight`. Samples-per-pixel and max-depth, by contrast, are pure rendering-quality knobs: they change how many rays are traced and how deep they bounce, which affects the *pixel* output of a render but never the *geometry* a 3D export walks. Baking them into the filename would imply that two exports with different spp values contain different scenes, when in fact — for a fixed seed/width/floating triple — they'd be byte-for-byte the same `.gltf` / `.obj`. Excluding them keeps the filename an honest description of what varies in the exported file.

The trailing timestamp exists for an orthogonal reason: re-exporting the *same* seed/width/floating combination twice (e.g. after tweaking spp and re-saving) would otherwise silently overwrite the earlier file, since seed/width/floating alone can repeat. The timestamp guarantees each export click produces a distinct file regardless.

§4. `exportSceneAsGLTF()`: one self-contained file

glTF is written as a single `.gltf` file with the mesh's binary vertex/index data embedded directly in the JSON as a base64 `data:` URI, rather than the more common two-file layout of a `.gltf` plus a companion `.bin`. The comment on the base64 helper states the reasoning:

```
// Standard base64 alphabet (RFC 4648), used to embed the glTF export's binary buffer as a
// data: URI rather than writing a companion .bin file — keeps a glTF export to one file.
std::string base64Encode( const uint8_t* data, size_t len )
```

`Export/SceneExporter.cpp:18–20`

The function itself is a plain three-bytes-in/four-characters-out RFC 4648 encoder with the usual `=` / `==` padding for a 1- or 2-byte remainder (`Export/SceneExporter.cpp:20–56`) — nothing glTF-specific, just infrastructure for what follows.

`exportSceneAsGLTF()`'s job is to walk every entity, hand its `TransformGPU` / `ShapeGPU` pair to Chapter 6's `buildEntityMesh()`, and fold the resulting `MeshData` into one shared binary buffer plus the JSON structures (`bufferViews`, `accessors`, `meshes`, `nodes`) that describe how to slice that buffer back into typed arrays. Two small lambdas do the buffer-appending, each returning the byte offset the just-appended data starts at (which becomes each `bufferView`'s `byteOffset`):

```
auto appendVec3s = [ &buffer ]( const std::vector<simd_float3>& v ) -> size_t {
        size_t offset = buffer.size();
        buffer.resize( offset + v.size() * sizeof( float ) * 3 );
        if ( !v.empty() )
                std::memcpy( buffer.data() + offset, v.data(), v.size() * sizeof( float )
* 3 );
        return offset;
};
auto appendIndices = [ &buffer ]( const std::vector<uint32_t>& v ) -> size_t {
        size_t offset = buffer.size();
        buffer.resize( offset + v.size() * sizeof( uint32_t ) );
        if ( !v.empty() )
                std::memcpy( buffer.data() + offset, v.data(), v.size() * sizeof( uint32_t
) );
        return offset;
};
```

`Export/SceneExporter.cpp:289–302`

The per-entity loop calls `buildEntityMesh()` exactly once per entity (skipping any entity whose mesh comes back empty — `buildEntityMesh()` returns an empty `MeshData` for a shape it doesn't recognize, though in the current scene every entity is a sphere or a pyramid and always produces geometry), computes the mesh's axis-aligned bounding box for glTF's required accessor `min`/`max` fields, and appends three glTF `bufferView`/`accessor` pairs per mesh — one each for positions, normals, and indices:

```
MeshData mesh = buildEntityMesh( scene.transforms[ e ], scene.shapes[ e ] );
if ( mesh.positions.empty() )
        continue;

simd_float3 minV = mesh.positions[ 0 ];
simd_float3 maxV = mesh.positions[ 0 ];
for ( const simd_float3& p : mesh.positions )
{
        minV.x = std::min( minV.x, p.x );
        /* ... analogous min/max for y, z, and maxV — elided for length */
}

size_t posOffset = appendVec3s( mesh.positions );
int    posBufferView = bufferViewCount++;
bufferViewsJson << ( posBufferView ? "," : "" ) << "{\"buffer\":0,\"byteOffset\":" <<
posOffset
                                << ",\"byteLength\":" << ( mesh.positions.size() * sizeof(
float ) * 3 ) << ",\"target\":34962}";
int posAccessor = accessorCount++;
accessorsJson << ( posAccessor ? "," : "" ) << "{\"bufferView\":" << posBufferView
                        << ",\"componentType\":5126,\"count\":" << mesh.positions.size()
<< ",\"type\":\"VEC3\",\"min\":[" << minV.x << ","
                        << minV.y << "," << minV.z << "],\"max\":[" << maxV.x << "," <<
maxV.y << "," << maxV.z << "]}";
```

`Export/SceneExporter.cpp:317–340`

(Normals and indices follow the identical pattern at `Export/SceneExporter.cpp:342–356`, using `componentType` 5126 (`GL_FLOAT`) for normals and 5125 (`GL_UNSIGNED_INT`) for indices, and `target` 34963 (`ELEMENT_ARRAY_BUFFER`) instead of 34962 (`ARRAY_BUFFER`) since indices are consumed differently by a GPU than vertex attributes are — these are the standard glTF/OpenGL enum values, not project-specific choices.) Each mesh then contributes one `mesh` object referencing its three accessors and its material index, and one `node` object referencing that mesh (`Export/SceneExporter.cpp:358–364`) — a flat one-node-per-entity scene graph, with no shared transform hierarchy, since `buildEntityMesh()` already bakes each entity's `TransformGPU` into world-space vertex positions (Chapter 6) rather than leaving it as a node-local transform for the glTF consumer to apply.

Once every entity has been folded in, the accumulated `buffer` is base64-encoded once and embedded as the sole entry in the top-level `"buffers"` array:

```
std::string base64 = base64Encode( buffer.data(), buffer.size() );

std::ostringstream json;
json << "{\"asset\":{\"version\":\"2.0\",\"generator\":\"RayTracerBench
SceneExporter\"},";
/* ... scene/nodes/meshes/materials/accessors/bufferViews assembled from the
ostringstreams above ... */
json << "\"buffers\":[{\"byteLength\":" << buffer.size() <<
",\"uri\":\"data:application/octet-stream;base64," << base64 << "\"}]";
json << "}";
```

`Export/SceneExporter.cpp:376–390`

The JSON is built with hand-rolled `std::ostringstream` concatenation rather than a JSON library — consistent with this project's preference (seen already in `Core/` and `CPU/`) for standing on plain standard-library and system facilities rather than pulling in third-party dependencies for a narrowly-scoped, fully-controlled output shape. The whole string is then written to a single `.gltf` file with no companion `.bin` (`Export/SceneExporter.cpp:392–399`).

§5. `exportSceneAsOBJ()`: a text format with a companion `.mtl`

OBJ, unlike glTF, has no notion of an embedded material or binary payload at all — Wavefront's format is a plain-text vertex/face list that refers to materials by name via `usemtl`, with the actual material definitions living in a separate `.mtl` file named by an `mtllib` directive. The exporter follows that convention directly, writing the `.mtl` first:

```
std::ofstream mtlFile( mtlPath );
/* ... */
mtlFile << "# RayTracerBench scene export – materials\n\n";
for ( size_t m = 0; m < scene.materials.size(); ++m )
{
        MTLFields f = mtlFieldsFor( scene.materials[ m ] );
        mtlFile << "newmtl Material_" << m << "\n";
        mtlFile << "Kd " << f.kd.x << " " << f.kd.y << " " << f.kd.z << "\n";
        mtlFile << "Ks " << f.ks.x << " " << f.ks.y << " " << f.ks.z << "\n";
        mtlFile << "Ns " << f.ns << "\n";
        mtlFile << "d " << f.d << "\n";
        mtlFile << "illum " << f.illum << "\n\n";
}
```

`Export/SceneExporter.cpp:204–221`

...then the `.obj` itself, which starts with the `mtllib` reference and, per entity, emits an `o` (object) name, its vertex positions (`v`), its vertex normals (`vn`), a `usemtl` directive naming

that entity's material, and a triangle-list of `f` (face) lines:

```
objFile << "# RayTracerBench scene export\n";
objFile << "mtllib " << stem << ".mtl\n\n";

const size_t entityCount = scene.transforms.size();
uint32_t     vertexOffset = 0; // OBJ vertex/normal indices are 1-based and file-global

for ( size_t e = 0; e < entityCount; ++e )
{
        MeshData mesh = buildEntityMesh( scene.transforms[ e ], scene.shapes[ e ] );
        if ( mesh.positions.empty() )
                continue;

        const char* shapeName = scene.shapes[ e ].type == SHAPE_SPHERE ? "Sphere" :
"Pyramid";
        objFile << "o Entity_" << e << "_" << shapeName << "\n";

        for ( const simd_float3& p : mesh.positions )
                objFile << "v " << p.x << " " << p.y << " " << p.z << "\n";
        for ( const simd_float3& n : mesh.normals )
                objFile << "vn " << n.x << " " << n.y << " " << n.z << "\n";

        objFile << "usemtl Material_" << scene.shapes[ e ].materialIndex << "\n";
        for ( size_t i = 0; i + 2 < mesh.indices.size(); i += 3 )
        {
                uint32_t a = mesh.indices[ i ] + 1 + vertexOffset;
                uint32_t b = mesh.indices[ i + 1 ] + 1 + vertexOffset;
                uint32_t c = mesh.indices[ i + 2 ] + 1 + vertexOffset;
                objFile << "f " << a << "//" << a << " " << b << "//" << b << " " << c <<
"//" << c << "\n";
        }

        vertexOffset += (uint32_t)mesh.positions.size();
        objFile << "\n";
}
```

`Export/SceneExporter.cpp:229–260`

The `vertexOffset` bookkeeping is the one piece of OBJ-specific plumbing worth calling out: OBJ vertex and normal indices are 1-based (hence the `+ 1`) *and* file-global, not per-object — every `v` / `vn` line in the whole file shares one running index space, so each subsequent entity's face indices have to be offset by the total vertex count of every entity written before it, not just reset to zero per object the way `buildEntityMesh()`'s own per-entity `MeshData::indices` are. The `a//a` face syntax (`vertex//normal`, no texture-coordinate slot) reflects that this exporter never generates UVs — `MeshData` (Chapter 6) carries only positions, normals, and indices, no texcoords, consistent with there being no texture-mapped materials anywhere in this scene.

§6. Material mapping: the same dielectric approximation in both formats

Both exporters need to translate `Core/ShaderTypes.h`'s `MaterialGPU` — a tagged `MAT_LAMBERTIAN` / `MAT_METAL` / `MAT_DIELECTRIC` struct, the same one `RayTraceCore.h`'s `scatter()` switches on at render time (Chapter 2) — into each format's own, differently-shaped native material model. glTF uses physically-based metallic-roughness (`baseColorFactor`, `metallicFactor`, `roughnessFactor`, plus an alpha channel and `alphaMode`); OBJ/MTL uses the older Phong-style `Kd` / `Ks` / `Ns` / `d` / `illum` fields (diffuse color, specular color, specular exponent, dissolve/opacity, and an illumination-model index). Two small free functions do the mapping, one per format:

```
// Maps a MaterialGPU onto glTF's PBR metallic-roughness parameters.
GLTFMaterialFields gltfMaterialFor( const MaterialGPU& mat )
{
        switch ( mat.type )
        {
                case MAT_LAMBERTIAN:
                        return GLTFMaterialFields{ mat.albedo, 0.0f, 1.0f, 1.0f };
                case MAT_METAL:
                        // fuzz's range here is [0, 0.5] (see Scene.cpp's fuzzDist) —
doubled to fill
                        // roughness's full [0,1] range.
                        return GLTFMaterialFields{ mat.albedo, 1.0f, std::min( mat.fuzz *
2.0f, 1.0f ), 1.0f };
                case MAT_DIELECTRIC:
                        // No KHR_materials_transmission here (deliberately, to keep this
a plain,
                        // universally-loadable core-spec glTF file) — approximated as a
clear, glossy,
                        // partly-transparent surface instead of true refraction.
                        return GLTFMaterialFields{ simd_make_float3( 1.0f, 1.0f, 1.0f ),
0.0f, 0.05f, 0.35f };
        }
        return GLTFMaterialFields{ simd_make_float3( 1.0f, 1.0f, 1.0f ), 0.0f, 1.0f, 1.0f
};
}
```

`Export/SceneExporter.cpp:89–107`

```
// Maps a MaterialGPU onto Wavefront MTL's Kd/Ks/Ns/d/illum fields.
MTLFields mtlFieldsFor( const MaterialGPU& mat )
{
        switch ( mat.type )
        {
                case MAT_LAMBERTIAN:
                        return MTLFields{ mat.albedo, simd_make_float3( 0.05f, 0.05f,
0.05f ), 8.0f, 1.0f, 2 };
                case MAT_METAL:
                {
                        // Sharper specular highlight (higher Ns) for lower fuzz,
mirroring fuzz's role as
                        // a roughness knob in RayTraceCore.h's scatter().
                        float roughness = std::min( mat.fuzz * 2.0f, 1.0f );
                        return MTLFields{ mat.albedo * 0.15f, mat.albedo, 8.0f + ( 1.0f -
roughness ) * 300.0f, 1.0f, 3 };
                }
                case MAT_DIELECTRIC:
                        // illum 4: "Transparency: glass on, reflection: ray trace on" —
the closest
                        // standard MTL illumination model to a dielectric surface.
                        return MTLFields{ simd_make_float3( 1.0f, 1.0f, 1.0f ),
simd_make_float3( 1.0f, 1.0f, 1.0f ), 96.0f, 0.35f, 4 };
        }
        return MTLFields{ simd_make_float3( 1.0f, 1.0f, 1.0f ), simd_make_float3( 0.0f,
0.0f, 0.0f ), 8.0f, 1.0f, 2 };
}
```

`Export/SceneExporter.cpp:119–138`

For lambertian and metal, the two mappings are close analogues of each other: lambertian gets the material's own `albedo` as its diffuse/base color with full roughness and no metallic contribution in glTF (`metallicFactor` 0, `roughnessFactor` 1), and a small, fixed 5% specular tint in MTL; metal gets `albedo` as base color with `metallicFactor` 1 in glTF, and `albedo` itself as the MTL specular color with a dimmed 15% diffuse tint. Both formats derive their roughness-like knob the same way — `fuzz` doubled to fill `[0,1]`, since `Scene.cpp`'s `fuzzDist` only ever produces values in `[0, 0.5]` — and MTL additionally converts that into a specular exponent (`Ns`) that's *larger* for *lower* fuzz, since a sharper (higher-exponent) Phong highlight is the classical-shading equivalent of a smoother PBR surface.

`MAT_DIELECTRIC` is where the interesting decision sits, and it's made identically in both functions: neither glTF's core specification nor OBJ/MTL has any real model of glass or light refraction. glTF has an extension for it — `KHR_materials_transmission` — but the exporter deliberately does not use it, so that the `.gltf` file stays plain core-spec glTF that any glTF-capable viewer can load, rather than one that silently renders wrong (opaque, or without the extension's intended look) in a viewer that doesn't implement that particular extension. Both

functions instead approximate a dielectric the same conceptual way: a clear (white) base color, a low roughness/high specular exponent for a glossy look, and partial transparency — `alpha` / `d` of `0.35` in both. This is stated candidly as an approximation, not a claim of physical equivalence: the exported mesh will look like a glossy, translucent object in a generic 3D viewer, not like the refracting glass sphere the ray tracer actually renders. Anyone comparing the exported `.gltf` / `.obj` against the app's own raytraced preview PNG (§8) should expect the dielectric spheres in particular to look different between the two — that's the nature of the approximation, not a bug in either path.

§7. `ensureSavedScenesDirectoryPath()`: a public seam for the preview PNG

`SceneExporter.cpp` already has a private `savedScenesDirectory()` / `ensureSavedScenesDirectory()` pair (`Export/SceneExporter.cpp:59–72`) that both `exportSceneAsGLTF()` and `exportSceneAsOBJ()` use internally to find and create `SavedScenes/`. When the preview-PNG feature (§8) was added, `AppDelegate::saveScene()` needed to write a `.png` into that exact same directory, using the exact same stem, right after the `.gltf` / `.obj` succeeds. Rather than have `AppDelegate.cpp` re-derive `executableDirectory() / "SavedScenes"` and re-implement the create-if-missing logic itself — which would risk the two call sites drifting apart if the directory name or resolution logic ever changed — the same internal helper was exposed as one small public function:

```
// The directory every export writes into — <executableDirectory()>/SavedScenes/ — created
if it
// doesn't already exist yet. Exposed so callers can place a companion file (e.g. a
preview image)
// alongside a 3D export in the exact same location, using the exact same stem. Returns an
empty
// string if the directory couldn't be created.
std::string ensureSavedScenesDirectoryPath();
```

`Export/SceneExporter.hpp:28–32`

```
// Creates (if needed) and returns the shared SavedScenes/ directory, so a caller writing
a
// companion file (e.g. a preview image) alongside a 3D export can use the exact same
location
// without duplicating the directory-resolution/creation logic above.
std::string ensureSavedScenesDirectoryPath()
{
        std::filesystem::path dir = savedScenesDirectory();
        std::string           error = ensureSavedScenesDirectory( dir );
        if ( !error.empty() )
                return "";
        return dir.string();
}
```

`Export/SceneExporter.cpp:174-184`

It's a small function, but it's a real API-design decision: `AppDelegate` (Chapter 8) calls this one function to get a directory it can trust is already created, rather than needing to know anything about `_NSGetExecutablePath` or the `"SavedScenes"` literal itself. The empty-string sentinel on failure lets the caller treat "couldn't create the directory" and "successfully resolved it" as a simple truthy/falsy check without a separate error-code out-parameter.

§8. `ImageWriter`: `writePNG()` via CoreGraphics/ImageIO

The preview image is written by a small, separate module rather than folded into `SceneExporter.cpp` itself, and the header explains both what it does and, implicitly, why it's kept apart:

```
// Writes an RGBA8, row-major, row-0-is-top pixel buffer (the same layout CPURenderer.hpp
// produces) to `path` as a PNG file, via CoreGraphics/ImageIO — real system C APIs, so
this needs
// no third-party image library and no Objective-C. Returns false (rather than aborting)
on
// failure, since a failed preview image shouldn't prevent the 3D export it accompanies
from being
// reported as successful.
bool writePNG( const std::string& path, const uint8_t* rgba, uint32_t width, uint32_t
height );
```

`Export/ImageWriter.hpp:6-11`

The implementation is a direct, unglamorous use of CoreGraphics to build a `CGImageRef` around the caller's pixel buffer, and ImageIO to encode and write it:

```
bool writePNG( const std::string& path, const uint8_t* rgba, uint32_t width, uint32_t
height )
{
	CGColorSpaceRef colorSpace = CGColorSpaceCreateDeviceRGB();
	CGDataProviderRef provider = CGDataProviderCreateWithData( nullptr, rgba,
(size_t)width * height * 4, nullptr );

	CGImageRef image = CGImageCreate( width, height, 8, 32, (size_t)width * 4,
colorSpace,
		kCGBitmapByteOrderDefault | kCGImageAlphaNoneSkipLast, provider, nullptr,
false, kCGRenderingIntentDefault );

	CGDataProviderRelease( provider );
	CGColorSpaceRelease( colorSpace );

	if ( !image )
		return false;

	CFURLRef url = CFURLCreateFromFileSystemRepresentation( nullptr, (const
UInt8*)path.c_str(), (CFIndex)path.size(), false );
	CGImageDestinationRef destination = CGImageDestinationCreateWithURL( url, CFSTR(
"public.png" ), 1, nullptr );
	CFRelease( url );

	if ( !destination )
	{
		CGImageRelease( image );
		return false;
	}

	CGImageDestinationAddImage( destination, image, nullptr );
	bool ok = CGImageDestinationFinalize( destination );

	CFRelease( destination );
	CGImageRelease( image );

	return ok;
}
```

`Export/ImageWriter.cpp:11–42`

`CGDataProviderCreateWithData` is given `rgba` directly with a null deallocator callback — the comment above it in the source notes this "borrows the pointer for the duration of this call" rather than copying it, so the caller's buffer must outlive the `CGImageCreate`/finalize sequence, which it does here since it's all synchronous. `"public.png"` is passed as a raw UTI string literal rather than linking the CoreServices/UniformTypeIdentifiers framework just to get the `kUTTypePNG` / `UTTypePNG` constant symbolically — a small dependency-avoidance choice in the same spirit as writing the glTF JSON by hand instead of pulling in a JSON library.

This module needs the CoreGraphics and ImageIO frameworks, which is precisely why it is *not* part of the framework-free `RayTracerCore` static library that `EntityMesh.cpp` and `SceneExporter.cpp` live in. Linking `ImageWriter.cpp` into `RayTracerCore` would force every consumer of that library — including, notably, `RayTracerBenchTests` (Chapter 11), which is meant to test the shared core in isolation without pulling in AppKit/CoreGraphics — to also link those two frameworks, purely to support a feature (`writePNG`) that tests have no reason to exercise. Instead, `ImageWriter.cpp` is linked only into the `RayTracerBench` app target itself, alongside the AppKit/Metal/QuartzCore code that already requires a full framework set. `EntityMesh` and `SceneExporter` stay framework-free because they only ever touch plain data (`SceneDescription`, `MeshData`, `std::string`) and the C++ standard library — no CoreGraphics types appear anywhere in their signatures.

§9. The preview PNG as a whole, and what it cost `AppDelegate`

Putting §4/§5 and §8 together: every successful `.gltf` or `.obj` export is immediately followed by a same-named `.png` written into the identical `SavedScenes/` directory (via `ensureSavedScenesDirectoryPath()`, §7). That PNG is not a placeholder, an icon, or a thumbnail render at reduced quality — it's a full `renderCPU()` invocation (Chapter 4) of the *same* `SceneDescription` at the *current* controls settings, i.e. exactly the image a "Render CPU" click would have produced for that seed. The point is that a saved `.gltf` / `.obj` file, viewed outside this app in a generic 3D tool, would otherwise show the material approximations from §6 with no way to compare them against what the ray tracer actually produced; the companion PNG gives a real, correct-quality reference image sitting right next to the exported geometry.

This had a real architectural consequence, recorded directly in `AppDelegate.cpp`'s comment on `saveScene()`:

```
// Builds a scene from the current controls settings (geometry depends only on seed/width/
// floating — samplesPerPixel/maxDepth affect rendering, not the exported geometry),
writes it via
// the requested exporter, and — on success — also renders it (at the current settings,
exactly
// like "Render CPU" would) and writes a same-named PNG preview alongside it, so the 3D
export has
// a quick-look image without needing a model viewer. Runs on a background thread like the
render
// actions above: once a full CPU render is part of this, it's no longer the fast, main-
thread-only
// operation it was when it only wrote geometry.
void AppDelegate::saveScene( bool asGLTF )
```

`App/AppDelegate.cpp:296–303`

Before the preview image existed, `saveScene()` only had to build a scene and write geometry to disk — cheap enough to do synchronously on the main thread without the UI stalling noticeably. Adding a full CPU render (`renderCPU()` on a scene the size of the default ~490-entity layout, at whatever samples-per-pixel/max-depth the controls currently specify) is no longer cheap, so `saveScene()` was moved onto a background `std::thread`, mirroring `startCPURender` / `startGPURender` / `startCompare`'s existing pattern exactly: settings are read synchronously on the main thread before the thread is spawned, the thread does the (now much heavier) work, and the result — including which files were written and whether the PNG succeeded — is marshaled back to the main thread via `dispatch_async` to show an `NS::Alert` and re-enable the controls. Chapter 8 covers `AppDelegate`'s threading pattern (including this exact function's body) in full; the point to take from this chapter is only that the *reason* it had to change is the preview PNG's real render.

§10. How this was verified

CLAUDE.md's project-status notes record two distinct verification passes for this feature, and it's worth stating plainly what each one checked, since neither is a stand-in for the other.

The first targeted the exporters' actual output correctness. A standalone harness exported the real default scene — `buildDefaultScene()`'s full ~490-entity layout, not a toy scene built just for the test — to both formats, and the results were checked with tools outside this codebase entirely: `assimp info` (a widely-used 3D-asset inspection tool) and Python's `trimesh` library. That check confirmed the glTF and OBJ exports agree with each other on triangle counts, that material colors and material counts came through correctly, and that the glTF JSON is well-formed per an independent parser — not merely "the file was written without a C++ exception," but "a third-party tool that has no knowledge of this project's internals can open it and agrees with what it should contain."

The second targeted the *integration* — the real button-click path, not just the exporter functions called directly. A temporary, environment-variable-gated hook was added to the app, the app was actually built and run, and `pkill` was used to terminate it afterward (rather than an in-process auto-terminate) specifically because the save flow is asynchronous, per §9, and has no natural "done" signal a test harness could wait on synchronously. That run confirmed a real `.gltf` / `.obj` + `.mtl` and a real, correctly-rendered `.png` land side by side with the exact same stem, next to the *actual* running binary — exercising `executableDirectory()` (§2) for real rather than assuming it resolves correctly, and spot-checking the PNG visually rather than only checking that a file with the right name

exists. The hook itself was then reverted rather than left in the committed source, consistent with this project's general rule of not shipping ad hoc test scaffolding — the same technique used elsewhere in this codebase (the main-thread heartbeat check for background rendering, the offscreen GPU-texture readback test) whenever a claim needed a live, running-app check that couldn't be made through a unit test alone.

Where this connects

- **Chapter 1** (`Core/ShaderTypes.h`, `Core/Scene.hpp/.cpp`) supplies the `SceneDescription` and `MaterialGPU` this chapter reads: `scene.transforms` / `scene.shapes` / `scene.materials` are walked directly by both exporters, and `MAT_LAMBERTIAN` / `MAT_METAL` / `MAT_DIELECTRIC` are exactly the tags §6's material-mapping functions switch on.
- **Chapter 6** (`Export/EntityMesh.hpp/.cpp`) is this chapter's geometry source: every mesh written into a `.gltf` or `.obj` file comes from one call to `buildEntityMesh()` per entity: §4 and §5's loops are, in effect, "for each entity, ask Chapter 6 for its mesh, then serialize it."
- **Chapter 4** (`CPU/CPURenderer.hpp/.cpp`) supplies `renderCPU()`, the function §9's preview PNG is built from — the same rendering path a "Render CPU" click uses, run here as a side effect of a successful export rather than as a user-initiated render.
- **Chapter 8** (`App/AppDelegate.hpp/.cpp`) is where `saveScene()` actually lives and runs: its background-`std::thread` structure, and the `dispatch_async` marshaling back to the main thread, are covered there in full — this chapter only explains *why* that structure became necessary.
- **Chapter 9** (`App/ControlsPanel.hpp/.cpp`) owns the "Save glTF"/"Save OBJ" buttons that trigger `saveScene()`, and `currentSettings()`, which supplies the seed/width/floating/spp/ maxDepth values this chapter's `sceneFilenameStem()` and `buildDefaultScene()` calls consume.

Chapter 8: The App Shell and Lifecycle

Abstract. Everything in the previous seven chapters — the shared ray-tracing core, both renderers, the mesh/export pipeline — exists to be *driven* by something. This chapter covers the thing that drives it: `main.cpp`'s eleven-line bootstrap of an `NS::ApplicationDelegate`, and `AppDelegate`, the one class that owns the window, wires the four UI panels together, and turns every button click into a background render or export followed by a main-thread UI update. The throughline is a single constraint stated in `CLAUDE.md`'s "What this project is" section: pure C++ and Metal via `metal-cpp` / `metal-cpp-extensions`, deliberately no SwiftUI, no Objective-C++, no Storyboard. This chapter is where that constraint is most visible, because application bootstrap and lifecycle is exactly the layer where a normal macOS app would reach for `NSApplicationMain`, a `.storyboard`, and Swift `@objc` closures, and this one instead reaches for `NS::Application::sharedApplication()`, hand-built `NS::Window` / `NS::View` trees, and `std::thread` + `dispatch_async`.

Files covered: `RayTracerBench/main.cpp`, `RayTracerBench/App/AppDelegate.hpp`, `RayTracerBench/App/AppDelegate.cpp`.

§1. `main.cpp`: the entire entry point

The whole program's `main()` is twenty-five lines, and reads almost like the pseudocode you'd sketch before writing a "real" AppKit app — because in this codebase, this sketch *is* the real app:

```
// Metal/QuartzCore's own private-implementation macros (MTL_PRIVATE_IMPLEMENTATION,
// CA_PRIVATE_IMPLEMENTATION) live in ImageDisplayView.cpp instead, since that's the one
// translation unit that actually includes those headers — each must be defined exactly
once,
// in whichever .cpp first includes the corresponding header.
#define NS_PRIVATE_IMPLEMENTATION
#include <AppKit/AppKit.hpp>

#include "App/AppDelegate.hpp"

// Entry point: sets up the NSApplication/AppDelegate pair and runs the main event loop.
int main( int argc, char* argv[] )
{
        NS::AutoreleasePool* pAutoreleasePool = NS::AutoreleasePool::alloc()->init();

        AppDelegate del;

        NS::Application* pSharedApplication = NS::Application::sharedApplication();
        pSharedApplication->setDelegate( &del );
        pSharedApplication->setActivationPolicy(
NS::ActivationPolicy::ActivationPolicyRegular );
        pSharedApplication->run();

        pAutoreleasePool->release();

        return 0;
}
```

(`RayTracerBench/main.cpp:1–25`)

There is no Swift, no `.storyboard`, no `Info.plist`-driven main nib. The whole "app" concept — a shared `NSApplication` singleton with a delegate and a run loop — is expressed directly through `metal-cpp`'s `NS::` wrappers: `NS::AutoreleasePool` stands in for the implicit autorelease pool a normal Cocoa `main` gets for free, `AppDelegate` is a plain C++ object (stack- allocated, not heap-allocated — its address is handed to `setDelegate` directly and it lives for the whole process), and `ActivationPolicyRegular` is what makes this a normal foreground app with a Dock icon and menu bar rather than a background agent. `pSharedApplication->run()` is the call that actually starts the AppKit event loop; everything else in this chapter happens either during the `applicationDidFinishLaunching` callback that fires once that loop is up, or asynchronously afterward from button clicks and background threads.

§2. `NS_PRIVATE_IMPLEMENTATION`, and why it lives here specifically

`metal-cpp`'s headers are pure declarations; the actual Objective-C runtime glue (the `Private::Class` / `Private::Selector` lookup tables that let `metal-cpp` send real Objective-

C messages from C++) is only emitted when a translation unit defines the corresponding `_PRIVATE_IMPLEMENTATION` macro before including the header. `main.cpp` defines `NS_PRIVATE_IMPLEMENTATION` right before `#include <AppKit/AppKit.hpp>` (`RayTracerBench/main.cpp:5–6`), and the comment directly above it states the rule this follows: `Metal.hpp`'s and `QuartzCore.hpp`'s equivalents, `MTL_PRIVATE_IMPLEMENTATION` and `CA_PRIVATE_IMPLEMENTATION`, are *not* defined here — they live in `App/ImageDisplayView.cpp` instead (Chapter 10 covers that file in full).

The general rule, stated plainly in that comment, is: **each such macro must be defined in exactly the one `.cpp` that first includes the corresponding header** — get it wrong and the symptom is not a compile error but an undefined-symbol failure at *link* time, for names like `Private::Class::s_k...` or `Private::Selector::s_k...`, because the implementation tables that back those declarations were simply never emitted anywhere in the link. `main.cpp` is the one file in this project that includes `<AppKit/AppKit.hpp>`, so it is the correct (and only correct) home for `NS_PRIVATE_IMPLEMENTATION`; `ImageDisplayView.cpp` is the one file that includes `<Metal/Metal.hpp>`/`<QuartzCore/QuartzCore.hpp>` for `CAMetalLayer` purposes, so the other two macros belong there. Defining a macro in more than one translation unit is just as broken as defining it in none — it would emit duplicate implementation tables and fail at link time the other way — so this is a strict one-macro-one-file assignment, not a "define it wherever's convenient" convention.

§3. `AppDelegate`'s shape

`AppDelegate.hpp` declares the class as an `NS::ApplicationDelegate` subclass owning every top-level UI object and the GPU device/renderer by raw pointer:

```
class AppDelegate : public NS::ApplicationDelegate
{
        public:
                // Releases the renderer, panels, views, window, and device.
                ~AppDelegate();

                // Builds the menu bar, window, and all subviews, wires up button
callbacks, and installs
                // the mouse-move monitor for the magnifier.
                void applicationDidFinishLaunching( NS::Notification* pNotification )
override;
                // Always true: this app has exactly one window, so closing it should
quit.
                bool applicationShouldTerminateAfterLastWindowClosed( NS::Application*
pSender ) override;
```

(`RayTracerBench/App/AppDelegate.hpp:11-21`)

with the member list at the bottom of the header showing exactly what it owns:

```
                NS::Window*       _pWindow = nullptr;
                MTL::Device*      _pDevice = nullptr;
                ControlsPanel*    _pControlsPanel = nullptr;
                ImageDisplayView* _pCPUImageView = nullptr;
                ImageDisplayView* _pGPUImageView = nullptr;
                ResultsPanel*     _pResultsPanel = nullptr;
                GPURenderer*      _pGPURenderer = nullptr;

                double _lastCPUTimeMs = -1.0;
                double _lastGPUTimeMs = -1.0; // GPU-only time - the headline metric per
CLAUDE.md
```

(`RayTracerBench/App/AppDelegate.hpp:54-63`)

`applicationDidFinishLaunching()` is where the actual window gets built. It creates the Metal device and the `GPURenderer` up front (device/pipeline creation is deliberately front-loaded, per Chapter 5, so first-render shader-compile latency never pollutes a timed run), then lays out a single content view holding four subviews top to bottom — controls, the two image previews side by side, and the results panel:

```
        _pDevice = MTL::CreateSystemDefaultDevice();
        _pGPURenderer = new GPURenderer( _pDevice );

        // Widened from the original 860 to fit the Save glTF / Save OBJ buttons added to
row 2 of
        // ControlsPanel without crowding the existing buttons.
        const CGRect windowFrame = ( CGRect ){ { 100.0, 100.0 }, { 950.0, 460.0 } };
        _pWindow = NS::Window::alloc()->init(
                windowFrame,
                NS::WindowStyleMaskClosable | NS::WindowStyleMaskTitled,
                NS::BackingStoreBuffered,
                false );
        _pWindow->setTitle( NS::String::string( "RayTracerBench", UTF8StringEncoding ) );
        _pWindow->setAcceptsMouseMovedEvents( true );

        const CGRect contentFrame = ( CGRect ){ { 0.0, 0.0 }, windowFrame.size };
        NS::View* pContentView = NS::View::alloc()->init( contentFrame );

        // Top to bottom: controls, two side-by-side previews, results.
        _pControlsPanel = new ControlsPanel( ( CGRect ){ { 10.0, 390.0 }, { 930.0, 60.0 }
} );
        pContentView->addSubview( _pControlsPanel->view() );

        const CGRect cpuImageFrame = ( CGRect ){ { 10.0, 155.0 }, { 460.0, 225.0 } };
        const CGRect gpuImageFrame = ( CGRect ){ { 480.0, 155.0 }, { 460.0, 225.0 } };
        _pCPUImageView = new ImageDisplayView( _pDevice, cpuImageFrame );
        pContentView->addSubview( _pCPUImageView->view() );
        _pGPUImageView = new ImageDisplayView( _pDevice, gpuImageFrame );
        pContentView->addSubview( _pGPUImageView->view() );

        _pResultsPanel = new ResultsPanel( ( CGRect ){ { 10.0, 15.0 }, { 930.0, 70.0 } }
);
        pContentView->addSubview( _pResultsPanel->view() );

        _pControlsPanel->onRenderCPU = [ this ]() { startCPURender( _pControlsPanel-
>currentSettings() ); };
        _pControlsPanel->onRenderGPU = [ this ]() { startGPURender( _pControlsPanel-
>currentSettings() ); };
        _pControlsPanel->onCompare = [ this ]() { startCompare( _pControlsPanel-
>currentSettings() ); };
        _pControlsPanel->onSaveGLTF = [ this ]() { saveScene( true ); };
        _pControlsPanel->onSaveOBJ = [ this ]() { saveScene( false ); };

        _pWindow->setContentView( pContentView );
        _pWindow->makeKeyAndOrderFront( nullptr );

        pApp->activateIgnoringOtherApps( true );

        NS::Event::addLocalMonitorForEventsMatchingMask( NS::EventMaskMouseMoved,
^NS::Event*( NS::Event* pEvent ) {
                handleMouseMoved( pEvent );
                return pEvent;
        } );
```

(`RayTracerBench/App/AppDelegate.cpp:89–134`)

`AppDelegate` is the wiring layer and nothing more: it doesn't know how `ControlsPanel` builds its text fields and checkbox (Chapter 9), how `ResultsPanel` formats its three lines (Chapter 9), or how `ImageDisplayView` turns a pixel buffer or `MTL::Texture` into something on screen via `CAMetalLayer` and `Blit.metal` (Chapter 10). It just instantiates all four, places them in the content view, and connects each `ControlsPanel` button callback — five plain `std::function` members (`onRenderCPU`, `onRenderGPU`, `onCompare`, `onSaveGLTF`, `onSaveOBJ`) — to one `AppDelegate` method each via a capturing lambda. The magnifier's mouse-move monitor (`NS::Event::addLocalMonitorForEventsMatchingMask` at line 131) is installed here too, once, at launch, and `handleMouseMoved()` fans the same normalized UV position out to both `ImageDisplayView`s (`RayTracerBench/App/AppDelegate.cpp:139–171`) — full detail on that mechanism belongs to Chapter 10, since it's really an `ImageDisplayView` feature that `AppDelegate` merely routes events into.

§4. The threading pattern, used identically four times

All three render actions and `saveScene()` follow one pattern, spelled out in the header comment above their declarations:

```
                // Each spawns a background std::thread that renders, then marshals the UI
update back to
                // the main thread via dispatch_async(dispatch_get_main_queue(), ...) —
GCD's plain C API,
                // no Objective-C needed. Controls are disabled for the duration to
prevent a second click
                // from racing an in-flight render against the same shared
GPURenderer/ImageDisplayView state.
```

(`RayTracerBench/App/AppDelegate.hpp:24–27`)

`startCPURender()` is the clearest instance of it:

```
void AppDelegate::startCPURender( const RenderSettings& settings )
{
        _pControlsPanel->setControlsEnabled( false );

        std::thread( [ this, settings ]()
        {
                SceneDescription scene = buildDefaultScene( settings.seed, settings.width,
kAspectRatio, settings.samplesPerPixel, settings.maxDepth, settings.floating );
                CPURenderResult  result = renderCPU( scene, settings.cpuMode );
                double           rps = raysPerSecond( scene.params,
result.renderTime.count() );

                dispatch_async( dispatch_get_main_queue(), ^{
                        _pCPUImageView->updatePixels( result.pixels.data(),
scene.params.width, scene.params.height );

                        char buf[ 160 ];
                        std::snprintf( buf, sizeof( buf ), "CPU (%s): %.1f ms | %.2fM
rays/s",
                                settings.cpuMode == CPUThreading::MultiThreaded ? "multi"
: "single",
                                result.renderTime.count(), rps / 1.0e6 );
                        _pResultsPanel->setCPULine( buf );

                        _lastCPUTimeMs = result.renderTime.count();
                        updateSpeedupIfPossible();

                        _pControlsPanel->setControlsEnabled( true );
                        std::printf( "CPU render: %.1f ms\n", result.renderTime.count() );
                        std::fflush( stdout );
                } );
        } ).detach();
}
```

(`RayTracerBench/App/AppDelegate.cpp:175–202`)

The shape is always the same: disable controls synchronously on the calling (main) thread, spawn a detached `std::thread` that captures `this` and a by-value copy of `settings`, do all the heavy work — building the scene and invoking the renderer — off the main thread, then hand the *results* back to the main thread through `dispatch_async(dispatch_get_main_queue(), ...)`, where the UI is actually touched (`updatePixels` / `displayTexture`, `setCPULine` / `setGPULine`, `setControlsEnabled(true)`). `startGPURender()` (`RayTracerBench/App/AppDelegate.cpp:206–232`) and `startCompare()` (`RayTracerBench/App/AppDelegate.cpp:236–273`, which does both renders on the same background thread before a single `dispatch_async` block updates both preview lines and the speedup line) are structurally identical, just with `GPURenderer::render()` or both renderers in place of `renderCPU()`.

The use of `dispatch_async` / `dispatch_get_main_queue()` here is itself a small instance of the project's larger "no Objective-C++" discipline: Grand Central Dispatch is a plain C API (declared in `<dispatch/dispatch.h>`, which `AppDelegate.cpp` includes at line 10), and its block-based callback argument compiles fine as a C++ Objective-C block literal without needing an `.mm` file — the same observation `CLAUDE.md` makes about `NS::Event::addLocalMonitorForEventsMatchingMask`'s block argument and `MTL::Buffer`'s deallocator parameter. Marshaling work *onto* a background `std::thread` uses the C++ standard library; marshaling the result *back* to the main thread uses GCD's C API. Neither needs Objective-C++.

§5. Why controls are disabled, and why `currentSettings()` is read where it is

The comment at `AppDelegate.hpp:24–27` (quoted above) gives the reason controls are disabled for a render's duration: `GPURenderer` and both `ImageDisplayView`s are shared, mutable state, and only one render should be touching them at a time. If a second click were allowed to slip in while the first render's background thread was still running, its `dispatch_async` block could stomp on the same texture, or `GPURenderer` could be asked to start a second dispatch while its first hasn't finished — a data race under one benign-looking button.

Look again at where `currentSettings()` gets called relative to `setControlsEnabled( false )` inside `applicationDidFinishLaunching()`'s wiring:

```
        _pControlsPanel->onRenderCPU = [ this ]() { startCPURender( _pControlsPanel-
>currentSettings() ); };
```

(`RayTracerBench/App/AppDelegate.cpp:120`)

`currentSettings()` runs synchronously, on the main thread, as an argument evaluated *before* `startCPURender` is even entered — and `startCPURender`'s very first statement is `_pControlsPanel->setControlsEnabled( false )` (`RayTracerBench/App/AppDelegate.cpp:177`), still on the main thread, still before the `std::thread` is constructed. So the full ordering for any one click is: read the text fields into a `RenderSettings` value → disable the controls → *then* spin up the background thread. There is no point between "the user could still click a control" and "the controls are now disabled" at which a second click could be dispatched, because both steps happen synchronously on the main thread's own event-handling call stack, with nothing yielding back to the run loop in between. `saveScene()` follows the exact same ordering — `currentSettings()` is read at `RayTracerBench/App/AppDelegate.cpp:307`, immediately after `setControlsEnabled( false`

`)` at line 305, both still on the main thread, before its own `std::thread` is constructed at line 309.

§6. Verifying "doesn't freeze the UI" without a display

The whole point of putting every render on a background thread is that the AppKit run loop should keep servicing events — window dragging, menu clicks, the eventual "controls re-enabled" update — the entire time a `Compare` run is grinding through both renderers. That's a claim about runtime behavior, not something readable from the source, and the obvious way to check it — script a UI interaction with `osascript`/System Events while a render is in flight and confirm the window stays responsive — turned out not to be available in this development environment: `osascript` returned error `-25211` (no assistive-access permission for a background session), confirmed by actually trying it, not assumed from general knowledge of sandboxed environments.

The fix, per `CLAUDE.md`'s "Project status" notes, was the same class of workaround already used elsewhere in this project for environment-limited verification (the offscreen GPU-texture readback test in Chapter 10 is the sibling case): add a temporary, env-var-gated self-test hook directly in the app — a main-thread heartbeat plus an auto-triggered `Compare` — observe the result, then revert the hook rather than leaving it committed. The heartbeat was a 200ms `dispatch_source` timer on the main thread; the self-test triggered a `Compare` run and let the heartbeat keep ticking underneath it. The observed result: the heartbeat ticked continuously and evenly straight through the entire ~6.5 second `Compare` run, with no missed or delayed ticks — direct evidence that the main run loop was never blocked by the CPU or GPU render work, because both were genuinely running on the background `std::thread` this chapter has been describing, not merely dispatched-and-immediately-waited-on from the main thread. That hook is not part of the committed source (by design — it was a diagnostic, not a feature), which is why it doesn't appear in `AppDelegate.cpp` today; this chapter records the result rather than the hook itself.

§7. `saveScene()`: a design that had to change under a later feature

`saveScene()` is worth reading as a small case study in how a design constraint can be overturned by a feature added afterward, because the code and its comments still show the seam. Building a `SceneDescription` and writing it out as glTF or OBJ+MTL (Chapter 7) is cheap — no ray tracing involved, just walking the entity arrays and emitting geometry — so when the export feature was first built, `saveScene()` was a fast, synchronous, main-thread-

only operation, exactly the kind of thing that's safe to run directly inside a button-click handler without ceremony.

That stopped being true once the companion preview PNG was added (Chapter 7's `ImageWriter`): writing a same-named `.png` next to the exported geometry means actually calling `renderCPU()` at the current samples-per- pixel and max-depth settings — a real, potentially multi-second CPU render, not a cheap geometry dump. Running *that* synchronously on the main thread would reintroduce exactly the UI freeze this whole chapter has been describing renders being kept off of. So `saveScene()` was moved onto the same background-thread-plus-`dispatch_async` pattern as the three render actions:

```
void AppDelegate::saveScene( bool asGLTF )
{
        _pControlsPanel->setControlsEnabled( false );

        RenderSettings settings = _pControlsPanel->currentSettings();

        std::thread( [ this, settings, asGLTF ]()
        {
                SceneDescription scene = buildDefaultScene( settings.seed, settings.width,
kAspectRatio, settings.samplesPerPixel, settings.maxDepth, settings.floating );
                std::string      stem = sceneFilenameStem( settings.seed, settings.width,
settings.floating );

                SceneExportResult exportResult = asGLTF ? exportSceneAsGLTF( scene, stem )
: exportSceneAsOBJ( scene, stem );
                std::string       message = exportResult.message;

                if ( exportResult.ok )
                {
                        CPURenderResult preview = renderCPU( scene, settings.cpuMode );
                        std::string     directory = ensureSavedScenesDirectoryPath();
                        std::string     pngPath = directory + "/" + stem + ".png";
                        bool            pngOk = !directory.empty() && writePNG( pngPath,
preview.pixels.data(), scene.params.width, scene.params.height );
                        message += pngOk ? ( "\nand " + pngPath ) : "\n(preview image
failed to write)";
                }

                dispatch_async( dispatch_get_main_queue(), ^{
                        using NS::StringEncoding::UTF8StringEncoding;

                        // NS::Alert, not release()'d here — matches AboutAlert.cpp's
existing pattern for this
                        // project's other one-off modal dialogs.
                        NS::Alert* pAlert = NS::Alert::alloc()->init();
                        pAlert->setMessageText( NS::String::string( exportResult.ok ?
"Scene Saved" : "Save Failed", UTF8StringEncoding ) );
                        pAlert->setInformativeText( NS::String::string( message.c_str(),
UTF8StringEncoding ) );
                        pAlert->runModal();

                        std::printf( "%s\n", message.c_str() );
                        std::fflush( stdout );

                        _pControlsPanel->setControlsEnabled( true );
                } );
        } ).detach();
}
```

(`RayTracerBench/App/AppDelegate.cpp:303–342`)

The header comment above `saveScene()`'s declaration states the causality directly: it "[r]uns on a background thread, like the render actions above — a full CPU render for the preview image can take a while, unlike the geometry export alone" (`RayTracerBench/App/AppDelegate.hpp:38–42`), and the comment immediately above the definition spells out the "before/after": "once a full CPU render is part of this, it's no longer the fast, main-thread-only operation it was when it only wrote geometry" (`RayTracerBench/App/AppDelegate.cpp:300–302`). Nothing about the export logic itself demanded a background thread; it was purely a consequence of composing it with a feature (the preview render) that arrived later and carries a real cost. Note too that this function's final `dispatch_async` block runs `pAlert->runModal()` — a *blocking*, modal call — but that's fine precisely because it's back on the main thread by then: the expensive work (scene build, export, and the full CPU render) already happened off-thread, so the only thing left to block on is the user dismissing a dialog, which is supposed to be modal.

One more small echo of the theme this chapter opened with: `AboutAlert` and `saveScene()`'s result alert both use `NS::Alert` without ever releasing it — a deliberate, consistent convention for this project's one-off modal dialogs, called out explicitly in the comment at `RayTracerBench/App/AppDelegate.cpp:329–330`, rather than an oversight in one call site that the other happens to repeat.

§8. A gap `AppDelegate` runs straight into (and hands off)

`applicationDidFinishLaunching()` calls `NS::View::alloc()->init( contentFrame )` to build the plain content view at `RayTracerBench/App/AppDelegate.cpp:104`, the same `alloc()`-then-`init()` idiom used for `NS::Window` a few lines above it. That idiom is not incidental: `metal-cpp`'s vendored `NS::View::init()` originally sent `initWithFrame:` to the *class* object rather than to an allocated instance — a bug this project found via a real crash, not by inspection — and the project-local fix is exactly the `alloc()` + `init()` two-step visible here. The bug, its diagnosis, and the fix live in `ThirdParty/metal-cpp-extensions`, not in `AppDelegate.cpp` itself, so full treatment of it belongs to Chapter 10; this chapter just notes that `AppDelegate` is a direct, working beneficiary of that fix every time it constructs a view.

Where this connects

- **Chapter 4 (The CPU Renderer)** — `startCPURender()` and the CPU half of `startCompare()` both call `renderCPU()`; `saveScene()` calls it a third time to produce the preview image.

- **Chapter 5 (The GPU Renderer)** — `startGPURender()` and the GPU half of `startCompare()` call `_pGPURenderer->render()`, on the `GPURenderer` instance `applicationDidFinishLaunching()` constructs once, up front, at `RayTracerBench/App/AppDelegate.cpp:90`.
- **Chapter 7 (Scene Export: glTF, OBJ, and Preview Images)** — `saveScene()` is entirely a thin driver around `exportSceneAsGLTF()` / `exportSceneAsOBJ()`, `sceneFilenameStem()`, `ensureSavedScenesDirectoryPath()`, and `writePNG()`; this chapter covers only why and how it's threaded, not the export/PNG logic itself.
- **Chapter 9 (UI Controls and Results)** — `ControlsPanel`, `ResultsPanel`, and `AboutAlert` are constructed and wired here but implemented there; `RenderSettings` / `currentSettings()` and the five `on*` callback members are `ControlsPanel`'s surface, not `AppDelegate`'s.
- **Chapter 10 (The Metal-Backed Image View, and Filling AppKit's Gaps)** — both `ImageDisplayView`s are constructed and placed here, and the magnifier's mouse routing lives in `AppDelegate::handleMouseMoved()`, but the `CAMetalLayer` blit machinery, the `MTL_PRIVATE_IMPLEMENTATION` / `CA_PRIVATE_IMPLEMENTATION` placement this chapter contrasted itself against, and the `NS::View::init()` bug fix §8 depends on all belong to that chapter.

Chapter 9: UI Controls and Results

Abstract. This chapter covers the three files that make up RayTracerBench's control surface: `ControlsPanel`, which owns every input field, toggle, checkbox, and button the user touches to configure and trigger a render or export; `ResultsPanel`, the deliberately dumb three-line label strip that displays whatever timing text `AppDelegate` hands it; and `AboutAlert`, the one-shot attribution dialog. None of these files render anything or know how long a render took — they parse input, forward clicks through `std::function` callbacks, and display strings. The interesting engineering in this chapter is not in what these classes compute (almost nothing) but in how they talk to AppKit without a line of Objective-C++: capture-less function-pointer trampolines standing in for target/action selectors, and a real `NSButtonTypeSwitch` checkbox that needs no click handler at all, contrasted against an older toggle button that has to fake its own checked state. Files covered: `App/ControlsPanel.hpp`, `App/ControlsPanel.cpp`, `App/ResultsPanel.hpp`, `App/ResultsPanel.cpp`, `App/AboutAlert.hpp`, `App/AboutAlert.cpp`.

§1. What `ControlsPanel` owns, and what it deliberately doesn't

`ControlsPanel`'s header states its own scope precisely: it "only parses settings and forwards clicks to whichever callbacks the owner (AppDelegate) sets."

```
// image-width, samples-per-pixel, max-depth, and scene-seed fields (+ randomize), a CPU-
threading
// toggle, and Render CPU / Render GPU / Compare buttons — per CLAUDE.md's UI
architecture. Actual
// rendering stays outside this class: it only parses settings and forwards clicks to
whichever
// callbacks the owner (AppDelegate) sets.
class ControlsPanel
```

`App/ControlsPanel.hpp:20–24`

That separation is why the class has no reference to `CPURenderer`, `GPURenderer`, or `SceneExporter` beyond the `CPUThreading::Mode` enum it needs to fill in a `RenderSettings` struct — the five `std::function<void()>` members are the entire interface `AppDelegate` uses to react to a click:

```
std::function<void()> onRenderCPU;
std::function<void()> onRenderGPU;
std::function<void()> onCompare;
std::function<void()> onSaveGLTF;
std::function<void()> onSaveOBJ;
```

`App/ControlsPanel.hpp:42–46`

The struct those settings get read into mirrors exactly the fields the panel exposes — four numeric text fields, a threading mode, and the floating checkbox:

```
struct RenderSettings
{
        uint32_t           width;
        uint32_t           samplesPerPixel;
        uint32_t           maxDepth;
        unsigned           seed;
        CPUThreading::Mode cpuMode;
        bool               floating; // "Floating?" checkbox — see Scene.hpp's
buildDefaultScene()
};
```

`App/ControlsPanel.hpp:10–18`

§2. Building the row of fields, and the 860→950 window widening

`ControlsPanel`'s constructor lays out two rows of subviews inside the frame it's given: the top row holds the four labeled numeric fields plus a Randomize button, and the bottom row holds the threading toggle, the three render buttons, the Floating? checkbox, and the two Save buttons, each placed at a hand-picked x-offset:

```
_pContainerView->addSubview( makeLabel( ( CGRect ){ { 0.0, 32.0 }, { 45.0, 20.0 } },
"Width:" ) );
_pWidthField = makeField( ( CGRect ){ { 48.0, 32.0 }, { 55.0, 22.0 } }, "400" );
_pContainerView->addSubview( _pWidthField );

_pContainerView->addSubview( makeLabel( ( CGRect ){ { 115.0, 32.0 }, { 35.0, 20.0 } },
"SPP:" ) );
_pSppField = makeField( ( CGRect ){ { 153.0, 32.0 }, { 55.0, 22.0 } }, "20" );
_pContainerView->addSubview( _pSppField );

_pContainerView->addSubview( makeLabel( ( CGRect ){ { 220.0, 32.0 }, { 45.0, 20.0 } },
"Depth:" ) );
_pMaxDepthField = makeField( ( CGRect ){ { 268.0, 32.0 }, { 55.0, 22.0 } }, "20" );
_pContainerView->addSubview( _pMaxDepthField );

_pContainerView->addSubview( makeLabel( ( CGRect ){ { 335.0, 32.0 }, { 40.0, 20.0 } },
"Seed:" ) );
_pSeedField = makeField( ( CGRect ){ { 378.0, 32.0 }, { 80.0, 22.0 } }, "1234" );
_pContainerView->addSubview( _pSeedField );
```

`App/ControlsPanel.cpp:71-85`

The bottom row keeps growing rightward as the app's feature set grew: the threading toggle starts at x=0, the three render buttons follow, the Floating? checkbox sits at x=550, and the two Save buttons land at x=680 and x=800, each 110pt wide plus a gap:

```
_pSaveGLTFButton = NS::Button::alloc()->init( ( CGRect ){ { 680.0, 2.0 }, { 110.0, 26.0 }
} );
_pSaveGLTFButton->setTitle( NS::String::string( "Save glTF", UTF8StringEncoding ) );
_pSaveGLTFButton->setTarget( _pSaveGLTFButton );
_pSaveGLTFButton->setAction( NS::MenuItem::registerActionCallback(
"controlsSaveGLTFClicked", onSaveGLTFClicked ) );
_pContainerView->addSubview( _pSaveGLTFButton );

_pSaveOBJButton = NS::Button::alloc()->init( ( CGRect ){ { 800.0, 2.0 }, { 110.0, 26.0 } }
);
_pSaveOBJButton->setTitle( NS::String::string( "Save OBJ", UTF8StringEncoding ) );
_pSaveOBJButton->setTarget( _pSaveOBJButton );
_pSaveOBJButton->setAction( NS::MenuItem::registerActionCallback(
"controlsSaveOBJClicked", onSaveOBJClicked ) );
_pContainerView->addSubview( _pSaveOBJButton );
```

`App/ControlsPanel.cpp:125-135`

The Save OBJ button's right edge lands at x=910 — close enough to the original 860pt-wide window that it would have been clipped or overlapped the window's edge chrome. Per CLAUDE.md's project-status notes, adding the two export buttons is exactly why "window widened from 860 to 950 to fit the latter two" — a one-line consequence of §2's absolute-

coordinate layout that is otherwise invisible from `ControlsPanel.cpp` alone, since the window's width is set in `AppDelegate`, not here. The two files have to agree on geometry even though neither owns the other's numbers.

§3. The CPU-threading toggle: a push button faking a checked state

Before the Floating? checkbox existed, the only two-state control in this panel was the CPU-threading toggle, and it had to fake being one. It's a plain `NS::Button` (the default AppKit button type, a push button) whose *title* is overwritten on every click to describe the state that click just switched *to*:

```
_pThreadingToggleButton = NS::Button::alloc()->init( ( CGRect ){ { 0.0, 2.0 }, { 170.0,
26.0 } } );
_pThreadingToggleButton->setTitle( NS::String::string( "CPU Threads: Multi",
UTF8StringEncoding ) );
_pThreadingToggleButton->setTarget( _pThreadingToggleButton );
_pThreadingToggleButton->setAction( NS::MenuItem::registerActionCallback(
"controlsThreadingToggleClicked", onThreadingToggleClicked ) );
_pContainerView->addSubview( _pThreadingToggleButton );
```

`App/ControlsPanel.cpp:94–98`

The click handler does two things a real checkbox's click handler wouldn't have to: flip a hand-maintained `bool`, and rewrite the button's title so the label keeps matching that bool:

```
// Flips the CPU-threading mode and updates the toggle button's title to match.
void ControlsPanel::handleThreadingToggleClicked()
{
        _multiThreaded = !_multiThreaded;
        _pThreadingToggleButton->setTitle( NS::String::string(
                _multiThreaded ? "CPU Threads: Multi" : "CPU Threads: Single",
                NS::StringEncoding::UTF8StringEncoding ) );
}
```

`App/ControlsPanel.cpp:198–205`

That `_multiThreaded` field is the class's only piece of state that isn't read straight out of an AppKit control on demand:

```
NS::Button* _pThreadingToggleButton;
bool        _multiThreaded;
```

`App/ControlsPanel.hpp:80–81`

`currentSettings()` has no choice but to consult it, since the button itself has no notion of "checked":

```
settings.cpuMode = _multiThreaded ? CPUThreading::MultiThreaded :
CPUThreading::SingleThreaded;
```

`App/ControlsPanel.cpp:165`

This works, and it's what CLAUDE.md calls, in its own words, "the CPU- threading toggle's older 'push button whose title changes' hack" — a hack in the specific sense that the button's on-screen state (its title text) and the panel's logical state (`_multiThreaded`) are two separate pieces of data that a bug could let drift apart, kept in sync only because the one click handler that changes either one changes both together.

§4. The Floating? checkbox: letting AppKit own its own state

The Floating? checkbox is a genuine `NSButtonTypeSwitch` button — the AppKit control macOS renders as a checkbox — created by setting the button type after allocation and never touched again:

```
// A real checkbox — AppKit manages its own checked state on click, so no target/action
wiring
// is needed at all; currentSettings() just reads state() on demand.
_pFloatingCheckbox = NS::Button::alloc()->init( ( CGRect ){ { 550.0, 2.0 }, { 110.0, 26.0
} } );
_pFloatingCheckbox->setButtonType( NS::ButtonTypeSwitch );
_pFloatingCheckbox->setTitle( NS::String::string( "Floating?", UTF8StringEncoding ) );
_pContainerView->addSubview( _pFloatingCheckbox );
```

`App/ControlsPanel.cpp:118–123`

Notice everything that's *missing* compared to every other button in this file: no `setTarget`, no `setAction`, no `registerActionCallback`, and no entry in the trampoline table at the top of `ControlsPanel.cpp`. That omission is the whole point.
`setButtonType` / `setState` / `state()` are project-local additions to `NSButton.hpp` (part of the metal-cpp-extensions gap-filling story Chapter 10 covers in full), and their contract is that AppKit itself flips the button's internal checked/unchecked state whenever the user clicks it — the class comment on the header spells this out directly:

```
// A real NSButtonTypeSwitch checkbox rather than the threading toggle's "push button
whose
// title changes" hack — AppKit manages its own checked/unchecked state on click, so this
// needs no click handler at all; currentSettings() just reads state() on demand.
NS::Button* _pFloatingCheckbox;
```

`App/ControlsPanel.hpp:83–86`

So where the threading toggle needs a `bool` field, a click handler, and a title rewrite just to answer "am I on or off," the checkbox needs none of that — `currentSettings()` just asks it:

```
settings.floating = _pFloatingCheckbox->state() != 0;
```

`App/ControlsPanel.cpp:166`

`state()` sends the real `NSButton` `state` selector via `Object::sendMessage`, returning `1` when checked and `0` when unchecked (the sign convention documented directly on `NSButton.hpp`'s declaration: "`state()` is 1 when checked, 0 when unchecked. AppKit toggles this itself on click — no manual bookkeeping needed, unlike a plain push button" — `ThirdParty/metal-cpp-extensions/AppKit/NSButton.hpp:39–43`). The two controls sit right next to each other in the same source file, one hand- rolling a piece of UI state AppKit already tracks internally, the other reading that internal state directly — the checkbox is strictly less code and strictly less room for the label and the boolean to disagree, which is exactly the improvement CLAUDE.md credits it with.

§5. `currentSettings()`: read once, synchronously, before any thread starts

`currentSettings()` itself is a small, ordinary function — four calls to a local `parseUInt` helper, one enum lookup, one `state()` read — but *when* it's called matters more than what it does. Every field is parsed with a clamped fallback rather than allowed to throw or read garbage on bad input:

```
// Parses a text field as an unsigned integer, clamped to [minVal, maxVal]; returns
`fallback`
// if the field doesn't start with a valid number.
uint32_t parseUInt( NS::TextField* pField, uint32_t minVal, uint32_t maxVal, uint32_t
fallback )
{
        const char* pText = pField->stringValue()->utf8String();
        char*       pEnd = nullptr;
        long        value = std::strtol( pText, &pEnd, 10 );
        if ( pEnd == pText )
                return fallback;
        if ( value < (long)minVal )
                value = minVal;
        if ( value > (long)maxVal )
                value = maxVal;
        return (uint32_t)value;
}
```

`App/ControlsPanel.cpp:21–35`

```
RenderSettings ControlsPanel::currentSettings() const
{
        RenderSettings settings;
        settings.width = parseUInt( _pWidthField, 50, 2000, 400 );
        settings.samplesPerPixel = parseUInt( _pSppField, 1, 2000, 20 );
        settings.maxDepth = parseUInt( _pMaxDepthField, 1, 100, 20 );
        settings.seed = parseUInt( _pSeedField, 0, 0xFFFFFFFFu, 1234 );
        settings.cpuMode = _multiThreaded ? CPUThreading::MultiThreaded :
CPUThreading::SingleThreaded;
        settings.floating = _pFloatingCheckbox->state() != 0;
        return settings;
}
```

`App/ControlsPanel.cpp:158–168`

Chapter 8 covers in full why `AppDelegate` always calls this on the main thread, synchronously, before spawning the `std::thread` that actually renders — briefly, the point is that every AppKit control (`NS::TextField`, `NS::Button`) may only be touched from the main thread, so `currentSettings()` takes its one and only snapshot of the UI's numeric/boolean state up front, hands a plain `RenderSettings` value (no AppKit objects, safe to touch from anywhere) into the background thread's capture list, and never gets called again until that render finishes. `setControlsEnabled(false)` — disabling every field and button for the render's duration — is the other half of that same guarantee: it closes the window during which a second click could change what `currentSettings()` would report, rather than trying to make `currentSettings()` itself thread-safe.

```
void ControlsPanel::setControlsEnabled( bool enabled )
{
        _pWidthField->setEnabled( enabled );
        _pSppField->setEnabled( enabled );
        _pMaxDepthField->setEnabled( enabled );
        _pSeedField->setEnabled( enabled );
        _pThreadingToggleButton->setEnabled( enabled );
        _pRandomizeSeedButton->setEnabled( enabled );
        _pRenderCPUButton->setEnabled( enabled );
        _pRenderGPUButton->setEnabled( enabled );
        _pCompareButton->setEnabled( enabled );
        _pFloatingCheckbox->setEnabled( enabled );
        _pSaveGLTFButton->setEnabled( enabled );
        _pSaveOBJButton->setEnabled( enabled );
}
```

`App/ControlsPanel.cpp:172–186`

§6. Randomize Seed: the one handler that needs no forwarding

Unlike the five render/export buttons, whose handlers are one-line `std::function` forwards to whatever `AppDelegate` set, Randomize Seed is handled entirely inside `ControlsPanel` — it never needs to leave this class, since randomizing the seed field is pure UI state with no renderer involved:

```
// Fills the seed field with a freshly generated random seed.
void ControlsPanel::handleRandomizeSeedClicked()
{
        std::random_device rd;
        unsigned           newSeed = rd();
        char               buf[ 16 ];
        std::snprintf( buf, sizeof( buf ), "%u", newSeed );
        _pSeedField->setStringValue( NS::String::string( buf,
NS::StringEncoding::UTF8StringEncoding ) );
}
```

`App/ControlsPanel.cpp:189–196`

It draws straight from `std::random_device` rather than the seeded `std::mt19937` `Scene::buildDefaultScene()` uses internally (Chapter 1) — appropriately, since this call exists to *pick* a new seed for that generator, not to be reproducible itself.

§7. The trampoline idiom, briefly — full story in Chapter 10

Every clickable button in this file follows the same four-line pattern: allocate, set a title, set the button as its own target, and register an action callback through a file-local free function:

```
_pRandomizeSeedButton = NS::Button::alloc()->init( ( CGRect ){ { 465.0, 30.0 }, { 90.0,
26.0 } } );
_pRandomizeSeedButton->setTitle( NS::String::string( "Randomize", UTF8StringEncoding ) );
_pRandomizeSeedButton->setTarget( _pRandomizeSeedButton );
_pRandomizeSeedButton->setAction( NS::MenuItem::registerActionCallback(
"controlsRandomizeSeedClicked", onRandomizeSeedClicked ) );
_pContainerView->addSubview( _pRandomizeSeedButton );
```

`App/ControlsPanel.cpp:87–91`

`registerActionCallback` wants a capture-less function pointer, not a lambda with captures, so `ControlsPanel.cpp` keeps a single file-local pointer to the live panel instance and a small table of trampolines that route each callback through it:

```
namespace
{
        ControlsPanel* gControlsPanel = nullptr;

        // Button click trampolines: metal-cpp-extensions' action callbacks are capture-
less function
        // pointers, so each reaches the current panel through the file-local
gControlsPanel pointer.
        void onRandomizeSeedClicked( void*, SEL, const NS::Object* ) { gControlsPanel-
>handleRandomizeSeedClicked(); }
        void onThreadingToggleClicked( void*, SEL, const NS::Object* ) { gControlsPanel-
>handleThreadingToggleClicked(); }
        void onRenderCPUClicked( void*, SEL, const NS::Object* ) { gControlsPanel-
>handleRenderCPUClicked(); }
        void onRenderGPUClicked( void*, SEL, const NS::Object* ) { gControlsPanel-
>handleRenderGPUClicked(); }
        void onCompareClicked( void*, SEL, const NS::Object* ) { gControlsPanel-
>handleCompareClicked(); }
        void onSaveGLTFClicked( void*, SEL, const NS::Object* ) { gControlsPanel-
>handleSaveGLTFClicked(); }
        void onSaveOBJClicked( void*, SEL, const NS::Object* ) { gControlsPanel-
>handleSaveOBJClicked(); }
        ...
}
```

`App/ControlsPanel.cpp:7–19`

This is the same idiom `AppDelegate` uses for its own menu-item callback (`onShowAboutClicked` in `App/AppDelegate.cpp:27–30`) and it's how every AppKit control

callback in this project gets wired without Objective-C++ — `NS::Control` / `NS::Button`'s `setTarget` / `setAction`, and `NS::MenuItem::registerActionCallback` underneath them, ultimately rest on `Object::sendMessage` / `class_addMethod`, the same mechanism `NSButton.hpp` itself is built from (§4 above). *Why* Apple's vendored `metal-cpp-extensions` needed `NS::Control` / `NS::Button` added at all, and how `registerActionCallback` bridges a plain C++ function pointer into an Objective-C selector, is Chapter 10's subject in full; this chapter only notes the pattern as it's used here.

§8. `ResultsPanel`: three labels, no arithmetic

`ResultsPanel`'s header is explicit that the class does no computation of its own:

```
// Three label lines (CPU / GPU / speedup) per CLAUDE.md's UI architecture. Deliberately
dumb: it
// just displays whatever text it's given — all render-time/rays-per-sec/speedup
formatting stays
// in AppDelegate, where the actual timing values are known.
class ResultsPanel
```

`App/ResultsPanel.hpp:7-10`

Its constructor places three plain (non-editable, non-bezeled) label fields stacked vertically, each starting with placeholder text:

```
_pCPULabel = makeLabel( ( CGRect ){ { 0.0, 46.0 }, { frame.size.width, 20.0 } }, "CPU: not
yet run" );
_pContainerView->addSubview( _pCPULabel );

_pGPULabel = makeLabel( ( CGRect ){ { 0.0, 24.0 }, { frame.size.width, 20.0 } }, "GPU: not
yet run" );
_pContainerView->addSubview( _pGPULabel );

_pSpeedupLabel = makeLabel( ( CGRect ){ { 0.0, 2.0 }, { frame.size.width, 20.0 } }, "Run
both CPU and GPU to compare" );
_pContainerView->addSubview( _pSpeedupLabel );
```

`App/ResultsPanel.cpp:19-26`

and its entire public surface beyond that is three setters that do nothing but forward a pre-formatted string into the matching label:

```
void ResultsPanel::setCPULine( const std::string& text )
{
        _pCPULabel->setStringValue( NS::String::string( text.c_str(),
NS::StringEncoding::UTF8StringEncoding ) );
}
```

`App/ResultsPanel.cpp:45–48`

The actual numbers — render time, GPU-only time, estimated rays/sec, and the labeled speedup ratio — are computed in `AppDelegate`, which is the only place that has both a `RenderParams` (width/height/samplesPerPixel) and a measured elapsed time in hand at the same moment. Rays/sec is a single helper shared by every render path:

```
// Estimates rays/sec as width*height*samplesPerPixel divided by elapsed time.
double raysPerSecond( const RenderParams& params, double milliseconds )
{
        double rays = (double)params.width * (double)params.height *
(double)params.samplesPerPixel;
        return rays / ( milliseconds / 1000.0 );
}
```

`App/AppDelegate.cpp:19–24`

and the speedup line is only ever produced once both a CPU and a GPU timing exist, formatted as whichever side is faster:

```
// Once both a CPU and a GPU timing are known, formats and displays whichever ratio is >=
1x
// ("GPU is Nx faster" or "CPU is Nx faster"). No-ops until both times exist.
void AppDelegate::updateSpeedupIfPossible()
{
        if ( _lastCPUTimeMs < 0.0 || _lastGPUTimeMs < 0.0 )
                return;

        char buf[ 128 ];
        if ( _lastGPUTimeMs < _lastCPUTimeMs )
        {
                double ratio = _lastCPUTimeMs / _lastGPUTimeMs;
                std::snprintf( buf, sizeof( buf ), "GPU is %.1fx faster than CPU", ratio
);
        }
        else
        {
                double ratio = _lastGPUTimeMs / _lastCPUTimeMs;
                std::snprintf( buf, sizeof( buf ), "CPU is %.1fx faster than GPU", ratio
);
        }
        _pResultsPanel->setSpeedupLine( buf );
}
```

`App/AppDelegate.cpp:277–294`

This division of labor mirrors `ControlsPanel`'s: the view-owning class knows only how to display text in a labeled field, and the class that owns the actual renderers and timers (`AppDelegate`, Chapter 8) is the only one that touches `RenderParams` or a stopwatch value.

§9. `AboutAlert`: attribution, via the project-local `NSAlert.hpp`

`AboutAlert` is the smallest file in this chapter — one free function, `showAboutAlert()`, that builds and runs an `NS::Alert` modally:

```
void showAboutAlert()
{
        using NS::StringEncoding::UTF8StringEncoding;

        NS::Alert* pAlert = NS::Alert::alloc()->init();
        pAlert->setMessageText( NS::String::string( "RayTracerBench", UTF8StringEncoding )
);
        pAlert->setInformativeText( NS::String::string(
                "A CPU-vs-GPU (Metal compute) ray tracing benchmark, built in pure C++ and
Metal via "
                "Apple's metal-cpp/metal-cpp-extensions.\n\n"
                "Based on the algorithm and structure of \"Ray Tracing in One Weekend\" by
Peter Shirley "
                "et al. (raytracing.github.io, CC0) and its CUDA port by Roger Allen "
                "(github.com/rogerallen/raytracinginoneweekendincuda, public domain).
Neither license "
                "requires attribution; this note is a courtesy credit, not a legal one.",
                UTF8StringEncoding ) );
        pAlert->runModal();
}
```

`App/AboutAlert.cpp:6-21`

Its header explains why the note exists at all even though nothing legally requires it:

```
// An NS::Alert carrying the courtesy attribution note this project's reference sources
don't
// legally require (both are CC0/public-domain) but credits anyway. See CLAUDE.md's
Attribution
// requirements section.
void showAboutAlert();
```

`App/AboutAlert.hpp:3-6`

That matches CLAUDE.md's attribution requirements directly: both reference repositories (`RayTracing/raytracing.github.io` and `rogerallen/raytracinginoneweekendincuda`) are CC0/public-domain and impose no legal obligation, but the project still credits both here (and in `README.md`), alongside an MIT license listing Pablo Figueroa and Claude as authors. `NS::Alert` itself — like `NS::Button` and `NS::TextField` used throughout this chapter — is a project-local addition to `metal-cpp-extensions` built with the same `Object::sendMessage` / `class_addMethod` idiom as §7's button trampolines, since Apple's vendored headers have no alert wrapper either; that gap-filling story belongs to Chapter 10.

Where this connects

- **Chapter 8** (`main.cpp`, `App/AppDelegate.hpp/.cpp`) owns the instances of `ControlsPanel` and `ResultsPanel` this chapter describes, wires the five `std::function` callbacks (`onRenderCPU`, `onRenderGPU`, `onCompare`, `onSaveGLTF`, `onSaveOBJ`) to its own `start*`/`saveScene` methods, is the sole caller of `currentSettings()` (always synchronously, on the main thread, before spawning the background `std::thread` that actually renders — §5), and computes every number `ResultsPanel`'s setters merely display (§8).
- **Chapter 10** (`App/ImageDisplayView.hpp/.cpp`, plus the `ThirdParty/metal-cpp-extensions` additions) tells the full story behind the `NS::Button`/`NS::TextField`/`NS::Alert`/`NS::Control` additions this chapter's panels are built entirely out of — the `Object::sendMessage`/`class_addMethod` idiom introduced only briefly here in §4 and §7.
- **Chapter 1** (`Core/Scene.hpp/.cpp`) is where the Floating? checkbox's `bool` actually does something: `settings.floating` (§4) flows into `Scene::buildDefaultScene()`'s `floating` parameter, which places the small randomized-field spheres at a random height instead of resting them on the ground.

Chapter 10: The Metal-Backed Image View, and Filling AppKit's Gaps

Abstract. `ImageDisplayView` is the widget that puts pixels on screen: two instances sit side by side in the app window, one showing the CPU renderer's output and one showing the GPU renderer's, and both support a synchronized magnifying-glass loupe for comparing fine detail between them. Getting there required more than writing the view itself, because Apple's `metal-cpp` and `metal-cpp-extensions` — the header-only C++ bindings this project uses in place of Objective-C++ — simply don't define several of the AppKit and QuartzCore classes this view needs: `CA::MetalLayer` doesn't exist at all, and Apple's own DTS has confirmed there is no `NS::Control` / `NS::Button` wrapper of any kind. This chapter covers both halves together: the view itself, and the project-local additions to `ThirdParty/metal-cpp-extensions` that had to be written, verified standalone, and only then built upon.

Files covered. `RayTracerBench/App/ImageDisplayView.hpp`, `RayTracerBench/App/ImageDisplayView.cpp`, and the project-local additions under `ThirdParty/metal-cpp-extensions/`: `QuartzCore/CAMetalLayer.hpp`, `AppKit/NSView.hpp`, `AppKit/NSControl.hpp`, `AppKit/NSButton.hpp`, `AppKit/NSTextField.hpp`, `AppKit/NSAlert.hpp`, and `AppKit/NSEvent.hpp`.

§1. A `CAMetalLayer`-backed view, deliberately not `NSImageView` or `MTKView`

The header comment states the design decision before anything else:

```
// A CAMetalLayer-backed NS::View that blits a texture onto screen via a tiny textured-
quad shader
// (Shaders/Blit.metal) — deliberately not NSImageView, per CLAUDE.md's UI architecture.
Used two
// ways: updatePixels() uploads a CPU-side RGBA8 buffer into a texture this view owns;
displayTexture()
// blits an already-existing MTL::Texture (the GPU renderer's output) directly, borrowed
rather than
// owned — it stays GPURenderer's responsibility to free.
```

`RayTracerBench/App/ImageDisplayView.hpp:9-13`

Two things are being ruled out here, not just one. `NSImageView` is rejected because the two sources of pixels this view has to display are fundamentally different: the CPU renderer

produces a plain RGBA8 byte buffer that has to be uploaded somewhere, while the GPU renderer's output is already a live `MTL::Texture` sitting in device memory. Routing both through `NSImageView` would mean wrapping a `MTL::Texture` back into a `CGImage` just to hand it to an image view that would, at some point, hand it back to the GPU to actually draw — a round trip with no purpose. `MTKView` is rejected too, even though it exists specifically to host Metal content, because it comes with its own CVDisplayLink-driven draw loop and delegate protocol, machinery this project has no use for (see §2) and that would have to be bridged through in Objective-C anyway. What `ImageDisplayView` actually is, per the field declarations, is a plain `NS::View` whose backing layer is manually swapped for a `CA::MetalLayer`, with the view's own render pipeline driving that layer directly:

```
MTL::Device*              _pDevice;
CA::MetalLayer*           _pMetalLayer;
NS::View*                 _pView;
MTL::CommandQueue*        _pCommandQueue;
MTL::RenderPipelineState* _pPipelineState;
CGSize                    _viewSize;

MTL::Texture* _pOwnedTexture; // created/uploaded by updatePixels(); released in the
destructor
uint32_t      _ownedTextureWidth;
uint32_t      _ownedTextureHeight;

MTL::Texture*      _pCurrentTexture; // whichever of the above (or an external texture)
render() should sample
MagnifierUniforms  _magnifier;
```

`RayTracerBench/App/ImageDisplayView.hpp:60–72`

The constructor wires the layer onto the view exactly the way a `wantsLayer` / `setLayer:`-based Cocoa view is supposed to be built, then compiles `Blit.metal` (Chapter 3) from source and builds a render pipeline state around it, the same "read shader source from a sibling file at runtime" approach `main.cpp`'s comments describe for this not being an app bundle with a compiled `default.metallib`:

```
_pView = NS::View::alloc()->init( frame );
_pView->setWantsLayer( true );

_pMetalLayer = CA::MetalLayer::layer();
_pMetalLayer->setDevice( _pDevice );
_pMetalLayer->setPixelFormat( MTL::PixelFormatBGRA8Unorm );
_pMetalLayer->setFramebufferOnly( true );
_pMetalLayer->setDrawableSize( frame.size );
_pView->setLayer( _pMetalLayer );

std::string source = readShaderSource( "Blit.metal" );

NS::Error* pError = nullptr;
MTL::Library* pLibrary = _pDevice->newLibrary( NS::String::string( source.c_str(),
UTF8StringEncoding ), nullptr, &pError );
```

`RayTracerBench/App/ImageDisplayView.cpp:47–60`

§2. `updatePixels()`: no continuous draw loop, because this isn't an animation

Every render this view shows is a one-shot: the CPU or GPU renderer finishes a frame, hands it over, and the view draws it exactly once. There is no per-frame callback, no display link, nothing ticking in the background — which is precisely the machinery `MTKView` would have supplied and that this view deliberately does without (§1). `updatePixels()` is the CPU-side entry point, and its whole job is: make sure the destination texture is the right size, upload the bytes, and immediately render+present:

```
// Uploads an RGBA8 buffer into the owned texture (recreating it if the size changed),
then
// immediately renders and presents.
void ImageDisplayView::updatePixels( const uint8_t* pRGBA, uint32_t width, uint32_t height
)
{
        rebuildOwnedTextureIfNeeded( width, height );

        MTL::Region region = MTL::Region( 0, 0, 0, width, height, 1 );
        _pOwnedTexture->replaceRegion( region, 0, pRGBA, static_cast<size_t>( width ) * 4
);

        _pCurrentTexture = _pOwnedTexture;
        _pMetalLayer->setDrawableSize( CGSizeMake( width, height ) );
        render();
}
```

`RayTracerBench/App/ImageDisplayView.cpp:124–134`

`rebuildOwnedTextureIfNeeded()` is the guard that keeps a texture from being thrown away and recreated on every single frame — a render preview at a fixed image width produces the same-sized buffer call after call, so the common case is simply overwriting the existing texture's contents via `replaceRegion`:

```
// (Re)creates _pOwnedTexture only when width/height actually differ from the last call.
void ImageDisplayView::rebuildOwnedTextureIfNeeded( uint32_t width, uint32_t height )
{
        if ( _pOwnedTexture && width == _ownedTextureWidth && height ==
_ownedTextureHeight )
                return;

        if ( _pOwnedTexture )
                _pOwnedTexture->release();

        MTL::TextureDescriptor* pDesc = MTL::TextureDescriptor::texture2DDescriptor(
MTL::PixelFormatRGBA8Unorm, width, height, false );
        pDesc->setStorageMode( MTL::StorageModeShared );
        pDesc->setUsage( MTL::TextureUsageShaderRead );

        _pOwnedTexture = _pDevice->newTexture( pDesc );
        _ownedTextureWidth = width;
        _ownedTextureHeight = height;
}
```

`RayTracerBench/App/ImageDisplayView.cpp:105–120`

The GPU path, `displayTexture()`, skips the upload step entirely — the `GPURenderer`'s output is already a texture in shared storage, so this view just points at it and presents, without ever taking ownership (the header comment above is explicit that the borrowed texture "stays GPURenderer's responsibility to free"):

```
// Points rendering at an externally-owned texture (e.g. the GPU renderer's output) and
presents.
void ImageDisplayView::displayTexture( MTL::Texture* pTexture )
{
        _pCurrentTexture = pTexture;
        _pMetalLayer->setDrawableSize( CGSizeMake( pTexture->width(), pTexture->height() )
);
        render();
}
```

`RayTracerBench/App/ImageDisplayView.cpp:137–142`

Both paths converge on the same private `render()`, which grabs one drawable from the `CAMetalLayer`, encodes a single full-screen-quad draw through `Blit.metal`'s pipeline, and presents:

```
CA::MetalDrawable* pDrawable = _pMetalLayer->nextDrawable();
if ( !pDrawable )
        return;
/* ... render-pass/encoder setup ... */
pEncoder->setFragmentTexture( _pCurrentTexture, 0 );
pEncoder->setFragmentBytes( &_magnifier, sizeof( _magnifier ), 0 );
pEncoder->drawPrimitives( MTL::PrimitiveTypeTriangle, ( NS::UInteger )0, ( NS::UInteger )6
);
pEncoder->endEncoding();

pCommandBuffer->presentDrawable( pDrawable );
pCommandBuffer->commit();
```

`RayTracerBench/App/ImageDisplayView.cpp:161-181` (elided: render-pass descriptor and color-attachment setup, `RayTracerBench/App/ImageDisplayView.cpp:165-170`)

`setMagnifier()` also calls `render()` directly whenever a texture is already showing, which is what lets the loupe (§5) track the mouse live without any new pixel data arriving — the same one-shot `render()` is reused for "new frame" and "same frame, new lens position" alike, because from this view's perspective they're the same operation: re-encode one blit with whatever the current `_magnifier` uniforms happen to be.

§3. `CA::MetalLayer`: forward-declared everywhere, defined nowhere

`ImageDisplayView` needs a real `CA::MetalLayer` type — something it can call `layer()`, `setDevice()`, `setPixelFormat()`, `setDrawableSize()`, and `nextDrawable()` on — but vendored `metal-cpp` doesn't provide one. The only place `CA::MetalLayer` appears in Apple's own headers is as a forward declaration, used solely as the return type of `CA::MetalDrawable::layer()`:

```
/*
 * Project-local addition — NOT part of Apple's metal-cpp-extensions.
 *
 * Base metal-cpp only forward-declares CA::MetalLayer (as the return type of
 * CA::MetalDrawable::layer()) and never defines it — there is no vendored
 * CAMetalLayer wrapper at all, in either metal-cpp or metal-cpp-extensions.
 * This header fills that gap using the same Object::sendMessage idiom
 * Apple's own headers use elsewhere. See also AppKit/NSControl.hpp.
 */
```

`ThirdParty/metal-cpp-extensions/QuartzCore/CAMetalLayer.hpp:1-11`

The fix is a project-local header at `ThirdParty/metal-cpp-extensions/QuartzCore/CAMetalLayer.hpp` that defines the type properly, using exactly the

idiom the rest of `metal-cpp` already uses everywhere else — `NS::Referencing<>` for retain/release-managed lifetime, and each method a thin `Object::sendMessage<>()` call resolving a selector by name:

```
class MetalLayer : public NS::Referencing<MetalLayer>
{
        public:
                static MetalLayer* layer();

                void setDevice( const MTL::Device* pDevice );
                void setPixelFormat( MTL::PixelFormat pixelFormat );
                void setFramebufferOnly( bool framebufferOnly );
                void setDrawableSize( CGSize size );

                MetalDrawable* nextDrawable();
};

_NS_INLINE CA::MetalLayer* CA::MetalLayer::layer()
{
        return NS::Object::sendMessage<MetalLayer*>( objc_getClass( "CAMetalLayer" ),
sel_registerName( "layer" ) );
}
```

`ThirdParty/metal-cpp-extensions/QuartzCore/CAMetalLayer.hpp:32-51`

Nothing here is a new idea — `objc_getClass` / `sel_registerName` / `Object::sendMessage<>()` is precisely how every other class in vendored `metal-cpp` is implemented — but the class itself simply had a hole where `CAMetalLayer` should have been, and `ImageDisplayView` is the one place in this codebase that needed it filled.

§4. `NS::View::init()`: a real crash, not a missing feature

Unlike `CA::MetalLayer`, `NS::View` does exist in vendored `metal-cpp-extensions` — but its `init()` had an outright bug, discovered the way most convincing bugs are discovered: it crashed. Apple's shipped version sent `initWithFrame:` to the `NSView` *class* object rather than to an instance that had actually been `alloc()`'d, which is the Objective-C runtime equivalent of calling a constructor on a type instead of an object — it raises `"unrecognized selector sent to class"` at runtime, not a compile error, since Objective-C message sends are resolved dynamically. The fix mirrors the pattern already used correctly elsewhere in the same file for `NS::Window`: a separate `alloc()` that returns an allocated-but-uninitialized instance, and an `init()` that operates on `this` rather than on the class:

```
// alloc() + this fixed init() are a project-local correction: Apple's
// original vendored init() sent initWithFrame: to the NSView *class*
// object instead of to an allocated instance, which crashes at runtime
// ("unrecognized selector sent to class") — verified by hand before
// fixing. addSubview()/setWantsLayer()/setLayer() are project-local
// additions filling in AppKit surface Apple's extensions never wrapped
// (see AppKit/NSControl.hpp for the same situation with NS::Button).
static View* alloc();
View*            init( CGRect frame );
```

`ThirdParty/metal-cpp-extensions/AppKit/NSView.hpp:39-47`

```
_NS_INLINE NS::View* NS::View::alloc()
{
        return Object::sendMessage< View* >( _APPKIT_PRIVATE_CLS( NSView ),
_NS_PRIVATE_SEL( alloc ) );
}

_NS_INLINE NS::View* NS::View::init( CGRect frame )
{
        return Object::sendMessage< View* >( this, _APPKIT_PRIVATE_SEL( initWithFrame_ ),
frame );
}
```

`ThirdParty/metal-cpp-extensions/AppKit/NSView.hpp:60-68`

Every `NS::View::alloc()->init( frame )` call in this codebase — including the two `ImageDisplayView` construction sites in `AppDelegate.cpp` and `ImageDisplayView.cpp:47` itself — depends on this fix; without it, simply constructing the view that hosts the render preview would crash the app before a single pixel could be drawn. The same header also picked up three methods that were missing outright rather than broken: `addSubview()` (used to attach both `ImageDisplayView`s, `ControlsPanel`, and `ResultsPanel` to the window's content view in `AppDelegate.cpp:108-118`), and `setWantsLayer()` / `setLayer()` — the pair `ImageDisplayView`'s constructor uses to swap in the `CA::MetalLayer` from §3 (`RayTracerBench/App/ImageDisplayView.cpp:48,55`). `convertPoint()`, the fourth method in this file, belongs to the magnifier feature and is covered in §5.

§5. The private-implementation macro rule

Metal and QuartzCore's `metal-cpp` bindings are header-only, but each of their headers still needs *some* translation unit to actually emit the real class/selector-cache symbols the header declares — that's what `MTL_PRIVATE_IMPLEMENTATION` and `CA_PRIVATE_IMPLEMENTATION` do when `#define`d immediately before the corresponding header is first included. Define one of

these macros in more than one `.cpp`, or in none, and the result is either duplicate-symbol or undefined-symbol linker errors — `Private::Class` / `Private::Selector` references with nothing backing them. The rule, as `main.cpp`'s own comment states it, is one macro per header, defined in whichever single translation unit includes that header first:

```
// Metal/QuartzCore's own private-implementation macros (MTL_PRIVATE_IMPLEMENTATION,
// CA_PRIVATE_IMPLEMENTATION) live in ImageDisplayView.cpp instead, since that's the one
// translation unit that actually includes those headers — each must be defined exactly
once,
// in whichever .cpp first includes the corresponding header.
#define NS_PRIVATE_IMPLEMENTATION
#include <AppKit/AppKit.hpp>
```

`RayTracerBench/main.cpp:1–6`

`main.cpp` owns `NS_PRIVATE_IMPLEMENTATION` because it's the translation unit that includes `<AppKit/AppKit.hpp>` (which pulls in Foundation). `ImageDisplayView.cpp` is, symmetrically, the one file in the project that actually includes `Metal/Metal.hpp` and `QuartzCore/CAMetalLayer.hpp` for real Metal-and-layer work (as opposed to just declaring pointers to `MTL::Device` / `MTL::Texture` the way other files do), so it carries the other two macros:

```
// This is the one translation unit that includes Metal/QuartzCore headers, so it's the
one that
// must actually define their private-implementation macros (each emits real global symbol
// definitions for its header's private class/selector caches — must happen in exactly one
TU).
// NS_PRIVATE_IMPLEMENTATION already lives in main.cpp, which owns Foundation/AppKit
instead.
#define MTL_PRIVATE_IMPLEMENTATION
#define CA_PRIVATE_IMPLEMENTATION
#include "ImageDisplayView.hpp"
```

`RayTracerBench/App/ImageDisplayView.cpp:1–7`

This is a small piece of code but an easy one to get wrong by copy-paste, since nothing about the macro name suggests *which* file should hold it — only which header it pairs with and the one-definition-per-header-inclusion rule above.

§6. The magnifier loupe, from the host side

The loupe's purpose, stated directly in `AppDelegate.hpp`'s comment, is comparison rather than mere magnification: hovering over *either* preview zooms the *same* normalized position in

both, so a user can see whether the CPU and GPU renderers actually agree on fine detail at a given pixel — not just get a bigger picture of one image in isolation.

```
// Magnifying-glass loupe: a local NS::EventMaskMouseMoved monitor (installed once, at
// launch) reports the same normalized UV position to both ImageDisplayViews whenever the
// mouse is over either preview, so hovering one image zooms the matching detail in both —
// the actual point of the feature being to compare CPU vs. GPU output at high zoom.
void handleMouseMoved( NS::Event* pEvent );
```

`RayTracerBench/App/AppDelegate.hpp:48–52`

The shader-side lens math (sampling a de-magnified region within a circular, aspect-corrected radius, with a thin border ring) belongs to `Blit.metal` and is Chapter 3's territory; what this chapter covers is how that shader gets fed live mouse coordinates from the host side, since `render()` (§2) only ever consumes whatever `_magnifier` currently holds. `ImageDisplayView` exposes exactly one entry point for this, `setMagnifier()`, taking a lens center in the same normalized `[0,1]` UV convention Blit.metal expects (`RayTracerBench/App/ImageDisplayView.hpp:37–41`) and immediately re-rendering if a texture is already on screen.

Feeding that entry point requires knowing, continuously, where the mouse is — and AppKit's mechanism for that is a local event monitor, which `metal-cpp` also had no wrapper for. `AppKit/NSEvent.hpp` is a third project-local addition, filling in just the one event mask this project needs (`NSEventTypeMouseMoved`'s value has been ABI-stable for decades, so it's hardcoded rather than reconstructed from a full enum) and the block-based registration API:

```
// NSEventMask is a 64-bit bitmask indexed by NSEventType (1ULL << type). Only the one
mask this
// project actually needs is defined here — NSEventTypeMouseMoved's value (5) has been
ABI-stable
// AppKit API for decades, so this is safe to hardcode rather than needing a full
NSEventType enum.
constexpr uint64_t EventMaskMouseMoved = 1ULL << 5;

class Event : public Referencing< Event >
{
        public:
                // A raw Objective-C block, same idiom metal-cpp's own
MTL::Device::newBuffer(...)
                // deallocator parameter already uses elsewhere in this vendored codebase.
                typedef Event* ( ^MonitorHandler )( Event* pEvent );

                static Event* addLocalMonitorForEventsMatchingMask( uint64_t mask,
MonitorHandler handler );

                CGPoint     locationInWindow() const;
                class Window* window() const;
};
```

`ThirdParty/metal-cpp-extensions/AppKit/NSEvent.hpp:25-41`

The comment's cross-reference is worth taking literally: `MonitorHandler` is declared as a genuine Objective-C block type (`Event (^)(Event)`), and that compiles and links fine from plain C++ because this project already relies on the same fact elsewhere — `MTL::Device::newBuffer`'s deallocator parameter, vendored by Apple itself, is a block passed through `sendMessage` the same way. `AppDelegate::setup()` installs the monitor once, at launch, using that same block syntax directly in C++:

```
NS::Event::addLocalMonitorForEventsMatchingMask( NS::EventMaskMouseMoved, ^NS::Event*(
NS::Event* pEvent ) {
        handleMouseMoved( pEvent );
        return pEvent;
} );
```

`RayTracerBench/App/AppDelegate.cpp:131-134`

`handleMouseMoved()` then has to turn a window-space point into a normalized UV coordinate local to whichever preview the mouse is over — and that's `NS::View::convertPoint()`, the fourth method added to `NSView.hpp` alongside the `init()` fix in §4, implementing Cocoa's documented `convertPoint:fromView:` (`pFromView == nullptr` meaning "convert from the window's own coordinate system"):

```
_NS_INLINE CGPoint NS::View::convertPoint( CGPoint point, const NS::View* pFromView )
const
{
        return Object::sendMessage< CGPoint >( this, sel_registerName(
"convertPoint:fromView:" ), point, pFromView );
}
```

`ThirdParty/metal-cpp-extensions/AppKit/NSView.hpp:85-88`

```
const CGPoint windowPoint = pEvent->locationInWindow();

const CGPoint cpuLocal = _pCPUImageView->view()->convertPoint( windowPoint, nullptr );
const CGSize  cpuSize = _pCPUImageView->size();
const bool    overCPU = cpuLocal.x >= 0.0 && cpuLocal.x <= cpuSize.width && cpuLocal.y >=
0.0 && cpuLocal.y <= cpuSize.height;
```

`RayTracerBench/App/AppDelegate.cpp:144-148`

The last step is the one coordinate-space mismatch that has to be corrected by hand: AppKit view coordinates are Y-up with the origin at the bottom-left, while the source texture's V convention — shared with `CPURenderer` and `Blit.metal` — is Y-down, row 0 at the top. `AppDelegate` flips it explicitly before ever calling `setMagnifier()`:

```
// AppKit view coordinates are Y-up (0 at the bottom); the source texture's V convention
is
// Y-down (V=0 at the top row — see Blit.metal / CPURenderer.hpp), hence the flip.
const float u = (float)( local.x / size.width );
const float v = 1.0f - (float)( local.y / size.height );

_pCPUImageView->setMagnifier( true, u, v );
_pGPUImageView->setMagnifier( true, u, v );
```

`RayTracerBench/App/AppDelegate.cpp:164-170`

Calling `setMagnifier()` on *both* views with the same `(u, v)` — regardless of which one the mouse is actually over — is the whole feature in one line: it's what makes the loupe a comparison tool rather than a per-image zoom.

§7. The permanent gap, and what fills it

`NSControl.hpp`'s own header comment states the scope of the underlying problem plainly, and cites its source:

```
/*
 * Project-local addition — NOT part of Apple's metal-cpp-extensions.
 *
 * Apple ships no NS::Control/NS::Button wrapper in metal-cpp-extensions; an Apple
 * DTS engineer confirmed this is a known, permanent gap in the AppKit headers
 * (developer.apple.com/forums/thread/722886). This header fills it using the
 * exact same idiom Apple's own headers use elsewhere (Object::sendMessage over
 * objc_msgSend, selectors resolved via sel_registerName) — no Objective-C++
 * required.
 */
```

`ThirdParty/metal-cpp-extensions/AppKit/NSControl.hpp:1–12`

This isn't a bug like §4's `NS::View::init()` — it's a hole Apple has confirmed it does not intend to fill, which is why the project treats extending `metal-cpp-extensions` by hand as the standing pattern rather than a one-off workaround. `NS::Control` itself is minimal — target/action wiring plus `setEnabled()` / `isEnabled()`, the latter pair used throughout `ControlsPanel` to disable every field and button for the duration of a render (Chapter 9) — and every other AppKit widget this project touches builds on it the same way `NS::Button` does, as a `Referencing< Button, Control >`:

```
constexpr long ButtonTypeSwitch = 3; // renders as a real checkbox, not a push button

class Button : public Referencing< Button, Control >
{
        public:
                static Button*  alloc();
                Button*                 init( CGRect frame );

                void                    setTitle( const String* pTitle );

                // For ButtonTypeSwitch: state() is 1 when checked, 0 when unchecked.
AppKit toggles
                // this itself on click — no manual bookkeeping needed, unlike a plain
push button.
                void                    setButtonType( long type );
                void                    setState( long state );
                long                    state() const;
};
```

`ThirdParty/metal-cpp-extensions/AppKit/NSButton.hpp:26–45`

Three sibling headers fill in the rest of the AppKit surface this project's UI layer needed and found similarly missing, each following the exact same `Object::sendMessage` / `sel_registerName` idiom:

- **`AppKit/NSTextField.hpp`** wraps `NSTextField` for two roles at once — an editable numeric input and, via `setEditable(false)` / `setBezeled(false)` / `setDrawsBackground(false)`, a plain non-interactive label — used throughout `ControlsPanel`'s fields and `ResultsPanel`'s output lines (Chapter 9).
- **`AppKit/NSAlert.hpp`** wraps `NSAlert`'s message/informative text and `runModal()`, used by `AboutAlert` (Chapter 9) and by the scene-export success/failure dialog `AppDelegate::saveScene()` shows.
- **`AppKit/NSEvent.hpp`** and `NS::View::convertPoint()` are the pair this chapter's §6 covers directly — mouse-position tracking for the magnifier loupe.

§8. Verified before being built on top of

Every one of these additions was checked with a real, standalone compile-and-run before any of the application code above was written against it — not written once and trusted. The clearest example is the loupe feature itself, verified three separate ways rather than assumed correct end to end in one shot: an offscreen shader test rendered a known 2-pixel marker through `Blit.metal` and confirmed exact 4x magnification with no positional drift (the shader-side half of this check is Chapter 3's); a coordinate-math test built windows and views with the exact frame dimensions the production app uses and confirmed `convertPoint()` together with the Y-up-to-V-down flip in §6 map to the correct normalized position; and a full end-to-end test ran the real scene/`CPURenderer`/`Blit.metal` pipeline together and confirmed the lens actually zoomed into recognizable, correctly-positioned scene detail rather than noise or a shifted region. The `NS::View::init()` fix in §4 has an even more direct provenance: it was found by actually hitting the crash Apple's original vendored code caused, not by code review — the fix exists because something broke first.

Where this connects

- **Chapter 3** (`Shaders/Blit.metal`) owns the shader this view drives: the vertex/fragment textured-quad blit `render()` (§2) dispatches every frame, and the `MagnifierUniforms` struct and lens sampling math that `setMagnifier()` (§6) feeds via `setFragmentBytes()`.
- **Chapter 8** (`main.cpp`, `App/AppDelegate.hpp/.cpp`) is where both `ImageDisplayView` instances are actually constructed and attached to the window's content view, where the `NS_PRIVATE_IMPLEMENTATION` half of §5's macro rule lives, and where the mouse-tracking

monitor and coordinate conversion in §6 are wired up around this view's `setMagnifier()` interface.

- **Chapter 9** (`App/ControlsPanel.hpp/.cpp`, `App/ResultsPanel.hpp/.cpp`, `App/AboutAlert.hpp/.cpp`) is where the rest of §7's additions — `NS::Control`, `NS::Button`, `NS::TextField`, `NS::Alert` — do their work, making this chapter and Chapter 9 the two consumers of the entire project-local `metal-cpp-extensions` layer described here.

Chapter 11 — Tests: Parity, Geometry, and the Core Itself

Abstract. `RayTracerBenchTests` is the project's one committed test binary: a plain command-line C++ tool with no XCTest and no Objective-C, built from a homegrown ~100-line test framework rather than a vendored one. It exercises three different layers of the program — Chapter 2's shared ray-tracing core (`RayTraceCoreTests.cpp`), Chapter 6's mesh-export geometry (`EntityMeshTests.cpp`), and a genuine CPU-vs-GPU numerical comparison (`DeterministicParityTests.cpp`) — and each layer is tested for a different reason. The centerpiece of this chapter is the last of those: a tolerance that was set, measured against, found wrong, and revised, which is as close as this codebase gets to a recorded scientific method. Eighteen tests exist across the three files; all eighteen currently pass.

Files covered: `RayTracerBenchTests/TestFramework.hpp`, `RayTracerBenchTests/main.cpp`, `RayTracerBenchTests/RayTraceCoreTests.cpp`, `RayTracerBenchTests/EntityMeshTests.cpp`, `RayTracerBenchTests/DeterministicParityTests.cpp`.

---

§1. A test framework small enough to read in one sitting

Every other part of this project's UI layer works around gaps in vendored Apple frameworks by writing project-local shims (Chapters 8–10). The test layer takes the same posture toward *testing* frameworks: rather than vendor doctest or Catch2, or reach for Xcode's XCTest (which would require Objective-C, the one dependency this whole codebase is built to avoid — see `CLAUDE.md`'s "What this project is"), it writes the ~50 lines of self-registration and assertion machinery it actually needs, once, in `TestFramework.hpp`. The file's own comment states the reasoning plainly:

```
// A tiny, dependency-free, self-registering test framework — RayTracerBenchTests is a
plain
// command-line C++ tool (no XCTest, no Objective-C), and this project already avoids
external
// dependencies beyond the vendored Metal bindings, so a single header beats vendoring
doctest.
```

`RayTracerBenchTests/TestFramework.hpp:3–5`

The registration mechanism is the classic "static object with a side-effecting constructor" trick, chosen specifically so that test files never need a hand-maintained list of which tests exist. A `TestCase` is just a name and a `std::function<void()>`, stored in a `std::vector` behind a Meyer's-singleton accessor:

```
// Meyer's-singleton accessor rather than a plain global, so registration order across
// translation units (each with its own static TestRegistrar instances) is well-defined.
inline std::vector<TestCase>& registry()
{
        static std::vector<TestCase> s_registry;
        return s_registry;
}

struct TestRegistrar
{
        // Constructing a static TestRegistrar (see the TEST_CASE macro) registers its
test function
        // as a side effect, before main() runs.
        TestRegistrar( const char* name, std::function<void()> fn ) {
registry().push_back( { name, std::move( fn ) } ); }
};
```

`RayTracerBenchTests/TestFramework.hpp:22–35`

`TEST_CASE(name)` is the macro that ties a test function's *definition* to its *registration* in one place, so writing a new test is just writing a function body — nothing else has to be touched:

```
#define TEST_CASE( name )
\
        static void          name();
\
        static ::TestFramework::TestRegistrar registrar_##name( #name, name );
\
        static void          name()
```

`RayTracerBenchTests/TestFramework.hpp:74–77`

This expands `TEST_CASE( hitSphere_hitsDeadOn ) { ... }` into a forward declaration, a file-scope `static TestRegistrar` whose constructor runs before `main()` and pushes `{"hitSphere_hitsDeadOn", hitSphere_hitsDeadOn}` into the registry, and then the function definition itself. Every `TEST_CASE` in every `.cpp` file linked into the binary adds itself to the same global list purely by existing — there is no `AllTests.cpp` enumerating them.

`main.cpp` is consequently almost content-free — the entire executable's entry point is "run whatever got registered":

```
#include "TestFramework.hpp"

// Entry point: runs every TEST_CASE registered across this executable's translation
units.
int main()
{
        return TestFramework::runAll();
}
```

`RayTracerBenchTests/main.cpp:1–7`

`runAll()` iterates the registry, calls each test function inside a `try` / `catch`, and prints one `[PASS]` / `[FAIL]` line per test plus a summary:

```
inline int runAll()
{
        int passed = 0;
        int failed = 0;
        for ( const TestCase& test : registry() )
        {
                try
                {
                        test.fn();
                        std::printf( "[PASS] %s\n", test.name.c_str() );
                        ++passed;
                }
                catch ( const CheckFailure& failure )
                {
                        std::printf( "[FAIL] %s: %s\n", test.name.c_str(),
failure.message.c_str() );
                        ++failed;
                }
                catch ( const std::exception& ex )
                {
                        std::printf( "[FAIL] %s: unexpected exception: %s\n",
test.name.c_str(), ex.what() );
                        ++failed;
                }
        }
        std::printf( "%d passed, %d failed, %d total\n", passed, failed, passed + failed
);
        return failed == 0 ? 0 : 1;
}
```

`RayTracerBenchTests/TestFramework.hpp:44–69`

A failed test doesn't abort the run — it throws a `CheckFailure`, which `runAll()` catches and logs as a `[FAIL]` line before moving on to the next registered test, so one broken assertion never hides the results of the tests after it. The process's exit code (0 or 1) is what

a CI system or a human running the binary directly checks; the printed `[PASS]` / `[FAIL]` lines are for a human reading the output.

The two assertion macros both build on the same "throw a `CheckFailure` carrying a `file:line`-prefixed message" primitive:

```
#define RT_CHECK_MESSAGE( cond, extra )
\
        do
\
        {
\
                if ( !( cond ) )
\
                {
\
                        std::ostringstream oss;
\
                        oss << __FILE__ << ":" << __LINE__ << ": CHECK failed: " #cond <<
" " << extra;            \
                        throw ::TestFramework::CheckFailure{ oss.str() };
\
                }
\
        } while ( false )

#define CHECK( cond ) RT_CHECK_MESSAGE( cond, "" )

#define CHECK_NEAR( a, b, eps )
\
        do
\
        {
\
                double rt_a = ( a );
\
                double rt_b = ( b );
\
                if ( std::fabs( rt_a - rt_b ) > ( eps ) )
\
                {
\
                        std::ostringstream oss;
\
                        oss << __FILE__ << ":" << __LINE__ << ": CHECK_NEAR failed: " #a "
(" << rt_a << ") vs "    \
                                << #b " (" << rt_b << "), eps=" << ( eps );
\
                        throw ::TestFramework::CheckFailure{ oss.str() };
\
                }
\
        } while ( false )
```

`RayTracerBenchTests/TestFramework.hpp:81–109`

`CHECK` is for exact boolean conditions (`result.hit`, `!result.hit`, `a1 == a2`); `CHECK_NEAR` exists because almost every geometric assertion in this suite is a floating-point comparison,

where exact equality is the wrong question to ask.

Why `xcodebuild test -scheme RayTracerBenchTests` does not work

Because `RayTracerBenchTests` is a plain `add_executable()` target with a `main()` that calls `runAll()` directly — not an XCTest bundle — the Xcode scheme that CMake's Xcode generator produces for it has no Test action configured at all. There is nothing in the scheme for `xcodebuild test` to invoke; the target has a Run action (it's an ordinary executable) but no XCTest target to attach a Test action to. `CLAUDE.md`'s build-and-test section documents this precisely, because it's exactly the kind of thing that looks like it should work and silently doesn't — worth quoting exactly rather than paraphrased, since it's operational instruction, not narrative:

> - Build: Xcode GUI (`Product > Run`), or `xcodebuild -project RayTracerBench.xcodeproj -scheme RayTracerBench -configuration Debug build`
> - Test: `xcodebuild -project RayTracerBench.xcodeproj -scheme RayTracerBenchTests -configuration Debug build`, then run the built binary directly (find it via `xcodebuild -showBuildSettings -scheme RayTracerBenchTests | grep TARGET_BUILD_DIR`, or just run it from Xcode's GUI). **`xcodebuild test -scheme RayTracerBenchTests` does not work** — confirmed by trying it, not assumed — since `RayTracerBenchTests` is a plain command-line C++ tool (no XCTest/ObjC), so its CMake-generated scheme has no Test action configured; `xcodebuild test` fails with "Scheme ... is not currently configured for the test action."

(`CLAUDE.md`, "Build and test commands")

The note that this was "confirmed by trying it, not assumed" matters as much as the fact itself: it's the same discipline this whole project applies everywhere else (the AppKit/metal-cpp gaps in Chapter 10, the `RayTraceCore.h`-cannot-be-compiled-from-a-string finding in Chapter 3) — don't write down a claim about tooling behavior without having actually observed the failure message. The correct sequence is therefore: build the `RayTracerBenchTests` target with a plain `xcodebuild ... build` (not `test`), locate the resulting binary's `TARGET_BUILD_DIR`, and execute it like any other command-line tool. Its `main()` (§1 above) does the rest — `runAll()`'s process exit code is what a script or CI step should check.

---

§2. `RayTraceCoreTests.cpp`: exercising Chapter 2 directly

`RayTraceCore.h` is dual-compiled — the exact same header is `#include`d verbatim by `Shaders/Raytracer.metal` (MSL) and `CPU/CPURenderer.cpp` (plain C++), which is the entire reason the CPU/GPU comparison in this app is apples-to-apples (Chapter 2). `RayTraceCoreTests.cpp` links against the *plain C++ compile* of that header — it never touches Metal — so it's really testing the one shared source file that both renderers execute, from the host side where it's cheap and fast to test.

The file opens with small helpers that build a `TransformGPU` / `ShapeGPU` pair for a sphere or an axis-aligned pyramid, since almost every test needs one:

```
// An identity-orientation Transform at `center` — sufficient for a sphere, whose
Transform
// component never uses right/up/forward.
TransformGPU sphereTransform( simd_float3 center )
{
        TransformGPU t;
        t.position = center;
        t.right = makeFloat3( 1.0f, 0.0f, 0.0f );
        t.up = makeFloat3( 0.0f, 1.0f, 0.0f );
        t.forward = makeFloat3( 0.0f, 0.0f, 1.0f );
        return t;
}
```

`RayTracerBenchTests/RayTraceCoreTests.cpp:6–16`

Thirteen `TEST_CASE`s follow, walking outward from `hitSphere()` through `hitPyramid()` to `hitEntity()`'s dispatch, then a couple of small pure-math utilities. Two representative cases:

`hitSphere_originInsideSphere_reportsBackFace` checks a specific correctness property of `RayTraceCore.h`'s `hitSphere()` — that the `frontFace` flag flips correctly when the ray origin is *inside* the sphere, since this is exactly the situation `scatter()`'s dielectric branch depends on to decide whether a ray is entering or exiting glass:

```
TEST_CASE( hitSphere_originInsideSphere_reportsBackFace )
{
        // Ray origin is inside the sphere: the nearest positive root is the far side of
the
        // sphere, and the outward normal should be flipped (frontFace == false) since it
points the
        // same direction as the ray.
        TransformGPU transform = sphereTransform( makeFloat3( 0.0f, 0.0f, 0.0f ) );
        ShapeGPU      shape = sphereShape( 1.0f );
        Ray           ray{ makeFloat3( 0.0f, 0.0f, 0.0f ), makeFloat3( 0.0f, 0.0f, -1.0f )
};

        HitResult result = hitSphere( transform, shape, ray, 0.001f, 1000.0f );

        CHECK( result.hit );
        CHECK_NEAR( result.record.t, 1.0, 1e-5 );
        CHECK( !result.record.frontFace );
}
```

`RayTracerBenchTests/RayTraceCoreTests.cpp:92-106`

`hitPyramid_hitsSideFaceWhenOffApexAxis` is a good example of the suite testing not just "does it hit" but "does it hit at the geometrically predicted `t` and `normal`" — here by working out, in the comment, exactly what the +X face's plane equation from `RayTraceCore.h` predicts for a ray fired straight down at `x = 0.9`, and then asserting the intersection lands precisely there:

```
TEST_CASE( hitPyramid_hitsSideFaceWhenOffApexAxis )
{
        // Same pyramid; a ray straight down at x=0.9 (off the apex axis) enters through
the sloped +X
        // side face rather than the apex — the "roof" at that (x,z) is lower than at the
apex, exactly
        // as the +X face's plane equation (RayTraceCore.h) predicts: y <= height -
(height/baseHalfWidth)*x
        // = 1 - 0.9 = 0.1 here, so the ray (descending from y=5) enters at y=0.1, i.e.
t=4.9.
        TransformGPU transform = pyramidTransform( makeFloat3( 0.0f, 0.0f, 0.0f ) );
        ShapeGPU      shape = pyramidShape( /*baseHalfWidth=*/1.0f, /*height=*/1.0f );
        Ray           ray{ makeFloat3( 0.9f, 5.0f, 0.0f ), makeFloat3( 0.0f, -1.0f, 0.0f )
};

        HitResult result = hitPyramid( transform, shape, ray, 0.001f, 1000.0f );

        CHECK( result.hit );
        CHECK_NEAR( result.record.t, 4.9, 1e-4 );
        CHECK( result.record.normal.x > 0.0f ); // +X side face, outward normal tilts
toward +X
        CHECK( result.record.normal.y > 0.0f );
}
```

`RayTracerBenchTests/RayTraceCoreTests.cpp:139–155`

`hitEntity_dispatchesByShapeTag` (`RayTracerBenchTests/RayTraceCoreTests.cpp:200–217`) is the test that specifically targets `hitEntity()` itself rather than either primitive's intersection routine — its comment states the point directly: "confirm it reaches `hitSphere()` and `hitPyramid()` for the matching tags rather than, say, always taking the sphere path," which is exactly the failure mode a tagged switch (as opposed to a virtual call) can silently fall into if a `case` is missing or mis-ordered.

The remaining two tests step outside intersection geometry entirely to cover small, pure utility functions the whole core depends on: `reflect3_mirrorsAboutNormal` (`RayTracerBenchTests/RayTraceCoreTests.cpp:220–230`) checks that reflecting a 45° incoming vector off a flat normal produces the mirrored vector `scatter()`'s metal branch needs, and `pcgHash_isDeterministicAndVaries` (`RayTracerBenchTests/RayTraceCoreTests.cpp:234–242`) checks the one property the per-thread RNG absolutely must have — same input, same output, every time — since that determinism is the entire premise §4's parity test relies on.

---

§3. `EntityMeshTests.cpp`: catching a winding-order bug that actually happened

Chapter 6's `buildEntityMesh()` is a third ECS "system" over the same Transform/Shape components `hitEntity()` dispatches on, but instead of testing a ray it emits a world-space triangle mesh for glTF/OBJ export. A triangle mesh has a property that a ray-intersection routine doesn't need to worry about explicitly: *winding order* — the order a triangle's three vertices are listed in determines, via the right-hand rule on the cross-product of its edges, which way its face normal points. Get the winding backwards and the geometry is still the right *shape*, but every face normal points into the solid instead of out of it — a bug that is very easy to miss by inspection (assimp/trimesh will happily load the mesh, and it silhouette-renders identically from most raytracers) and would only show up as inverted lighting in some downstream renderer.

`EntityMeshTests.cpp`'s central helper, `allTrianglesFaceAwayFrom()`, is exactly this project's answer to that risk: for every triangle in a `MeshData`, take its edge cross product as the face normal, and check that the vector from a known interior point to the triangle's centroid points in the *same* half-space as that normal (a positive dot product):

```
// True if every triangle in `mesh` winds so its face normal (via cross product) points
away
// from `interiorPoint` — the same check used to hand-derive EntityMesh.cpp's winding
orders,
// re-run here so a future edit that breaks winding fails a test instead of only looking
wrong
// in a 3D viewer.
bool allTrianglesFaceAwayFrom( const MeshData& mesh, simd_float3 interiorPoint )
{
        for ( size_t i = 0; i + 2 < mesh.indices.size(); i += 3 )
        {
                simd_float3 a = mesh.positions[ mesh.indices[ i ] ];
                simd_float3 b = mesh.positions[ mesh.indices[ i + 1 ] ];
                simd_float3 c = mesh.positions[ mesh.indices[ i + 2 ] ];
                simd_float3 faceNormal = simd_cross( b - a, c - a );
                simd_float3 centroid = ( a + b + c ) / 3.0f;
                if ( dot3( faceNormal, centroid - interiorPoint ) <= 0.0f )
                        return false;
        }
        return true;
}
```

`RayTracerBenchTests/EntityMeshTests.cpp:23-40`

The comment is explicit that this isn't a check added defensively after the fact for good hygiene — it's "the same check used to hand-derive `EntityMesh.cpp`'s winding orders." `CLAUDE.md`'s project-status notes name the actual outcome of running it: "an initial sphere-winding attempt was in fact backwards and caught this way" — meaning the very first version of the sphere tessellation in `buildEntityMesh()` had its indices listed in the wrong order, and this test (or the standalone harness it was promoted from) is what surfaced it, rather than a visual inspection in a 3D viewer.

`buildEntityMesh_sphere_hasNoDegenerateTriangles_andFacesOutward` applies that check to a tessellated sphere, and additionally confirms every emitted vertex actually lies on the sphere's surface (a check that would catch a tessellation-math bug independent of winding):

```
TEST_CASE( buildEntityMesh_sphere_hasNoDegenerateTriangles_andFacesOutward )
{
        simd_float3 center = simd_make_float3( 1.0f, 2.0f, 3.0f );
        float       radius = 5.0f;

        TransformGPU transform = identityTransformAt( center );
        ShapeGPU     shape;
        shape.type = SHAPE_SPHERE;
        shape.radius = radius;
        shape.baseHalfWidth = 0.0f;
        shape.height = 0.0f;
        shape.materialIndex = 0;

        MeshData mesh = buildEntityMesh( transform, shape );

        CHECK( !mesh.positions.empty() );
        CHECK( mesh.indices.size() % 3 == 0 );
        CHECK( allTrianglesFaceAwayFrom( mesh, center ) );

        // Every position should lie on the sphere's surface (within float tolerance).
        for ( const simd_float3& p : mesh.positions )
        {
                simd_float3 offset = p - center;
                CHECK_NEAR( std::sqrt( dot3( offset, offset ) ), radius, 1e-3 );
        }
}
```

`RayTracerBenchTests/EntityMeshTests.cpp:45-70`

`buildEntityMesh_pyramid_hasExactlySixFacetedTriangles_andFacesOutward` (`RayTracerBenchTests/EntityMeshTests.cpp:74-93`) does the same winding check for the pyramid's exact 5-plane geometry, plus a structural assertion specific to how Chapter 6 builds pyramid meshes — 6 triangles × 3 unshared vertices each (`mesh.indices.size() == 18`, `mesh.positions.size() == 18`), confirming the faceted, flat-normal construction (as opposed to a shared-vertex, smooth-normal one, which is what the sphere tessellation uses instead) actually emits unique vertices per triangle rather than accidentally reusing indices.

The third test, `buildEntityMesh_pyramid_respectsOrientationBasis` (`RayTracerBenchTests/EntityMeshTests.cpp:95-122`), checks a different axis of correctness entirely: that a pyramid's `TransformGPU` orientation basis (the same `right`/`up`/`forward` frame `hitPyramid()` uses to transform rays into local space) is honored by the mesh builder too, by rotating a pyramid 90° so its local "up" points along world +X and confirming the apex vertex lands at world `(1, 0, 0)` rather than the untransformed `(0, 1, 0)`.

---

§4. `DeterministicParityTests.cpp`: the revised-tolerance story

This is the one test file in the suite that actually runs both renderers and is only meaningful *because* `RayTraceCore.h` is shared verbatim between them (Chapter 2) — a CPU/GPU pixel diff on two independently-written ray tracers would tell you nothing except that two different programs produce different output. Here, a mismatch is potentially informative precisely because both sides execute the same intersection/scatter/`rayColor()` logic, so any divergence has to come from somewhere identifiable: RNG *stream* differences (both sides use the same `pcgHash`/`randomFloat` function, but seeded per-thread differently — see Chapter 2 and 4/5), or sub-ULP floating-point differences between CPU and GPU transcendental functions (`sqrt`, `pow`), and nothing else.

`measureParity()` builds one fixed-seed `SceneDescription` via `buildDefaultScene()`, renders it with `renderCPU()`, renders the identical scene with a real `GPURenderer` (headless — `MTL::CreateSystemDefaultDevice()` with no window, matching the "plan doc's `DeterministicParityTests` intent" noted in its own comment), reads the GPU texture back with `getBytes()`, and diffs every RGB channel (skipping alpha) between the two:

```
ParityStats measureParity( uint32_t width, uint32_t spp, uint32_t maxDepth, unsigned seed,
int tolerance )
{
        const float      aspectRatio = 16.0f / 9.0f;
        SceneDescription scene = buildDefaultScene( seed, width, aspectRatio, spp,
maxDepth );

        CPURenderResult cpuResult = renderCPU( scene, CPUThreading::MultiThreaded );

        MTL::Device* pDevice = MTL::CreateSystemDefaultDevice();
        GPURenderer  gpuRenderer( pDevice );
        GPURenderResult gpuResult = gpuRenderer.render( scene );

        const uint32_t       w = scene.params.width;
        const uint32_t       h = scene.params.height;
        std::vector<uint8_t> gpuPixels( (size_t)w * h * 4 );
        gpuResult.pTexture->getBytes( gpuPixels.data(), (size_t)w * 4, MTL::Region( 0, 0,
0, w, h, 1 ), 0 );

        size_t mismatches = 0;
        int    maxDiff = 0;
        const size_t totalChannels = (size_t)w * h * 3; // RGB only, skip alpha

        for ( uint32_t y = 0; y < h; ++y )
        {
                for ( uint32_t x = 0; x < w; ++x )
                {
                        size_t idx = ( (size_t)y * w + x ) * 4;
                        for ( int c = 0; c < 3; ++c )
                        {
                                int diff = std::abs( (int)cpuResult.pixels[ idx + c ] -
(int)gpuPixels[ idx + c ] );
                                maxDiff = std::max( maxDiff, diff );
                                if ( diff > tolerance )
                                        ++mismatches;
                        }
                }
        }

        pDevice->release();

        return ParityStats{ (double)mismatches / (double)totalChannels, maxDiff };
}
```

`RayTracerBenchTests/DeterministicParityTests.cpp:28–66`

What was assumed, what was measured, and what changed

`CLAUDE.md`'s verification-approach notes record the actual history behind this function's two parameters — `tolerance` and the spp at which the resulting assertion is set — and it is

worth reconstructing precisely, because the file's current test only encodes the *end* of the story, not the middle.

The original plan called for a strict per-pixel bound: every corresponding CPU/GPU pixel should agree to within roughly 2/255. That was the "originally planned" tolerance CLAUDE.md refers to, and it was optimistic. Measuring it for real — at a low sample count, spp=16, on the 64×36 preview size this suite still uses today — found that per-channel mismatches over that tolerance-of-2 affected **~8.8% of all channels**, with a **max diff of 61/255**: a large, occasional disagreement, not universal noise. Critically, this was *not* treated as evidence of a bug, because of where the mismatches did and didn't occur: diffs were exactly zero on every pure-sky pixel — the case with no scatter/branching at all, where CPU and GPU execute an identical, branch-free code path — and non-zero only where rays actually hit geometry and had to make a scatter or reflectance decision. Raising spp from 16 to 200 then *cut* both figures — mismatch rate 8.8% → 4.9%, max diff 61 → 13 — which is the signature of a chaotic Monte Carlo system, not a systematic bug: a real bug (a sign error, a wrong constant, a mixed-up axis) produces an error that doesn't shrink as more samples get averaged together, whereas a sub-ULP floating-point difference between CPU's and GPU's `sqrt`/`pow` implementations flipping an occasional scatter/reflectance branch produces exactly what was observed — two ray paths that fully diverge from the flip point onward (hence the large max diff on the pixels that are affected at all), diluted more and more as spp rises and each pixel averages over more independent samples.

The engineering rule this produced, stated explicitly in CLAUDE.md, is: **assert on mismatch rate at high spp, not a strict per-pixel bound at low spp.** The 8.8%-at-spp=16 measurement itself is *not* what's encoded in the committed test — it lives only in CLAUDE.md's record of the manual investigation that established the rule. What the committed test actually asserts is the high-spp side of that same investigation:

```
// See CLAUDE.md's verification notes: a strict ~2/255 per-pixel bound does NOT hold at
low sample
// counts (chaotic Monte Carlo branch-divergence from sub-ULP CPU/GPU float differences
flips
// occasional scatter/reflectance decisions, causing large but *isolated* per-pixel
differences —
// not a bug, confirmed by mismatches being exactly zero on every pure-sky pixel and
shrinking as
// spp rises). So this asserts on mismatch RATE at a reasonably high spp, where that
chaotic
// divergence has mostly averaged out, rather than a strict per-pixel bound at low spp.
TEST_CASE( cpuGpuParity_fixedSeed_highSampleCount )
{
        ParityStats stats = measureParity( /*width=*/64, /*spp=*/200, /*maxDepth=*/16,
/*seed=*/777u, /*tolerance=*/2 );

        std::printf( "  parity: %.3f%% channels mismatched (>2/255), max diff %d\n",
stats.mismatchFraction * 100.0, stats.maxDiff );

        CHECK( stats.mismatchFraction < 0.06 ); // observed ~4.9% at spp=200 in prior
manual runs
}
```

`RayTracerBenchTests/DeterministicParityTests.cpp:69-82`

Two things about this assertion are worth noticing precisely, since they're the load-bearing detail of the whole chapter. First, the per-channel tolerance itself is *still* 2/255 — that part of the original plan wasn't wrong, it was just never going to be satisfied on 100% of channels, so what changed is not the per-pixel threshold but the *statistic asserted over it* (a rate, not a universal bound) and the *spp at which that statistic is checked* (200, not 16). Second, the assertion's own bound, `< 0.06`, is deliberately looser than the ~4.9% actually observed at spp=200 — the comment names the observed figure directly as the basis for the threshold, leaving headroom above it rather than pinning the assertion to the exact measured value, which would make the test brittle to the ordinary run-to-run variance of a Monte Carlo renderer.

The second test in this file is a narrower, cheaper sanity check that the parity test above implicitly depends on: that `buildDefaultScene()` plus `renderCPU()` are actually deterministic for a fixed seed, so that any CPU/GPU divergence measured above can be attributed to CPU-vs-GPU execution rather than to scene-construction or RNG non-determinism on the CPU side alone.

```
TEST_CASE( cpuGpuParity_isDeterministic_sameSeedSameResult )
{
        // Rendering the same seed twice (CPU side only, cheap) must produce byte-
identical output —
        // this is a sanity check on the scene/RNG determinism the parity test above
depends on, not a
        // CPU/GPU comparison.
        const float      aspectRatio = 16.0f / 9.0f;
        SceneDescription sceneA = buildDefaultScene( 42u, 48, aspectRatio, 8, 8 );
        SceneDescription sceneB = buildDefaultScene( 42u, 48, aspectRatio, 8, 8 );

        CPURenderResult resultA = renderCPU( sceneA, CPUThreading::SingleThreaded );
        CPURenderResult resultB = renderCPU( sceneB, CPUThreading::SingleThreaded );

        CHECK( resultA.pixels == resultB.pixels );
}
```

`RayTracerBenchTests/DeterministicParityTests.cpp:84-97`

It deliberately renders `SingleThreaded` — with `CPUThreading::MultiThreaded` (the CPU renderer's default; see Chapter 4), row-partitioned worker threads could in principle finish and touch shared state in a different order between two runs, which would make this test about thread-scheduling noise rather than about the RNG/scene determinism it actually intends to check.

---

§5. Eighteen tests, all passing

Counting `TEST_CASE` occurrences across the three test files: thirteen in `RayTraceCoreTests.cpp` (`hitSphere_hitsDeadOn`, `hitSphere_missesEntirely`, `hitSphere_respectsTMaxRange`, `hitSphere_originInsideSphere_reportsBackFace`, `hitSphere_picksCloserOfTwoRoots`, `hitPyramid_hitsApexFromAbove`, `hitPyramid_hitsSideFaceWhenOffApexAxis`, `hitPyramid_hitsBaseFromBelow`, `hitPyramid_missesEntirely`, `hitPyramid_sideNormalPointsOutwardAndUpward`, `hitEntity_dispatchesByShapeTag`, `reflect3_mirrorsAboutNormal`, `pcgHash_isDeterministicAndVaries`), three in `EntityMeshTests.cpp`, and two in `DeterministicParityTests.cpp` — 13 + 3 + 2 = **18**, matching the count `CLAUDE.md`'s project-status notes give ("18 tests, all passing"), so the number hasn't drifted since that note was written. `CMakeLists.txt` links exactly these three test `.cpp`s plus `main.cpp` into the `RayTracerBenchTests` executable target (`CMakeLists.txt:113-117`), so this count is also the complete registry `runAll()` (§1) will walk at runtime — nothing else contributes a `TEST_CASE` to this binary.

Where this connects

- **Chapter 2** (`Core/RayTraceCore.h`) is the subject of nearly all of §2's tests directly, and is the entire reason §4's CPU/GPU diff can be interpreted at all — without the dual-compiled shared core, a pixel mismatch would be uninformative noise between two unrelated programs rather than a signal about RNG streams and floating-point precision.
- **Chapter 6** (`Export/EntityMesh.hpp/.cpp`) is what §3 tests; the winding-order check there is a direct, reusable form of the same hand-verification technique used to derive `buildEntityMesh()`'s geometry in the first place, and is the reason a genuine sphere-winding bug was caught before shipping rather than after.
- **Chapters 4 and 5** (`CPU/CPURenderer.hpp/.cpp`, `GPU/GPURenderer.hpp/.cpp`) are the two renderers §4's `measureParity()` actually invokes and diffs — this chapter's parity story is really about the divergence between those two chapters' execution of the one shared core from Chapter 2.
- **Chapter 12** (`CMakeLists.txt`) is the build system that compiles this test binary and is the source of the "no Test action configured" scheme limitation discussed in §1 — and its own text should be read alongside the exact `xcodebuild ... build` + run-the-binary-directly sequence quoted from `CLAUDE.md` above, since that sequence is the only way this chapter's tests actually get executed.

Chapter 12: The Build System

Abstract. RayTracerBench is not built by hand-editing an Xcode project — it is built by CMake, with the `.xcodeproj` itself treated as a generated artifact that happens to be checked in for convenience. `CMakeLists.txt` is short (136 lines) but carries real weight: it defines the three-target layout (a framework-free static library, the app, and a headless test binary), and it contains the one genuinely unusual build step in the whole project — shelling out to the real Metal command-line compiler at build time, because `Shaders/Raytracer.metal`'s verbatim `#include`s of the shared core headers make it impossible to compile the way `Blit.metal` is compiled (from an in-memory string at runtime). This chapter walks the file top to bottom: the target layout, the `.metallib` custom command and the `RT_RAYTRACER_METALLIB` / `RT_SHADERS_DIR` compile definitions it and its sibling produce, the vendored `ThirdParty` include paths, the operational rule for regenerating the Xcode project after a source change, and the build/test commands this project actually uses (including the one that looks like it should work and doesn't).

Files covered: `CMakeLists.txt`.

§1. Three targets: a framework-free core library, the app, and a headless test binary

The whole file builds exactly three CMake targets, and the split between them is not incidental — it is the same "framework-free vs. needs AppKit/Metal" line that runs through the rest of the project's module layout. The first is `RayTracerCore`, a static library:

```
# CMakeLists.txt:20-29
# RayTraceCore.h and ShaderTypes.h are header-only and dual-compiled into
# Shaders/Raytracer.metal directly (not through this library) once the GPU
# renderer exists. CPURenderer.cpp is framework-free C++17 like Core/, so it
# lives in the same static library rather than a separate CMake target.
add_library(RayTracerCore STATIC
    RayTracerBench/Core/Scene.cpp
    RayTracerBench/CPU/CPURenderer.cpp
    RayTracerBench/Export/EntityMesh.cpp
    RayTracerBench/Export/SceneExporter.cpp
)

target_include_directories(RayTracerCore PUBLIC
    RayTracerBench/Core
    RayTracerBench/CPU
    RayTracerBench/Export
)
```

Every `.cpp` in this list is plain C++17 with no Apple-framework dependency: `Scene.cpp` (Chapter 1) only touches `<simd/simd.h>` types and the standard library's `std::mt19937`; `CPURenderer.cpp` (Chapter 4) is explicitly framework-free by design; and `EntityMesh.cpp` / `SceneExporter.cpp` (Chapters 6 and 7) build meshes and write glTF/OBJ text with nothing but the standard library. `RayTraceCore.h` and `ShaderTypes.h` themselves never appear in this target's source list at all — as the comment says, they are header-only and get pulled into two separate compilations instead: this library's `.cpp` files include them as plain C++, and `Shaders/Raytracer.metal` (§2 below) includes the identical files again as MSL. `RayTracerCore` is a library of *consumers* of that shared header pair, not a place where it is compiled once and reused.

Note what is conspicuously absent from this list: `Export/ImageWriter.cpp`. It builds meshes and writes 3D-format text like its siblings above, but it also calls into CoreGraphics/ImageIO to rasterize a PNG (Chapter 7), and `RayTracerCore` is deliberately kept framework-free so it never needs to link against anything beyond the C++ standard library. `ImageWriter.cpp` is compiled into the app target instead:

```
# CMakeLists.txt:37-46
add_executable(RayTracerBench
    RayTracerBench/main.cpp
    RayTracerBench/App/AppDelegate.cpp
    RayTracerBench/App/AboutAlert.cpp
    RayTracerBench/App/ControlsPanel.cpp
    RayTracerBench/App/ResultsPanel.cpp
    RayTracerBench/App/ImageDisplayView.cpp
    RayTracerBench/GPU/GPURenderer.cpp
    RayTracerBench/Export/ImageWriter.cpp
)
```

`RayTracerBench` is where everything that actually needs a framework lives: all of `App/` (AppKit, via `metal-cpp-extensions`), `GPU/GPURenderer.cpp` (Metal), and `Export/ImageWriter.cpp` (CoreGraphics/ImageIO). It links `RayTracerCore` in later (§5) to get the shared scene/CPU/export code, so the app target is really "the framework-dependent half of the program, plus the framework-free half linked in" rather than a self-contained list of every source file the app needs.

The third target, `RayTracerBenchTests` (Chapter 11), reuses the same split again — it links `RayTracerCore` for the scene/CPU code its tests exercise, and separately compiles `GPU/GPURenderer.cpp` directly into itself (rather than linking against `RayTracerBench`, which doesn't exist as a library) so its parity tests can drive the real GPU renderer too:

```
# CMakeLists.txt:113-119
add_executable(RayTracerBenchTests
    RayTracerBenchTests/main.cpp
    RayTracerBenchTests/RayTraceCoreTests.cpp
    RayTracerBenchTests/DeterministicParityTests.cpp
    RayTracerBenchTests/EntityMeshTests.cpp
    RayTracerBench/GPU/GPURenderer.cpp
)
```

Its own comment on this target states the same reasoning explicitly:

```
# CMakeLists.txt:111-112
# RayTracerBenchTests: a plain command-line C++ tool (no XCTest, no Objective-C) —
headless, so it
# only needs Metal + Foundation, not AppKit/metal-cpp-extensions/Cocoa.
```

§2. The `.metallib` custom command: compiling Raytracer.metal to a real file on disk

`Shaders/Raytracer.metal` cannot be handed to Metal as an in-memory source string the way `Shaders/Blit.metal` is (Chapter 3). Blit.metal is self-contained — a small textured-quad vertex/fragment shader with no local `#include`s — so `ImageDisplayView` can read its text off disk at runtime and hand the raw string to `newLibrary(sourceString, ...)`. Raytracer.metal is different: it `#include`s `Core/ShaderTypes.h` and `Core/RayTraceCore.h` verbatim, using ordinary relative-path `#include` directives, and those directives have nothing to resolve against when the "file" being compiled is just a string sitting in memory with no location of its own. The build system's answer is to make Raytracer.metal a real, separately compiled artifact, using the actual command-line Metal toolchain:

```
# CMakeLists.txt:61-83
# Raytracer.metal #includes Core/ShaderTypes.h and Core/RayTraceCore.h verbatim (the dual-
compile
# design), so it CANNOT be compiled from a raw in-memory source string the way Blit.metal
is -
# there's no file location for a relative #include to resolve against. Compile it to a
real
# .metallib at build time instead (still command-line xcrun, not an Xcode "Metal Compiler"
build
# phase, to stay consistent with this project being built headlessly), and load that
compiled
# library at runtime via newLibrary(filepath, &error) rather than newLibrary(sourceString,
...).
set(RAYTRACER_METAL_SOURCE ${CMAKE_SOURCE_DIR}/RayTracerBench/Shaders/Raytracer.metal)
set(RAYTRACER_METAL_AIR ${CMAKE_BINARY_DIR}/Raytracer.air)
set(RAYTRACER_METALLIB ${CMAKE_BINARY_DIR}/Raytracer.metallib)

add_custom_command(
    OUTPUT ${RAYTRACER_METALLIB}
    COMMAND xcrun -sdk macosx metal -c ${RAYTRACER_METAL_SOURCE} -o ${RAYTRACER_METAL_AIR}
    COMMAND xcrun -sdk macosx metallib ${RAYTRACER_METAL_AIR} -o ${RAYTRACER_METALLIB}
    DEPENDS
        ${RAYTRACER_METAL_SOURCE}
        ${CMAKE_SOURCE_DIR}/RayTracerBench/Core/ShaderTypes.h
        ${CMAKE_SOURCE_DIR}/RayTracerBench/Core/RayTraceCore.h
    COMMENT "Compiling Raytracer.metal -> Raytracer.metallib"
    VERBATIM
)
add_custom_target(RaytracerMetalLib DEPENDS ${RAYTRACER_METALLIB})
add_dependencies(RayTracerBench RaytracerMetalLib)
```

This is a two-stage pipeline, mirroring exactly what Xcode's own "Metal Compiler" build phase would do under the hood, but invoked as two plain `xcrun` command lines instead:

`xcrun -sdk macosx metal -c ... -o Raytracer.air` compiles the `.metal` source (with its `#include`s now resolving normally, since it's a real file on disk in its real directory) down to Apple's intermediate representation (`.air`), and `xcrun -sdk macosx metallib ...` links that `.air` file into the final `.metallib` container `MTL::Device::newLibrary(filepath, &error)` can load. The `DEPENDS` list is what makes this a correct incremental build rather than a one-shot script: naming `Raytracer.metal` *and* the two headers it `#include`s means CMake/Ninja/Xcode's build system will re-run this custom command whenever any of the three changes, not just when the `.metal` file itself is touched — exactly the dependency an `#include` relationship requires, expressed the CMake way instead of relying on a compiler-generated dependency file. `add_custom_target(RaytracerMetalLib ...)` gives the custom command's single output file a named target that `RayTracerBench` can depend on via `add_dependencies`, which is the standard CMake idiom for making an executable target wait on a custom command that doesn't produce one of its own directly-linked object files.

The comment's parenthetical — "still command-line `xcrun`, not an Xcode 'Metal Compiler' build phase" — is a deliberate consistency choice, not an Xcode limitation: Xcode *does* have a built-in Metal Compiler phase that could do this same job when the `.xcodeproj` is opened in the GUI, but using it would mean the `.metallib` only gets built correctly when Xcode generates that phase itself, which reintroduces exactly the kind of GUI-project-template dependency §4 explains this project avoids. Shelling out to `xcrun` from a `CMakeLists.txt` custom command works identically whether the project is built from Xcode, from `xcodebuild` on the command line, or (on another platform's generator) from Ninja or Make — it is the one build step that has to reach outside CMake's own compiler abstractions, and it does so in the same headless-friendly way as everything else here.

§3. `RT_RAYTRACER_METALLIB` and `RT_SHADERS_DIR`: two different answers to "where do I find my shader"

The `.metallib` custom command produces a file at a path CMake controls (`${CMAKE_BINARY_DIR}/Raytracer.metallib`), and `GPURenderer` needs to know that exact path at runtime to call `newLibrary(filepath, &error)` on it (Chapter 5). Rather than hard-coding a path or reading an environment variable, the build injects it directly as a preprocessor definition on the target that needs it:

```
# CMakeLists.txt:85-87
target_compile_definitions(RayTracerBench PRIVATE
    RT_RAYTRACER_METALLIB="${RAYTRACER_METALLIB}"
)
```

and the same definition is repeated for the test target, since `RayTracerBenchTests` also constructs a real `GPURenderer` to drive its parity tests:

```
# CMakeLists.txt:126-128
target_compile_definitions(RayTracerBenchTests PRIVATE
    RT_RAYTRACER_METALLIB="${RAYTRACER_METALLIB}"
)
```

`RT_SHADERS_DIR` solves a related but distinct problem, for the *other* shader. `Blit.metal` (Chapter 3, and used by `ImageDisplayView`, Chapter 10) is compiled from an in-memory source string, which means something in the app still has to read that string off disk first — and since `RayTracerBench` is deliberately a plain executable rather than an app bundle with resources copied alongside a compiled `default.metallib`, there is no bundle-relative resource path to resolve against. The build injects the shader source directory itself instead, so `ImageDisplayView` can `fopen`/read `Blit.metal`'s text directly from the source tree at runtime:

```
# CMakeLists.txt:53-59
# ImageDisplayView compiles Shaders/Blit.metal from source at runtime (newLibrary(), not
# newDefaultLibrary()) since this is deliberately a plain executable, not an app bundle
with a
# compiled default.metallib resource — see ImageDisplayView.cpp's readShaderSource()
comment.
# Blit.metal has no local #includes, so a raw source string is fine for it.
target_compile_definitions(RayTracerBench PRIVATE
    RT_SHADERS_DIR="${CMAKE_SOURCE_DIR}/RayTracerBench/Shaders"
)
```

The two definitions are answers to the same underlying question — "where is my shader at runtime" — but they differ in exactly the way §2 explains the two `.metal` files differ: `RT_RAYTRACER_METALLIB` names a *compiled artifact* CMake produced during the build, because Raytracer.metal needed real on-disk `#include` resolution and thus needed to be compiled ahead of time; `RT_SHADERS_DIR` names a *source directory* CMake never touches, because Blit.metal is self-contained and can be compiled from a string at the moment it's needed. Both are `${CMAKE_SOURCE_DIR}`- or `${CMAKE_BINARY_DIR}`-rooted absolute paths baked in at configure/build time, which is why they only ever make sense as compile definitions on a specific build tree, not as something meaningful to a shipped, relocated binary — consistent with this being a benchmark tool built and run in place, not a distributed app bundle.

§4. Regenerating the Xcode project from CMake, not editing it by hand

`RayTracerBench.xcodeproj` (the file actually opened by `Product > Run` and invoked by `xcodebuild`) is not a hand-built Xcode project — it is CMake's Xcode generator output, copied into the repository root:

```
cmake -S . -B <scratch-dir> -G Xcode
```

followed by copying the resulting `.xcodeproj` over the tracked one. This is a direct consequence of how this project is developed: project creation here happens headlessly, with no interactive Xcode session available to click through "New Project" wizards, add target membership, or wire up framework search paths by hand in a GUI inspector. CMake's Xcode generator produces the exact same effect — an `.xcodeproj` with the right targets, sources, include paths, compile definitions, and linked frameworks — purely from this text file, which is something a headless session can actually run.

The operational consequence is a rule, not just a fact about how the file was first created: **`CMakeLists.txt` is the source of truth.** After adding, removing, or moving a source file, or after changing an include path, a compile definition, or a linked framework, the correct next step is to regenerate the `.xcodeproj` from `CMakeLists.txt` again (the same `cmake -S . -B <scratch-dir> -G Xcode` invocation, then re-copy the result over the tracked copy) — never to open the tracked `.xcodeproj` and add the change by hand in Xcode's file inspector or build-phase editor. A hand-edit would work until the next regeneration silently discards it, and in the meantime the tracked project would describe a build that `CMakeLists.txt` itself doesn't produce, which is exactly the kind of drift a single source-of-truth file exists to prevent. This is the sort of rule a maintainer needs to be told once, in writing, and then never violate by instinct — which is why it belongs in this chapter rather than left to be rediscovered the first time a hand-edited build phase mysteriously disappears.

§5. The vendored `ThirdParty` include paths

`RayTracerCore`'s own include directories (`RayTracerBench/Core`, `RayTracerBench/CPU`, `RayTracerBench/Export`, from §1) are marked `PUBLIC`, which is what lets `RayTracerBench` reach `Core/Scene.hpp` and friends by a bare `#include "Scene.hpp"` merely by linking against `RayTracerCore` — CMake propagates `PUBLIC` include directories to anything that links the library, without `RayTracerBench` having to repeat them itself:

```
# CMakeLists.txt:89
target_link_libraries(RayTracerBench PRIVATE RayTracerCore)
```

The vendored AppKit/Metal bindings need their own, separate include paths, since they are headers-only and never compiled into a library of their own. They are wired directly onto the app target:

```
# CMakeLists.txt:48-51
target_include_directories(RayTracerBench PRIVATE
    ThirdParty/metal-cpp
    ThirdParty/metal-cpp-extensions
)
```

`ThirdParty/metal-cpp` is Apple's own header-only C++ binding to Metal/Foundation (`<Metal/Metal.hpp>`, `<Foundation/Foundation.hpp>`, and so on); `ThirdParty/metal-cpp-extensions` is the AppKit/QuartzCore extension headers from the same `LearnMetalCPP.zip` sample bundle, plus this project's own additions filling Apple's gaps (`NSControl.hpp`, `NSButton.hpp`, `NSTextField.hpp`, `NSAlert.hpp`, `CAMetalLayer.hpp`, `NSEvent.hpp`, all covered in Chapter 10). Both directories are `PRIVATE` to `RayTracerBench` because nothing in `RayTracerCore` — the framework-free half of the program — ever needs to see a Metal or AppKit type.

`RayTracerBenchTests` needs a narrower slice of the same vendored tree. Since it is headless (no AppKit at all, per its own comment in §1) but still constructs a real `GPURenderer`, it only needs `metal-cpp`'s Metal bindings, plus `RayTracerBench/GPU` so it can `#include "GPURenderer.hpp"` directly (it compiles `GPURenderer.cpp` straight into itself rather than linking against the app target, which isn't a library):

```
# CMakeLists.txt:121-124
target_include_directories(RayTracerBenchTests PRIVATE
    ThirdParty/metal-cpp
    RayTracerBench/GPU
)
```

Note what's absent here relative to `RayTracerBench`'s include list: `ThirdParty/metal-cpp-extensions` never appears for the test target, which is a direct, load-bearing consequence of the "no XCTest, no Objective-C, headless" design called out in §1 — a test binary that never touches AppKit has no reason to see the AppKit extension headers at all.

§6. Frameworks linked, and why ImageWriter's two extra ones only reach the app target

`RayTracerBench` links the frameworks the whole app needs across UI, rendering, and export:

```
# CMakeLists.txt:91–108
find_library(COCOA_FRAMEWORK Cocoa REQUIRED)
find_library(METAL_FRAMEWORK Metal REQUIRED)
find_library(METALKIT_FRAMEWORK MetalKit REQUIRED)
find_library(QUARTZCORE_FRAMEWORK QuartzCore REQUIRED)
find_library(FOUNDATION_FRAMEWORK Foundation REQUIRED)
# ImageWriter.cpp writes the scene-export preview PNG via CoreGraphics/ImageIO's plain C
APIs —
# real system frameworks, not a vendored image library, and no Objective-C needed.
find_library(COREGRAPHICS_FRAMEWORK CoreGraphics REQUIRED)
find_library(IMAGEIO_FRAMEWORK ImageIO REQUIRED)

target_link_libraries(RayTracerBench PRIVATE
    ${COCOA_FRAMEWORK}
    ${METAL_FRAMEWORK}
    ${METALKIT_FRAMEWORK}
    ${QUARTZCORE_FRAMEWORK}
    ${COREGRAPHICS_FRAMEWORK}
    ${IMAGEIO_FRAMEWORK}
    ${FOUNDATION_FRAMEWORK}
)
```

`Cocoa`, `Metal`, `MetalKit`, `QuartzCore`, and `Foundation` cover the UI layer (AppKit widgets, the `CAMetalLayer`-backed `ImageDisplayView`) and the GPU renderer. `CoreGraphics` and `IMAGEIO_FRAMEWORK` are there for exactly one reason, called out in the adjacent comment: `Export/ImageWriter.cpp` (Chapter 7) rasterizes the scene-export preview PNG using CoreGraphics/ImageIO's plain C `CGImageCreate` / `CGImageDestination*` APIs, which is also precisely why `ImageWriter.cpp` was kept out of `RayTracerCore` in §1 — linking those two frameworks into the framework-free static library would have broken the one property that library exists to have. `RayTracerBenchTests`, correspondingly, links only the two frameworks it actually needs:

```
# CMakeLists.txt:132–136
target_link_libraries(RayTracerBenchTests PRIVATE
    RayTracerCore
    ${METAL_FRAMEWORK}
    ${FOUNDATION_FRAMEWORK}
)
```

`Metal` because it drives a real `GPURenderer`, `Foundation` because `metal-cpp`'s `NS::` types depend on it — and nothing else, since it never touches AppKit, CoreGraphics, ImageIO, or

QuartzCore.

§7. Build and test commands, and the one that doesn't work

The build command for the app is the ordinary `xcodebuild` invocation against the generated (§4), checked-in `.xcodeproj`:

```
xcodebuild -project RayTracerBench.xcodeproj -scheme RayTracerBench -configuration Debug
build
```

The equivalent command for the test target looks like it should be `xcodebuild test -scheme RayTracerBenchTests`, and it is worth stating precisely why that specific command fails, since it is the kind of thing that looks like an environment problem the first time someone hits it and is actually a structural fact about how this target is defined. From `CLAUDE.md`:

> `xcodebuild test -scheme RayTracerBenchTests` does not work — confirmed by trying it, not assumed — since `RayTracerBenchTests` is a plain command-line C++ tool (no XCTest/ObjC), so its CMake-generated scheme has no Test action configured; `xcodebuild test` fails with "Scheme ... is not currently configured for the test action."

The root cause traces straight back to §1: `add_executable(RayTracerBenchTests ...)` in `CMakeLists.txt` declares an ordinary command-line executable target, the same kind of target `RayTracerBench` itself is, with no XCTest bundle, no `XCTestCase` subclasses, and nothing for CMake's Xcode generator to wire into a scheme's Test action — there is no XCTest target for `xcodebuild test` to even attempt to run. `RayTracerBenchTests/main.cpp` (Chapter 11) is instead a plain `main()` that runs `RayTraceCoreTests.cpp`, `DeterministicParityTests.cpp`, and `EntityMeshTests.cpp`'s checks directly and reports pass/fail on its own, the same as any other C++ command-line program would. The correct way to run it is therefore to build the scheme as an ordinary product and then execute the resulting binary directly, locating it first:

```
xcodebuild -project RayTracerBench.xcodeproj -scheme RayTracerBenchTests -configuration
Debug build
xcodebuild -showBuildSettings -scheme RayTracerBenchTests | grep TARGET_BUILD_DIR
```

and then invoking `<TARGET_BUILD_DIR>/RayTracerBenchTests` directly (or running it from Xcode's GUI, which amounts to the same "build, then run the plain executable" flow rather than any Test-action machinery). This is the `add_dependencies(RayTracerBenchTests`

`RaytracerMetalLib)` line (§2) doing its job in the background during that build step — the test binary needs `Raytracer.metallib` to exist before it runs, exactly as `RayTracerBench` does, since it constructs a real `GPURenderer` against the same compiled library (§3).

Where this connects

- **Chapter 3** (`Shaders/Raytracer.metal`, `Shaders/Blit.metal`) is the pair of shader sources this file treats so differently: Raytracer.metal is the one whose verbatim `#include`s of `Core/ShaderTypes.h` and `Core/RayTraceCore.h` force the real `xcrun`/`.metallib` compile step in §2, while Blit.metal's lack of local `#include`s is exactly why it can stay a runtime source string under `RT_SHADERS_DIR` (§3) instead.
- **Chapter 5** (`GPU/GPURenderer.hpp/.cpp`) is the consumer of `RT_RAYTRACER_METALLIB`: its constructor calls `_pDevice->newLibrary(NS::String::string(RT_RAYTRACER_METALLIB, ...), &pError)` on exactly the path this file's custom command produces, never a source string, for the reason spelled out in both files' comments.
- **Chapter 7** (`Export/SceneExporter.hpp/.cpp`, `Export/ImageWriter.hpp/.cpp`) is why `RayTracerCore` and `RayTracerBench` split the way they do in §1 and §6: `ImageWriter.cpp`'s CoreGraphics/ImageIO dependency is the one thing in the whole `Export/` module that isn't framework-free, so it alone is compiled into and linked against the app target rather than the shared static library.
- **Chapter 11** (`RayTracerBenchTests/*`) is the target this file defines in §1 and links in §6, and whose actual checks (RayTraceCore unit tests, deterministic CPU/GPU parity, EntityMesh winding-order verification) are what running the binary produced by §7's build/locate/run sequence reports pass or fail on.